

机2单变量线性回归

根据城市人口数量，预测开小吃店的利润 数据在ex1data1.txt里，第一列是城市人口数量，第二列是该城市小吃店利润。

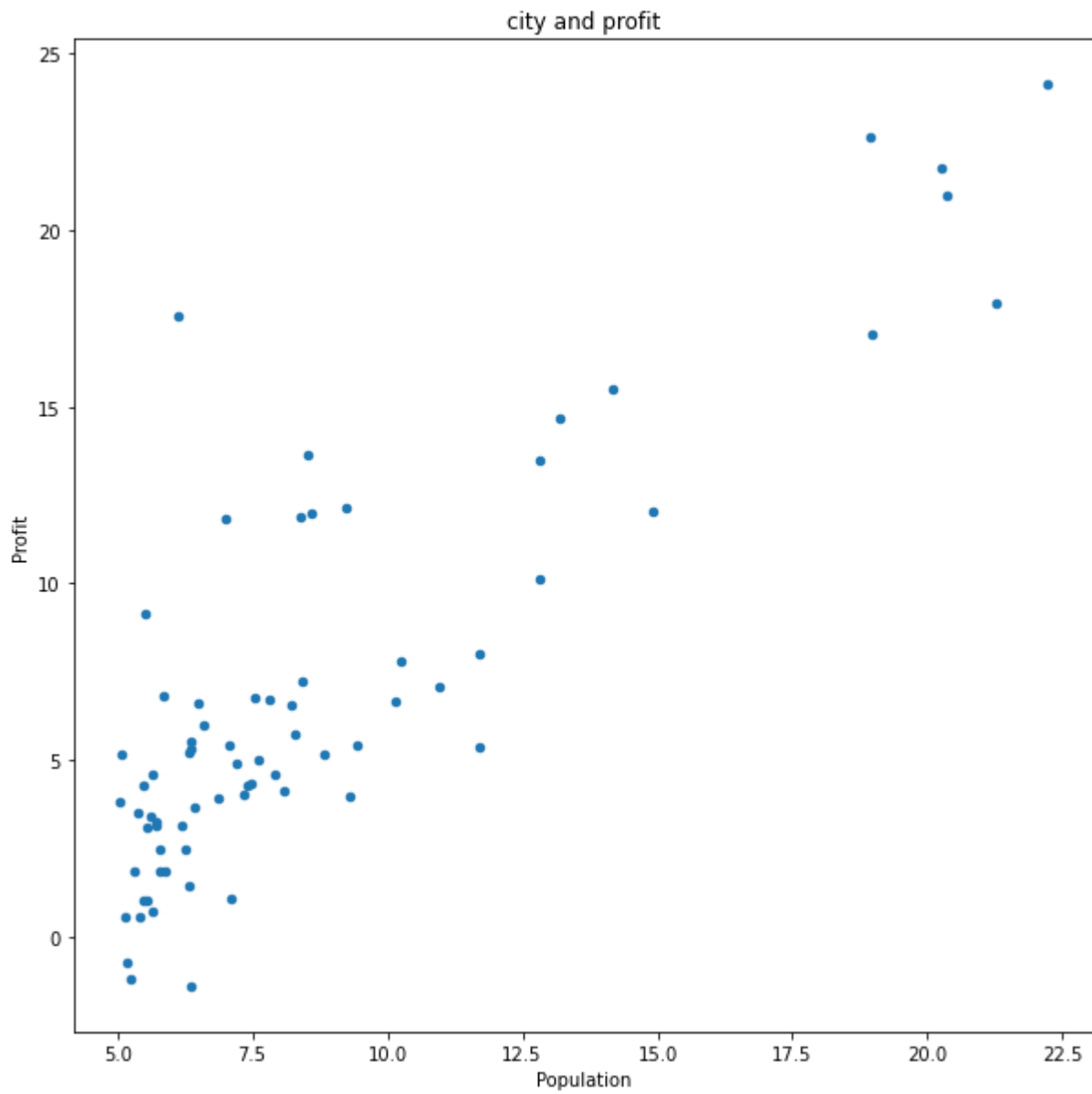
```
In [1]: import numpy as np #Numpy是使用C语言实现的一个数据计算库，它用来处理相同类型，固定长度的元素。
#使用numpy操作数据时，系统运行的速度比使用python代码快很多。numpy中还提供了很多的数据处理函数，例如傅里叶变化，矩阵操作，数据拟合等操作
import pandas as pd #Pandas 是基于NumPy 的一种工具，该工具是为了解决数据分析任务而创建的。
#Pandas纳入了大量库和一些标准的数据模型，提供了大量能使我们快速便捷地处理数据的函数和方法
import matplotlib.pyplot as plt #2D绘图库。提供一个类似matlab的绘图框架
```

```
In [2]: data = pd.read_csv('2.txt', header=None, names=['Population', 'Profit']) #header:指定标题行
data.head() #head函数可以默认读取前5行的数据
```

```
Out[2]:
```

	Population	Profit
0	6.1101	17.5920
1	5.5277	9.1302
2	8.5186	13.6620
3	7.0032	11.8540
4	5.8598	6.8233

```
In [3]: data.plot(kind='scatter',title='city and profit',x='Population',y='Profit',figsize=(10,10)) #scatter:散点图 figsize: 宽高
plt.show()
```



梯度下降

这个部分需要在现有数据集上，训练线性回归的参数 θ

```
In [4]: #定义costFunction
def computeCost(X,y,theta):
    inner=np.power((X*theta.T)-y,2) #power(x, y):计算 x 的 y 次方,theta.T:theta的转置
    return np.sum(inner)/(2*len(X)) #len:字符串长度
```

```
In [5]: data.insert(0,'Ones',1) #在data的第0列插入1，名为Ones，用于更新  $\theta_0$ 
```

```
In [6]: #变量初始化
cols=data.shape[1] #读取data列数 若shape[0]则是行数
X=data.iloc[:, :-1] #X是data里的除最后一列，前面的冒号就是取行数，后面的冒号是取列数
y=data.iloc[:, 2:] #y是data最后一列
```

```
In [7]: X.head()
```

```
Out[7]:
```

	Ones	Population
0	1	6.1101
1	1	5.5277
2	1	8.5186
3	1	7.0032
4	1	5.8598

```
In [8]: y.head()
```

```
Out[8]:
```

	Profit
0	17.5920
1	9.1302
2	13.6620
3	11.8540
4	6.8233

```
In [9]: X=np.matrix(X.values) #用于从类数组对象或数据字符串返回矩阵
y=np.matrix(y.values)
theta=np.matrix(np.array([0,0]))
```

```
In [10]: X.shape,y.shape,theta.shape,theta
```

```
Out[10]: ((72, 2), (72, 1), (1, 2), matrix([[0, 0]]))
```

```
In [11]: computeCost(X,y,theta)
```

```
Out[11]: 40.175290826974305
```

梯度下降

一个检查梯度下降是不是在正常运作的方式，是打印出每一步J()的值，看他是不是一直都在减小，并且最后收敛至一个稳定的值。最后的结果会用来预测小吃店在35000及70000人城市规模的利润

```
In [12]: # θ 更新
def gradientDescent(X,y,theta,alpha,itters): #X:人口, y: 利润, theta: [ θ 0,  θ 1]初始化为[0,0], alpha:学习率, iters: 迭代次数
    temp=np.matrix(np.zeros(theta.shape)) #返回来一个theta形状和类型的用0填充的数组
    parameters=int(theta.ravel().shape[1]) #将数组维度拉成一维数组, 取列数 (2),  θ 的数量
    cost=np.zeros(itters) #初始化

    for i in range(itters): #迭代1500次
        error=(X*theta.T)-y #每次迭代都检查J ( θ ) 的值

        for j in range(parameters):
            term=np.multiply(error,X[:,j]) #j=0时相当于都*1
            temp[0,j]=theta[0,j]-((alpha/len(X))*np.sum(term)) #len:字符串长度即m, 更新 θ

        theta=temp #确定本次迭代的 θ
        cost[i]=computeCost(X,y,theta)

    return theta,cost
```

```
In [13]: alpha=0.01
         iters=1500
```

```
In [14]: g,cost=gradientDescent(X,y,theta,alpha,itters)
         g
```

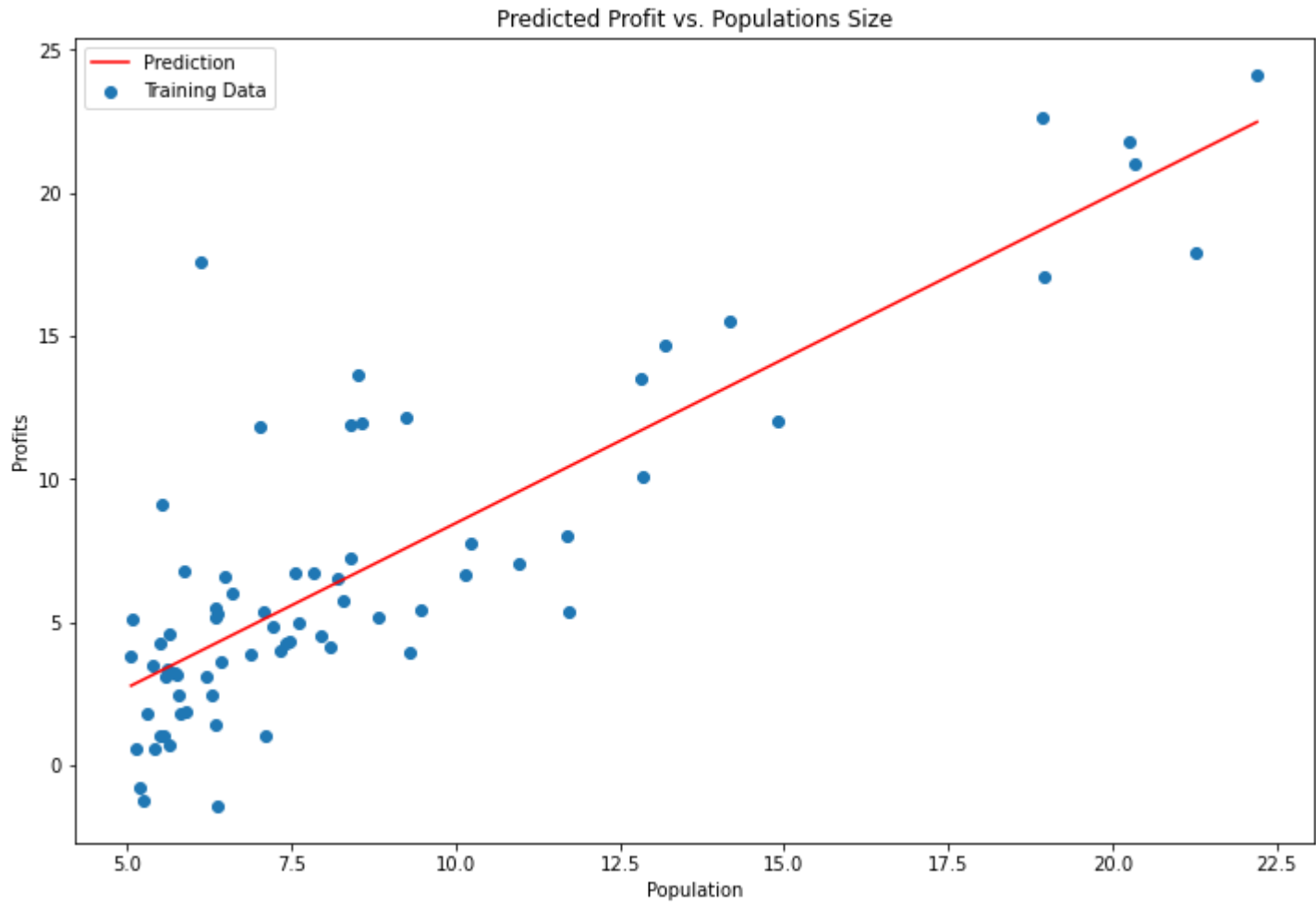
```
Out[14]: matrix([[ -3.01902296,  1.14867812]])
```

```
In [15]: predict1=[1,3.5]*g.T
         print("predict1:",predict1)
         predict2=[1,7]*g.T
         print("predict2:",predict2)
```

```
predict1: [[1.00135045]]
predict2: [[5.02172387]]
```

```
In [16]: x=np.linspace(data.Population.min(),data.Population.max(),100) #创建等差数列
         f=g[0,0]+(g[0,1]*x)
         fig,ax=plt.subplots(figsize=(12,8)) #fig:绘图窗口(Figure),ax:绘图窗口上的坐标系(axis),figsize:子图的宽度和高度
         ax.plot(x,f,'r',label='Prediction') #'r':红线, label: 定义图例
         ax.scatter(data.Population,data.Profit,label='Training Data') #散点图
         ax.legend(loc=2) #在图上标明一个图例, 用于说明每条曲线的文字显示, loc参数, 用于控制图例的所在象限
         ax.set_xlabel('Population')
         ax.set_ylabel('Profits')
         ax.set_title('Predicted Profit vs. Populations Size')
         plt.show
```

```
Out[16]: <function matplotlib.pyplot.show(close=None, block=None)>
```



机4多变量线性回归

```
In [17]: data2=pd.read_csv('4.txt',header=None,names=['Size','Bedrooms','Price'])
         data2.head()
```

	Size	Bedrooms	Price
0	2104	3	399900
1	1600	3	329900
2	2400	3	369000
3	1416	2	232000
4	3000	4	539900

特征归一化

```
In [18]: #观察数据发现, size变量是bedrooms变量的1000倍大小,统一量级会让梯度下降收敛的更快。做法就是, 将每类特征减去他的平均值后除以标准差
         data2=(data2-data2.mean())/data2.std() #std:计算标准差
         data2.head()
```

	Size	Bedrooms	Price
0	0.130010	-0.223675	0.475747
1	-0.504190	-0.223675	-0.084074
2	0.502476	-0.223675	0.228626
3	-0.735723	-1.537767	-0.867025
4	1.257476	1.090417	1.595389

```
In [19]: # 加一列常数项
```

```
data2.insert(0, 'Ones', 1)

# 初始化X和y
cols = data2.shape[1]
X2 = data2.iloc[:,0:cols-1]
y2 = data2.iloc[:,cols-1:cols]

# 转换成matrix格式，初始化theta
X2 = np.matrix(X2.values)
y2 = np.matrix(y2.values)
theta2 = np.matrix(np.array([0,0,0]))

# 运行梯度下降算法
g2, cost2 = gradientDescent(X2, y2, theta2, alpha, iters)
g2
```

Out [19]: matrix([[-1.10856950e-16, 8.84042349e-01, -5.24551809e-02]])

正规方程

```
In [20]: def normalEqn(X, y):
        theta = np.linalg.inv(X.T@X)@X.T@y #X.T@X等价于X.T.dot(X)，np.linalg.inv(): 矩阵求逆
        return theta
```

```
In [21]: final_theta2=normalEqn(X, y)#这里用的是data1的数据
        final_theta2
```

Out [21]: matrix([[-3.20203277],
 [1.16586523]])

6.逻辑回归

在训练的初始阶段，我们将要构建一个逻辑回归模型来预测，某个学生是否被大学录取。设想你是大学相关部分的管理者，想通过申请学生两次测试的评分，来决定他们是否被录取。现在你拥有之前申请学生的可以用于训练逻辑回归的训练样本集。对于每一个训练样本，你有他们两次测试的评分和最后是被录取的结果。

```
In [1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

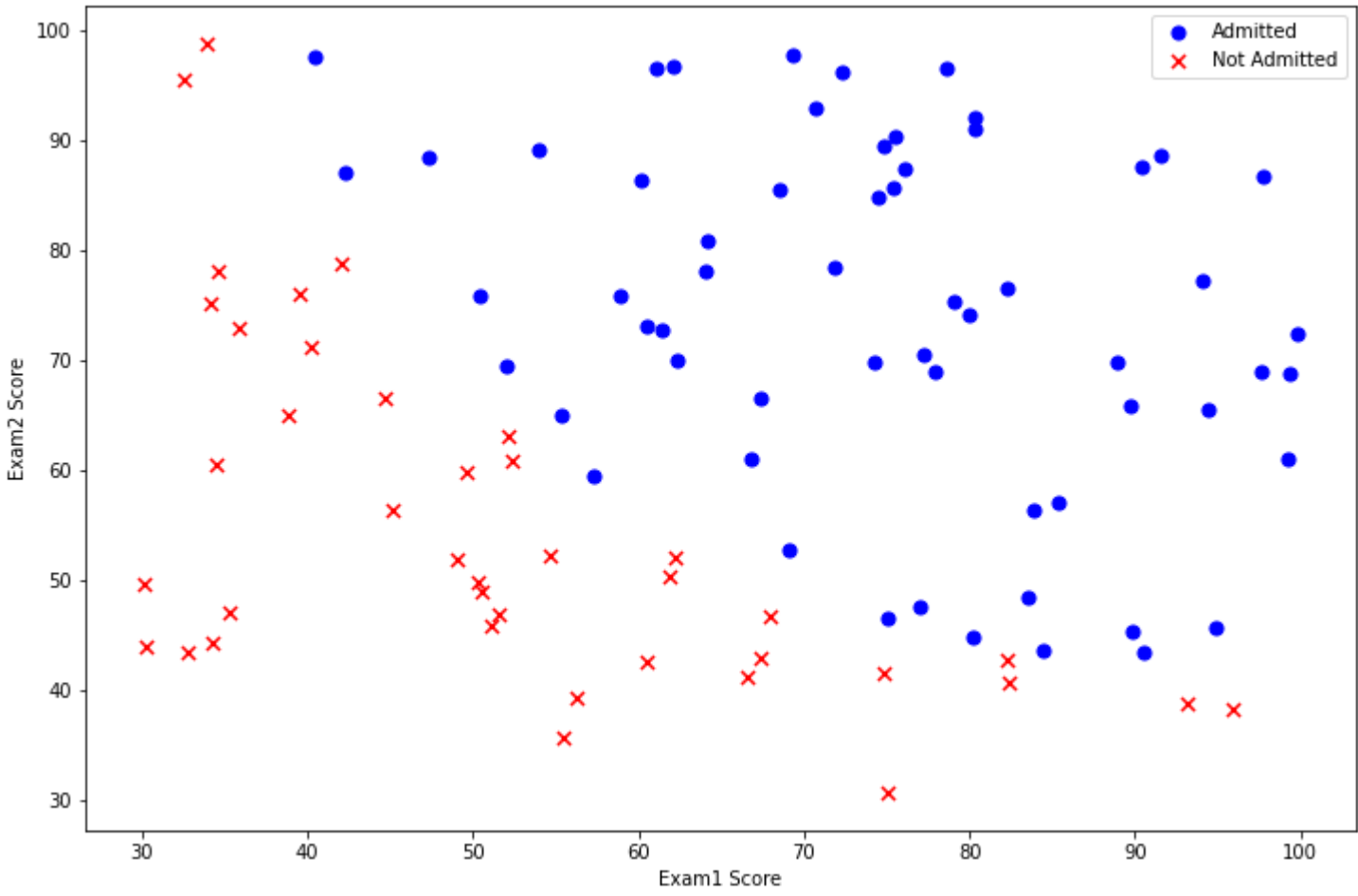
```
In [2]: data=pd.read_csv('6.1.txt',header=None,names=['Exam1','Exam2','Admitted'])
data.head()
```

Out[2]:

	Exam1	Exam2	Admitted
0	34.623660	78.024693	0
1	30.286711	43.894998	0
2	35.847409	72.902198	0
3	60.182599	86.308552	1
4	79.032736	75.344376	1

```
In [3]: positive = data[data['Admitted'].isin([1])]
negative = data[data['Admitted'].isin([0])]

fig, ax = plt.subplots(figsize=(12,8))
ax.scatter(positive['Exam1'], positive['Exam2'], s=50, c='b', marker='o', label='Admitted')
ax.scatter(negative['Exam1'], negative['Exam2'], s=50, c='r', marker='x', label='Not Admitted')
ax.legend()
ax.set_xlabel('Exam1 Score')
ax.set_ylabel('Exam2 Score')
plt.show()
```



```
In [4]: #sigmoid 函数
def sigmoid(z):
    return 1/(1 + np.exp(-z))
```

```
In [5]: #实现代价函数
def cost(theta,X,y):
    theta=np.matrix(theta)
    X=np.matrix(X)
    y=np.matrix(y)
    first=np.multiply(-y,np.log(sigmoid(X*theta.T)))
    second=np.multiply((1-y),np.log(1-sigmoid(X*theta.T)))
    return np.sum(first-second)/(len(X))
```

```
In [6]: #初始化X, y, 0

# 加一列常数列
data.insert(0, 'Ones', 1)

# 初始化X, y, 0
cols = data.shape[1]
X = data.iloc[:,0:cols-1]
y = data.iloc[:,cols-1:cols]
theta = np.zeros(3)

# 转换X, y的类型
X = np.array(X.values) #array:产生数组
y = np.array(y.values) #values:返回值
```

```
In [7]: # 检查矩阵的维度
X.shape, theta.shape, y.shape
```

Out[7]: ((100, 3), (3,), (100, 1))

```
In [8]: # 用初始 0 计算代价
cost(theta, X, y)
```

Out[8]: 0.6931471805599453

```
In [9]: # 实现梯度计算的函数（并没有更新 0）
def gradient(theta, X, y):
    theta = np.matrix(theta) #数组化
    X = np.matrix(X)
    y = np.matrix(y)

    parameters = int(theta.ravel().shape[1]) #ravel: 将多维数组转换为一维数组，参数theta的数量
    grad = np.zeros(parameters)

    error = sigmoid(X * theta.T) - y

    for i in range(parameters):
        term = np.multiply(error, X[:,i])
        grad[i] = np.sum(term) / len(X)
```

```
        return grad
```

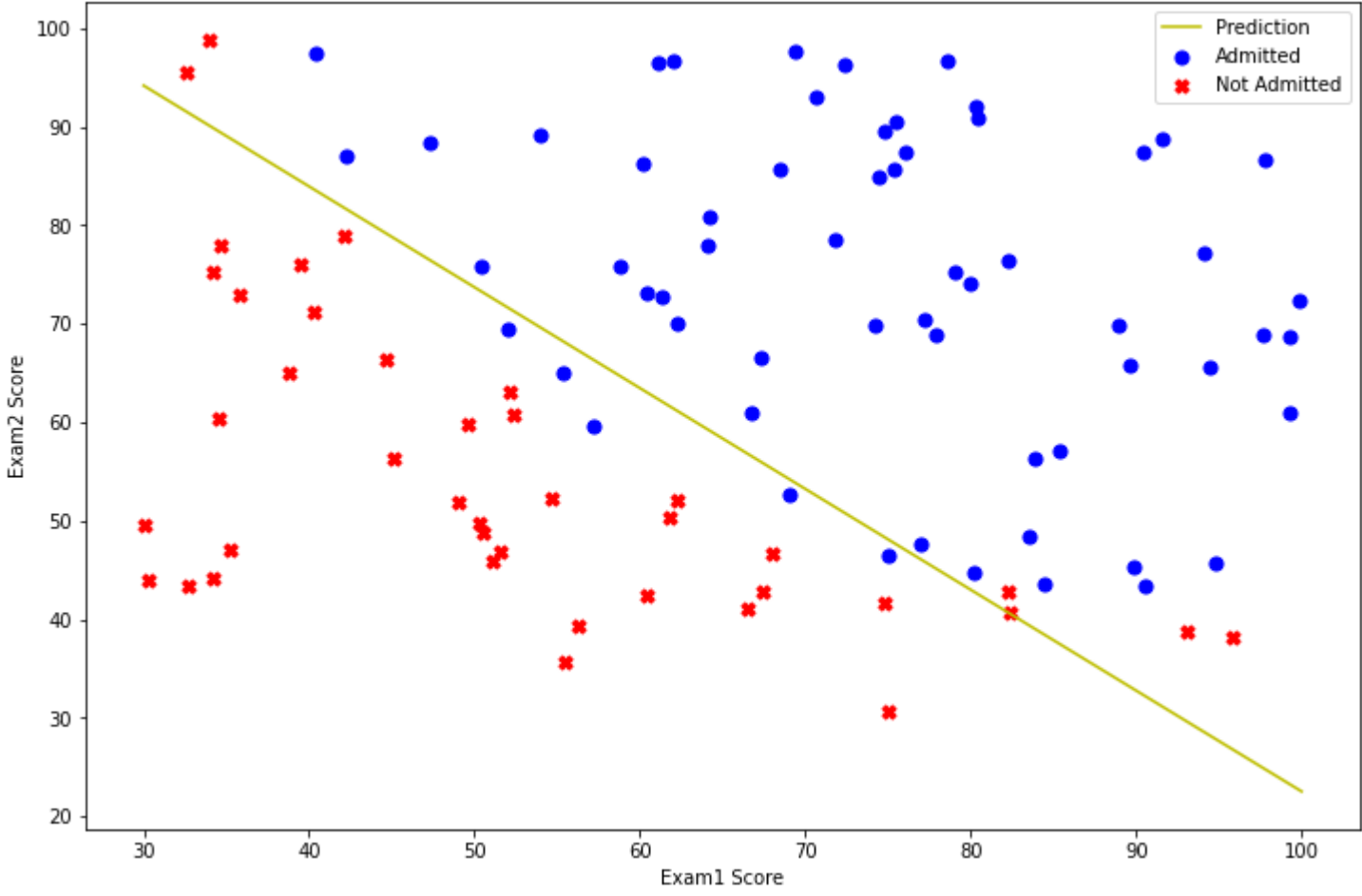
```
In [10]: import scipy.optimize as opt
result=opt.fmin_tnc(func=cost,x0=theta,fprime=gradient,args=(X,y))
#调用一个已有的库，不用自己定义迭代次数和步长，功能会直接告诉我们最优解
#参数: func:优化的目标函数，x0:初值，fprime:提供优化函数func的梯度函数，否则优化函数func必须返回函数值和梯度，或者设置approx_grad=True
#approx_grad :如果设置为True，会给出近似梯度，args: 元组，是传递给优化函数的参数
#返回: x: 0,nfeval: 整数，function evaluations的数目,在一次迭代过程中会有多次function evaluation
result
```

Out[10]: (array([-25.16131863, 0.20623159, 0.20147149]), 36, 0)

```
In [11]: # 用 0 的计算结果代回代价函数计算
cost(result[0], X, y)
```

Out[11]: 0.20349770158947458

```
In [12]: #画出决策曲线
plotting_x1=np.linspace(30,100,100) #等差数列，表示在区间[30, 100]之间取100个点作为横坐标 X
plotting_h1=(-result[0][0]-result[0][1]*plotting_x1)/result[0][2] # 0 t*X=0为决策边界(x0=1) y
#result[0]是theta3个值的一个数组，result[0][0]表示从theta的三个值中取第一个值
fig,ax=plt.subplots(figsize=(12,8)) #创建绘图空间
ax.plot(plotting_x1,plotting_h1,'y',label='Prediction') #画线
ax.scatter(positive['Exam1'], positive['Exam2'], s=50, c='b', marker='o', label='Admitted')
ax.scatter(negative['Exam1'], negative['Exam2'], s=50, c='r', marker='x', label='Not Admitted')
ax.legend()
ax.set_xlabel('Exam1 Score')
ax.set_ylabel('Exam2 Score')
plt.show()
```



```
In [13]: # 实现h 0
def hfunc1(theta, X):
    return sigmoid(np.dot(theta.T, X))
hfunc1(result[0],[1,45,85])
```

Out[13]: 0.7762906240463825

```
In [14]: # 定义预测函数
def predict(theta, X):
    probability = sigmoid(X * theta.T) #dot是乘法，*是点乘
    return [1 if x >= 0.5 else 0 for x in probability]
```

```
In [15]: # 统计预测正确率
theta_min = np.matrix(result[0]) #取 0 数组
predictions = predict(theta_min, X)
correct = [1 if ((a == 1 and b == 1) or (a == 0 and b == 0)) else 0 for (a, b) in zip(predictions, y)] #zip: 压缩，保留一一对应的关系
accuracy = (sum(map(int, correct)) % len(correct)) #map()会根据提供的函数对指定的顺序做映射。
print ('accuracy = {}%'.format(accuracy))

accuracy = 89%
```

7.正则化逻辑回归

在训练的第二部分，我们将实现加入正则项提升逻辑回归算法。设想你是工厂的生产主管，你有一些芯片在两次测试中的测试结果，测试结果决定是否芯片要被接受或抛弃。你有一些历史数据，帮助你构建一个逻辑回归模型。

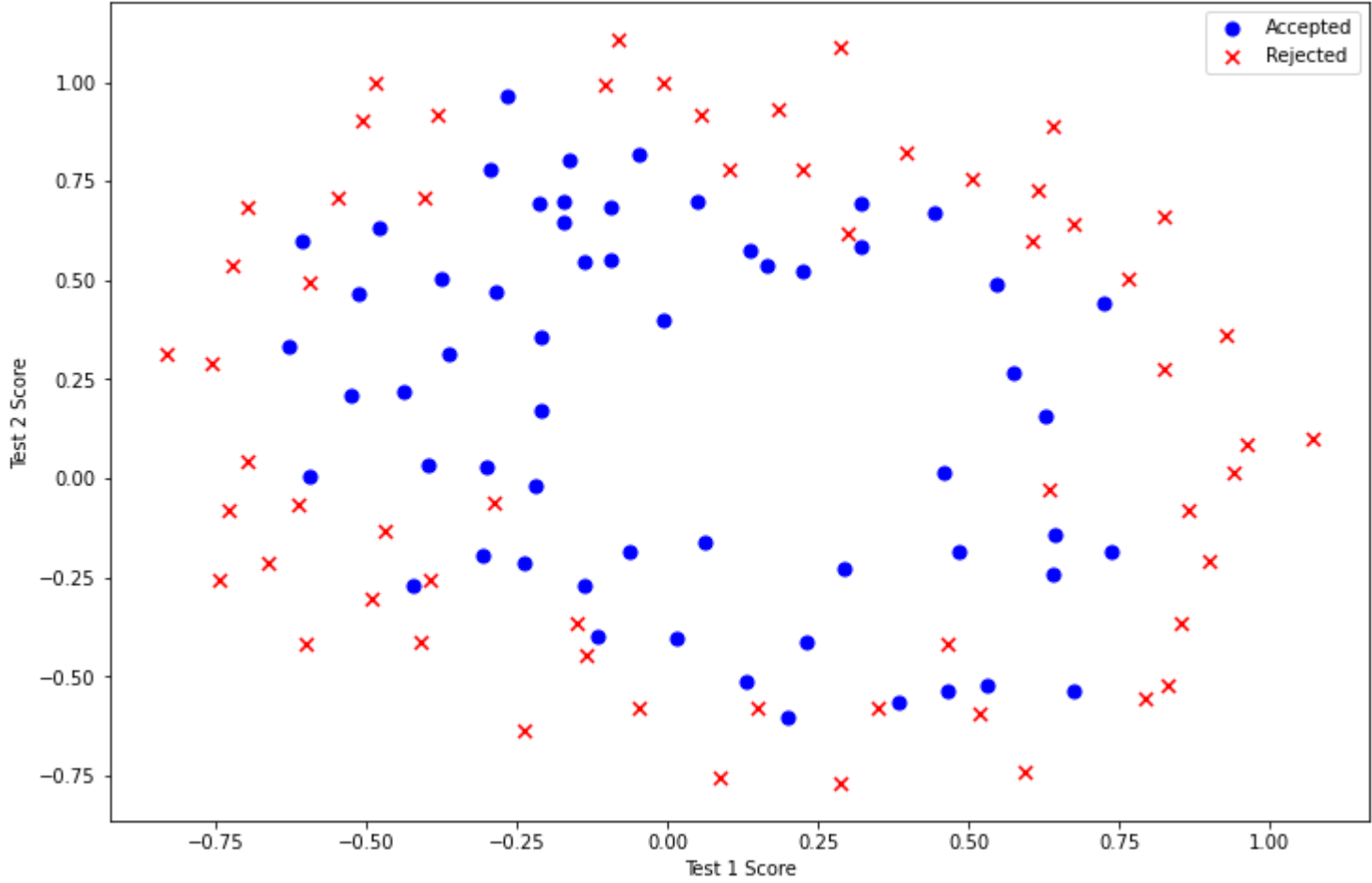
```
In [16]: data_init = pd.read_csv('6.2.txt', header=None, names=['Test 1', 'Test 2', 'Accepted'])
data_init.head()
```

Out[16]:

	Test 1	Test 2	Accepted
0	0.051267	0.69956	1
1	-0.092742	0.68494	1
2	-0.213710	0.69225	1
3	-0.375000	0.50219	1
4	-0.513250	0.46564	1

```
In [17]: positive2 = data_init[data_init['Accepted'].isin([1])]
negative2 = data_init[data_init['Accepted'].isin([0])]

fig, ax = plt.subplots(figsize=(12,8))
ax.scatter(positive2['Test 1'], positive2['Test 2'], s=50, c='b', marker='o', label='Accepted')
ax.scatter(negative2['Test 1'], negative2['Test 2'], s=50, c='r', marker='x', label='Rejected')
ax.legend()
ax.set_xlabel('Test 1 Score')
ax.set_ylabel('Test 2 Score')
plt.show()
```



特征映射

一种更好的使用数据集的方式是为每组数据创造更多的特征。所以我们为每组 x₁,x₂ 添加了最高到6次幂的特征

```
In [18]: degree = 6
data2 = data_init
x1 = data2['Test 1']
x2 = data2['Test 2']

data2.insert(3, 'Ones', 1)

for i in range(1, degree+1):
    for j in range(0, i+1):
        data2['F' + str(i-j) + str(j)] = np.power(x1, i-j) * np.power(x2, j) #power: 求幂

data2.drop('Test 1', axis=1, inplace=True) #drop:删除指定行列
data2.drop('Test 2', axis=1, inplace=True)

data2.head()
```

Out[18]:	Accepted	Ones	F10	F01	F20	F11	F02	F30	F21	F12	...	F23	F14	F05	F60	F51	F42	F33	F24	F15	F06
0	1	1	0.051267	0.69956	0.002628	0.035864	0.489384	0.000135	0.001839	0.025089	...	0.000900	0.012278	0.167542	1.815630e-08	2.477505e-07	0.000003	0.000046	0.000629	0.008589	0.117206
1	1	1	-0.092742	0.68494	0.008601	-0.063523	0.469143	-0.000798	0.005891	-0.043509	...	0.002764	-0.020412	0.150752	6.362953e-07	-4.699318e-06	0.000035	-0.000256	0.001893	-0.013981	0.103256
2	1	1	-0.213710	0.69225	0.045672	-0.147941	0.479210	-0.009761	0.031616	-0.102412	...	0.015151	-0.049077	0.158970	9.526844e-05	-3.085938e-04	0.001000	-0.003238	0.010488	-0.033973	0.110047
3	1	1	-0.375000	0.50219	0.140625	-0.188321	0.252195	-0.052734	0.070620	-0.094573	...	0.017810	-0.023851	0.031940	2.780914e-03	-3.724126e-03	0.004987	-0.006679	0.008944	-0.011978	0.016040
4	1	1	-0.513250	0.46564	0.263426	-0.238990	0.216821	-0.135203	0.122661	-0.111283	...	0.026596	-0.024128	0.021890	1.827990e-02	-1.658422e-02	0.015046	-0.013650	0.012384	-0.011235	0.010193

5 rows × 29 columns

代价函数和梯度

```
In [19]: # 实现正则化的代价函数
def costReg(theta, X, y, learningRate): #learningRate: 正则化参数
    theta = np.matrix(theta)
    X = np.matrix(X)
    y = np.matrix(y)
    first = np.multiply(-y, np.log(sigmoid(X * theta.T)))
    second = np.multiply((1 - y), np.log(1 - sigmoid(X * theta.T)))
    reg = (learningRate / (2 * len(X))) * np.sum(np.power(theta[:,1:theta.shape[1]], 2)) #正则化项，不取 0 0
    #数组切片: 逗号 “,” 分隔各个维度, “:” 表示各个维度内的切片, 只有:表示取这个维度的全部值
    #X[:,1:] 第一维取下标2之前的, 即第2行之前 (0, 1两行), 列上从第一列开始取, 不要第0列
    return np.sum(first - second) / len(X) + reg
```

```
In [20]: # 实现正则化的梯度函数
def gradientReg(theta, X, y, learningRate):
    theta = np.matrix(theta)
    X = np.matrix(X)
    y = np.matrix(y)

    parameters = int(theta.ravel().shape[1]) #shape[1]:theta的列数, 即参数 0 的数量
    grad = np.zeros(parameters)

    error = sigmoid(X * theta.T) - y

    for i in range(parameters):
        term = np.multiply(error, X[:,i])

        if (i == 0):
            grad[i] = np.sum(term) / len(X) # 0 0不正则化
        else:
            grad[i] = (np.sum(term) / len(X)) + ((learningRate / len(X)) * theta[:,i]) #正则化梯度下降

    return grad
```

```
In [21]: # 初始化X, y, 0
cols = data2.shape[1]
X2 = data2.iloc[:,1:cols]
y2 = data2.iloc[:,0:1]
theta2 = np.zeros(cols-1)

# 进行类型转换
X2 = np.array(X2.values)
y2 = np.array(y2.values)

# λ 设为1
learningRate = 1
```

```
In [22]: # 计算初始代价
costReg(theta2, X2, y2, learningRate)
```

Out[22]: 0.6931471805599454

```
In [23]: result2 = opt.fmin_tnc(func=costReg, x0=theta2, fprime=gradientReg, args=(X2, y2, learningRate))
result2
```

Out[23]: (array([1.27271027, 0.62529965, 1.18111686, -2.01987399, -0.9174319 ,
-1.43166929, 0.12393227, -0.36553118, -0.35725403, -0.17516292,
-1.45817009, -0.05098418, -0.61558552, -0.27469165, -1.19271298,
-0.24217841, -0.20603297, -0.04466178, -0.27778952, -0.29539513,
-0.45645981, -1.04319155, 0.02779373, -0.29244872, 0.01555761,

```
-0.32742406, -0.1438915 , -0.92467487]],  
32,  
1)
```

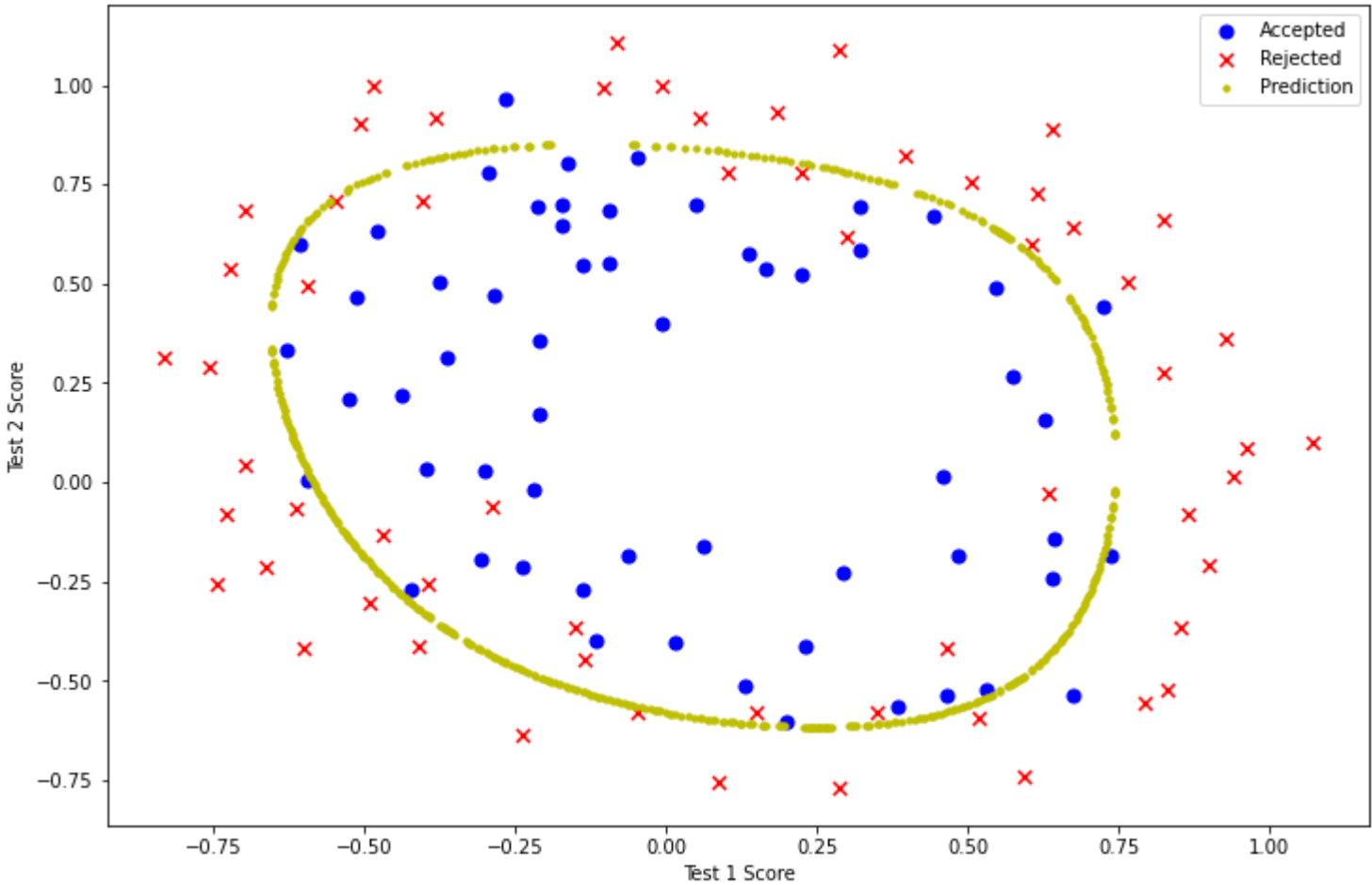
```
In [24]: #预测函数查看在训练数据上的准确度  
theta_min = np.matrix(result2[0])  
predictions = predict(theta_min, X2)  
correct = [1 if ((a == 1 and b == 1) or (a == 0 and b == 0)) else 0 for (a, b) in zip(predictions, y2)]  
accuracy = (sum(map(int, correct)) % len(correct))  
print ('accuracy = {0}%'.format(accuracy))
```

accuracy = 98%

```
In [25]: #画出决策的曲线  
def hfunc2(theta, x1, x2):  
    temp = theta[0][0]  
    place = 0  
    for i in range(1, degree+1):  
        for j in range(0, i+1):  
            temp+= np.power(x1, i-j) * np.power(x2, j)* theta[0][place+1] #假设函数  
            place+=1  
    return temp
```

```
In [26]: #找决策边界  
def find_decision_boundary(theta):  
    t1 = np.linspace(-1, 1.5, 1000) #在指定的间隔内返回均匀间隔的数字 1000为样本数  
    t2 = np.linspace(-1, 1.5, 1000)  
  
    coordinates = [(x, y) for x in t1 for y in t2]  
    x_cord, y_cord = zip(*coordinates) #zip(*):将原来的行的列表转换为列的列表; zip():将两个不同序列的元素以元组形式一一配对  
    h_val = pd.DataFrame({'x1':x_cord, 'x2':y_cord}) #DataFrame是由多种类型的列构成的二维标签数据结构  
    h_val['hval'] = hfunc2(theta, h_val['x1'], h_val['x2']) #代入假设函数  
  
    decision = h_val[np.abs(h_val['hval'])< 2 * 10**-3 ] #abs(): 函数返回数字的绝对值; ** : 乘方 ; 2 * 10**-1: 边界宽度(描点)  
    return decision.x1, decision.x2
```

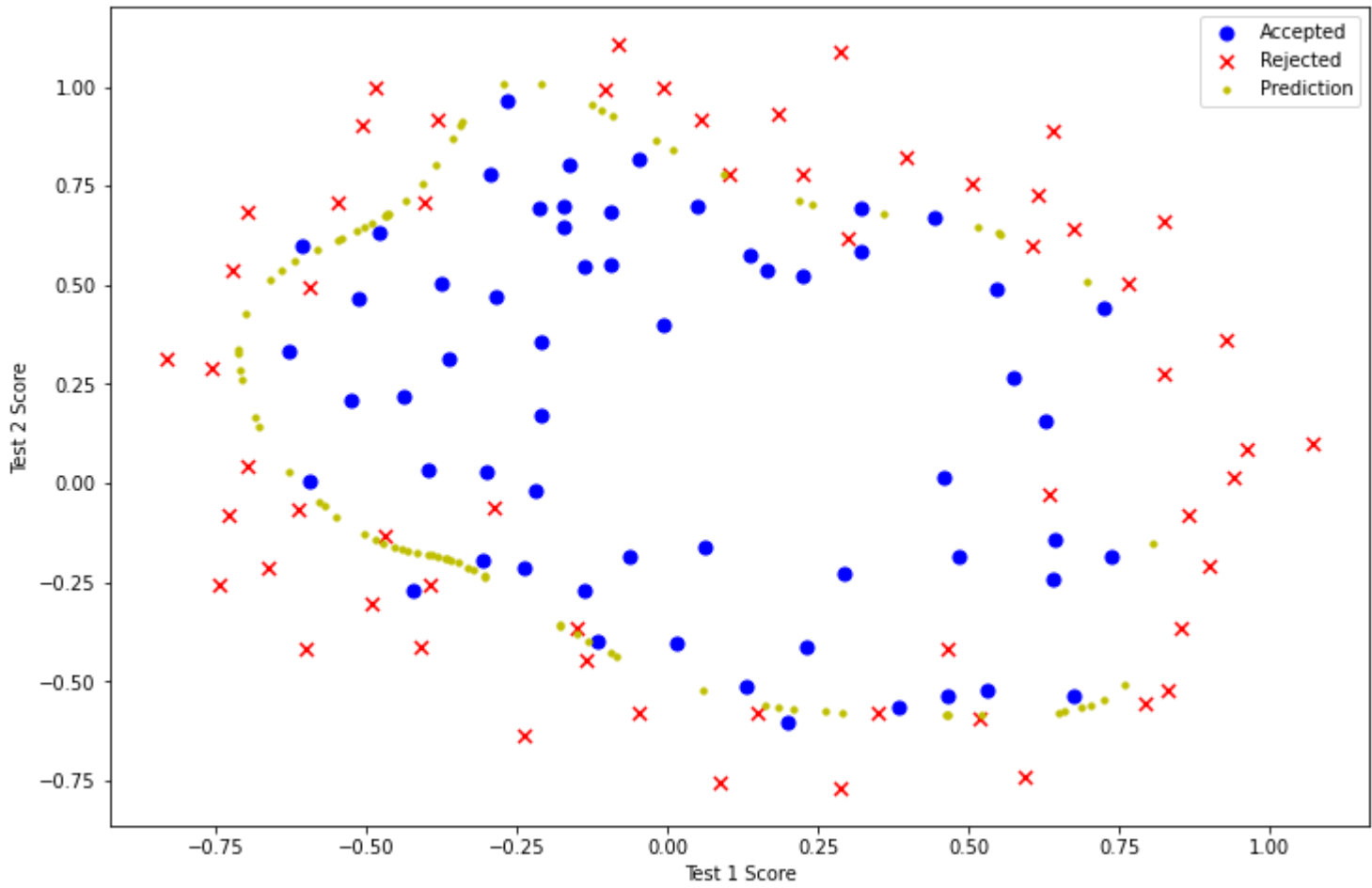
```
In [27]: fig, ax = plt.subplots(figsize=(12,8))  
ax.scatter(positive2['Test 1'], positive2['Test 2'], s=50, c='b', marker='o', label='Accepted')  
ax.scatter(negative2['Test 1'], negative2['Test 2'], s=50, c='r', marker='x', label='Rejected')  
ax.set_xlabel('Test 1 Score')  
ax.set_ylabel('Test 2 Score')  
  
x, y = find_decision_boundary(result2) #把 0 代入  
plt.scatter(x, y, c='y', s=10, label='Prediction')  
ax.legend()  
plt.show()
```



```
In [28]: #改变 λ，观察决策曲线  
#过拟合  
learningRate2 = 0  
result3 = opt.fmin_tnc(func=costReg, x0=theta2, fprime=gradientReg, args=(X2, y2, learningRate2))  
result3
```

```
Out[28]: (array([ 9.11192364e+00,  1.18840465e+01,  6.30828094e+00, -8.39706468e+01,  
-4.48639810e+01, -3.81221435e+01, -9.42525756e+01, -8.14257602e+01,  
-4.22413355e+01, -3.52968361e+00,  2.95734207e+02,  2.51308760e+02,  
 3.64155830e+02,  1.61036970e+02,  5.70100234e+01,  1.71716716e+02,  
 2.72109672e+02,  3.12447535e+02,  1.41764016e+02,  3.22495698e+01,  
-1.75836912e-01, -3.58663811e+02, -4.82161916e+02, -7.49974915e+02,  
-5.03764307e+02, -4.80978435e+02, -1.85566236e+02, -3.83936243e+01]),  
280,  
3)
```

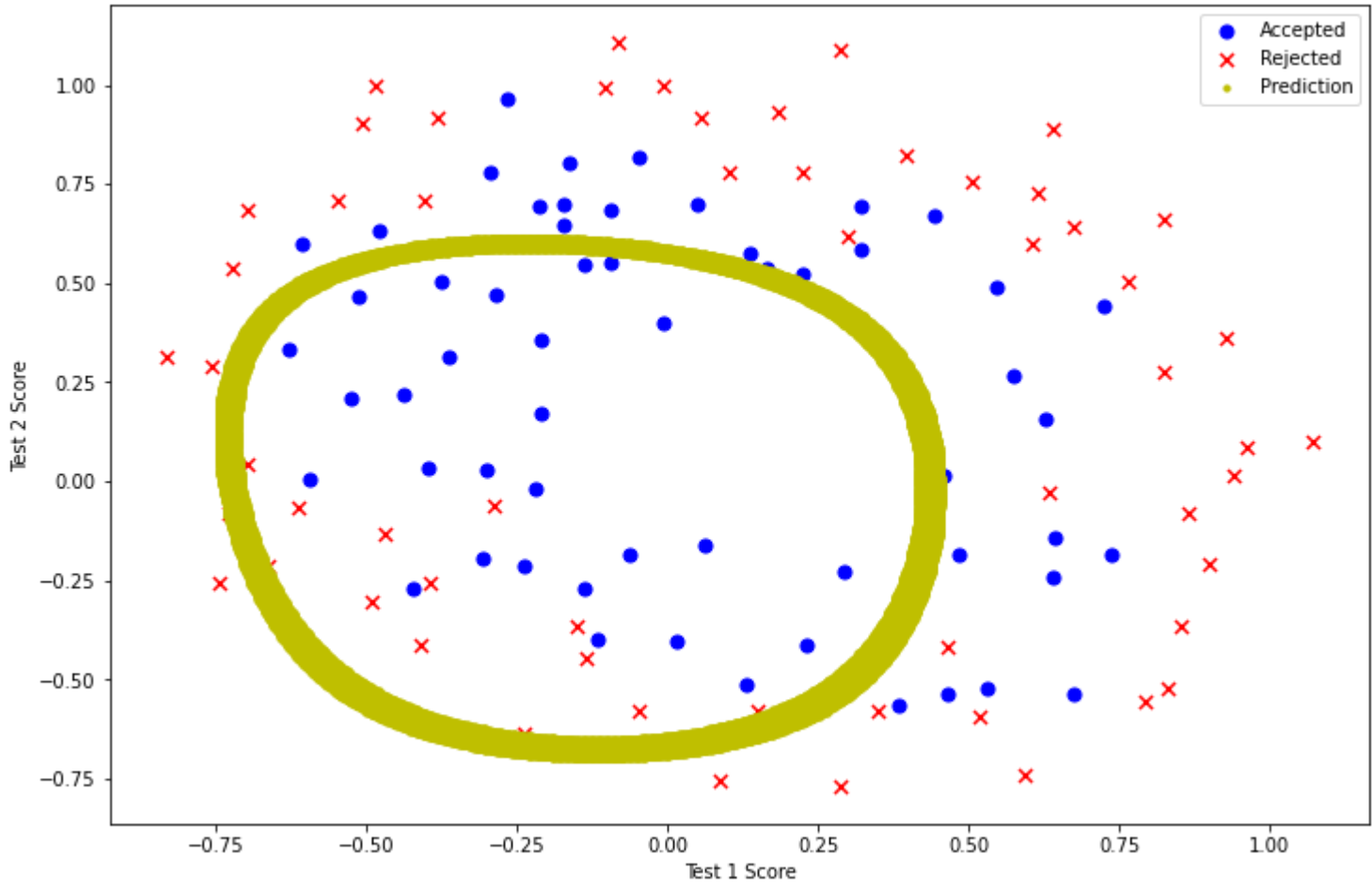
```
In [29]: fig, ax = plt.subplots(figsize=(12,8))  
ax.scatter(positive2['Test 1'], positive2['Test 2'], s=50, c='b', marker='o', label='Accepted')  
ax.scatter(negative2['Test 1'], negative2['Test 2'], s=50, c='r', marker='x', label='Rejected')  
ax.set_xlabel('Test 1 Score')  
ax.set_ylabel('Test 2 Score')  
  
x, y = find_decision_boundary(result3)  
plt.scatter(x, y, c='y', s=10, label='Prediction')  
ax.legend()  
plt.show()
```



```
In [30]: #欠拟合
learningRate3 = 100
result4 = opt.fmin_tnc(func=costReg, x0=theta2, fprime=gradientReg, args=(X2, y2, learningRate3))
```

```
In [31]: fig, ax = plt.subplots(figsize=(12,8))
ax.scatter(positive2['Test 1'], positive2['Test 2'], s=50, c='b', marker='o', label='Accepted')
ax.scatter(negative2['Test 1'], negative2['Test 2'], s=50, c='r', marker='x', label='Rejected')
ax.set_xlabel('Test 1 Score')
ax.set_ylabel('Test 2 Score')

x, y = find_decision_boundary(result4)
plt.scatter(x, y, c='y', s=10, label='Prediction')
ax.legend()
plt.show()
```



6-7 多类别分类：一对多

这个部分需要实现手写数字（0到9）的识别。需要扩展之前的逻辑回归，并将其应用于一对多的分类。

数据集是一个MATLAB格式的.m文件，其中包含5000个20*20像素的手写字体图像，以及他对应的数字。另外，数字0的y值，对应的是10

In [1]:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib
from scipy.io import loadmat #使用模块scipy.io的函数loadmat和savemat可以实现Python对mat数据的读写
from sklearn.metrics import classification_report #评价报告
```

In [2]:

```
data=loadmat(' Coursera-ML-using-matlab-python-master\ex3data1.mat')
data
```

Out[2]:

```
{'__header__': b'MATLAB 5.0 MAT-file, Platform: GLNXA64, Created on: Sun Oct 16 13:09:09 2011',
 '__version__': '1.0',
 '__globals__': [],
 'X': array([[0., 0., 0., ..., 0., 0., 0.],
            [0., 0., 0., ..., 0., 0., 0.],
            [0., 0., 0., ..., 0., 0., 0.],
            ...,
            [0., 0., 0., ..., 0., 0., 0.],
            [0., 0., 0., ..., 0., 0., 0.],
            [0., 0., 0., ..., 0., 0., 0.]],
 'y': array([[10],
            [10],
            [10],
            ...,
            [ 9],
            [ 9],
            [ 9]], dtype=uint8)}
```

In [3]:

```
data['X'].shape,data['y'].shape #5000个训练样本
```

Out[3]:

```
((5000, 400), (5000, 1))
```

In [4]:

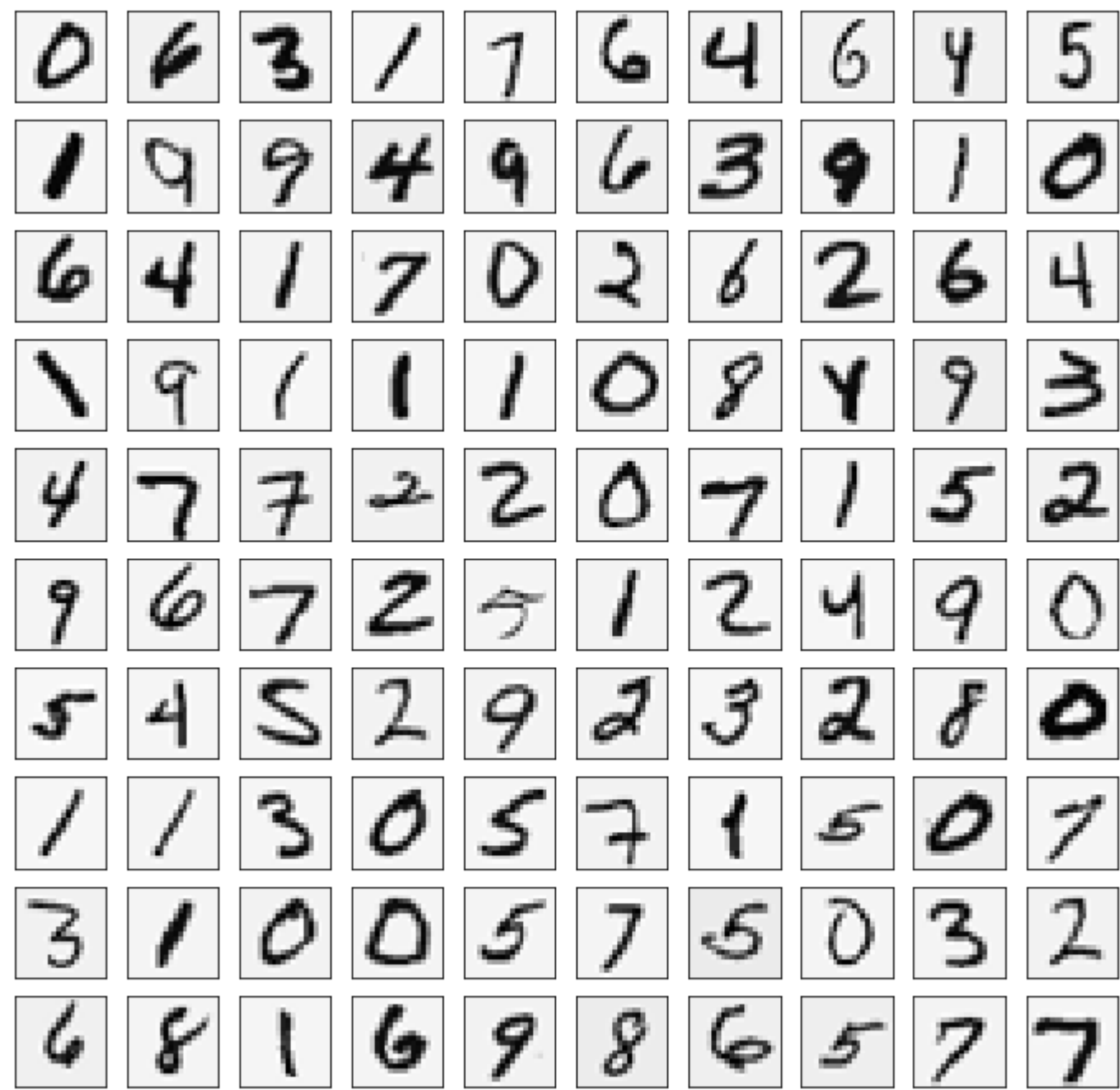
```
sample_idx = np.random.choice(np.arange(data['X'].shape[0]), 100) #随机取100行
sample_images = data['X'][sample_idx, :]
sample_images
```

Out[4]:

```
array([[0., 0., 0., ..., 0., 0., 0.],
       [0., 0., 0., ..., 0., 0., 0.],
       [0., 0., 0., ..., 0., 0., 0.],
       ...,
       [0., 0., 0., ..., 0., 0., 0.],
       [0., 0., 0., ..., 0., 0., 0.],
       [0., 0., 0., ..., 0., 0., 0.]])
```

In [5]:

```
fig,ax_array=plt.subplots(nrows=10,ncols=10,sharey=True,sharex=True,figsize=(12,12)) #设为True或 'all' 时，所有子图共享 x 轴或者 y 轴
for r in range(10):
    for c in range(10):
        ax_array[r,c].imshow(np.array(sample_images[10*r+c].reshape((20,20))).T,cmap=matplotlib.cm.binary) #取1-100的所有图片
plt.xticks(np.array([])) #x轴的刻度(tick)内容的范围
plt.yticks(np.array([])) #去除刻度，美观
```



1.3 将逻辑回归向量化

你将用多分类逻辑回归做一个分类器。因为现在有10个数字类别，所以你需要训练10个不同的逻辑回归分类器。为了让训练效率更高，将逻辑回归向量化是非常重要的，不要用循环。

In [6]:

```
def sigmoid(z):
    return 1 / (1 + np.exp(-z))
```

In [7]:

```
def cost(theta, X, y, learningRate):
    theta = np.matrix(theta)
    X = np.matrix(X)
    y = np.matrix(y)
    first = np.multiply(-y, np.log(sigmoid(X * theta.T)))
    second = np.multiply((1 - y), np.log(1 - sigmoid(X * theta.T)))
    reg = (learningRate / (2 * len(X))) * np.sum(np.power(theta[:,1:theta.shape[1]], 2)) #正则化
    return np.sum(first - second) / len(X) + reg
```

In [8]:

```
#向量化正则化逻辑回归
def gradient(theta, X, y, learningRate):
    theta = np.matrix(theta)
    X = np.matrix(X)
    y = np.matrix(y)

    parameters = int(theta.ravel().shape[1])
```

```
error = sigmoid(X * theta.T) - y

grad = ((X.T * error) / len(X)).T + ((learningRate / len(X)) * theta)
grad[0, 0] = np.sum(np.multiply(error, X[:,0])) / len(X) # 0 0不正则化

return np.array(grad).ravel()
```

一对多分类器

```
In [9]: #把分类器训练包含在一个函数中，该函数计算10个分类器中的每个分类器的最终权重，并将权重返回为k*(n + 1)数组，其中n是参数数量
from scipy.optimize import minimize #非线性优化

def one_vs_all(X, y, num_labels, learning_rate):
    rows = X.shape[0] #行数
    params = X.shape[1] #列数（参数）

    all_theta = np.zeros((num_labels, params + 1)) #返回来一个给定形状和类型的用0填充的数组；k * (n + 1)

    X = np.insert(X, 0, values=np.ones(rows), axis=1) #insert插入元素,axis选行列；ones()返回全1;在X第一列前插入全为1的列向量作常数项

    for i in range(1, num_labels + 1):
        the = np.zeros(params + 1)
        y_i = np.array([1 if label == i else 0 for label in y]) #对于y，若将某个类别i拿出来之后剩下的类别构成一类
        y_i = np.reshape(y_i, (rows,1)) #按列排放

    #最小化目标函数，fun:求最小值的目标函数；x0:变量的初始猜测值；args:常数；
    #method:求极值的方法，计算误差，反向传播的方式不同，（TNC：截断牛顿法）；jac：该参数就是计算梯度的函数
    fmin = minimize(fun=cost, x0=theta, args=(X, y_i, learning_rate), method='TNC', jac=gradient)

    all_theta[i-1,:] = fmin.x #返回使目标函数最小化的x值

    return all_theta
```

```
In [10]: rows = data['X'].shape[0]
params = data['X'].shape[1]

all_theta = np.zeros((10, params + 1))

X = np.insert(data['X'], 0, values=np.ones(rows), axis=1)

theta = np.zeros(params + 1)

y_0 = np.array([1 if label == 0 else 0 for label in data['y']])
y_0 = np.reshape(y_0, (rows, 1))

X.shape, y_0.shape, theta.shape, all_theta.shape,np.unique(data['y']) #unique去除数组中的重复数字，进行排序之后输出;检查y中的类标签
```

```
Out[10]: ((5000, 401),
(5000, 1),
(401,),
(10, 401),
array([ 1,  2,  3,  4,  5,  6,  7,  8,  9, 10], dtype=uint8))
```

```
In [11]: all_theta = one_vs_all(data['X'], data['y'], 10, 1)
all_theta.shape
```

```
Out[11]: (10, 401)
```

```
In [12]: def predict_all(X, all_theta):
    rows = X.shape[0]
    params = X.shape[1]
    num_labels = all_theta.shape[0] #10类

    X = np.insert(X, 0, values=np.ones(rows), axis=1)

    X = np.matrix(X)
    all_theta = np.matrix(all_theta)

    h = sigmoid(X * all_theta.T) #计算每个训练实例上每个类的类概率

    h_argmax = np.argmax(h, axis=1) #创建概率最大的索引数组，argmax用于返回一个数组中最大值的索引值

    h_argmax = h_argmax + 1 #因为数组是0-9索引的，所以需要加1来预测真实的标签1-10

    return h_argmax
```

```
In [13]: y_pred = predict_all(data['X'], all_theta)
print(classification_report(data['y'], y_pred))
#classification_report (y_true, y_pred) 用于显示主要分类指标的文本报告-每个类的精确度，召回率，F1值等信息，support为每个标签出现的次数
```

	precision	recall	f1-score	support
1	0.95	0.99	0.97	500
2	0.95	0.92	0.93	500
3	0.95	0.91	0.93	500
4	0.95	0.95	0.95	500
5	0.92	0.92	0.92	500
6	0.97	0.98	0.97	500
7	0.95	0.95	0.95	500
8	0.93	0.92	0.92	500
9	0.92	0.92	0.92	500
10	0.97	0.99	0.98	500
accuracy			0.94	5000
macro avg	0.94	0.94	0.94	5000
weighted avg	0.94	0.94	0.94	5000

8 神经网络

在前面一个部分，我们已经实现了多分类逻辑回归来识别手写数字。但是，逻辑回归并不能承载更复杂的假设，因为它就是个线性分类器。这部分，需要实现一个可以识别手写数字的神经网络。神经网络可以表示一些非线性复杂的模型。权重已经预先训练好，目标是在现有权重基础上，实现前馈神经网络（FNN）。

输入是图片的像素值，20*20像素的图片有400个输入层单元，不包括需要额外添加常数项。材料已经提供了训练好的神经网络的参数Θ1,Θ2，有25个隐层单元和10个输出单元（10个输出）

```
In [14]: weight = loadmat("~/Coursera-ML-using-matlab-python-master/ex3weights.mat")
theta1, theta2 = weight['Theta1'], weight['Theta2']
theta1.shape, theta2.shape #25个隐层单元和10个输出单元
```

```
Out[14]: ((25, 401), (10, 26))
```

```
In [15]: # 插入常数项
X2 = np.matrix(np.insert(data['X'], 0, values=np.ones(X.shape[0]), axis=1))
y2 = np.matrix(data['y'])
X.shape,X2.shape, y2.shape
```

```
Out[15]: ((5000, 401), (5000, 401), (5000, 1))
```

```
In [16]: a1 = X2
z2 = a1 * theta1.T
```

```
a2 = sigmoid(z2)
z2.shape, a2.shape
```

Out[16]: ((5000, 25), (5000, 25))

```
a2 = np.insert(a2, 0, values=np.ones(a2.shape[0]), axis=1)
z3 = a2 * theta2.T
z3.shape
```

Out[17]: (5000, 10)

```
a3 = sigmoid(z3)
a3
```

Out[18]: matrix([[1.12661530e-04, 1.74127856e-03, 2.52696959e-03, ...,
4.01468105e-04, 6.48072305e-03, 9.95734012e-01],
[4.79026796e-04, 2.41495958e-03, 3.44755685e-03, ...,
2.39107046e-03, 1.97025086e-03, 9.95696931e-01],
[8.85702310e-05, 3.24266731e-03, 2.55419797e-02, ...,
6.22892325e-02, 5.49803551e-03, 9.28008397e-01],
...,
[5.17641791e-02, 3.81715020e-03, 2.96297510e-02, ...,
2.15667361e-03, 6.49826950e-01, 2.42384687e-05],
[8.30631310e-04, 6.22003774e-04, 3.14518512e-04, ...,
1.19366192e-02, 9.71410499e-01, 2.06173648e-04],
[4.81465717e-05, 4.58821829e-04, 2.15146201e-05, ...,
5.73434571e-03, 6.96288990e-01, 8.18576980e-02]])

```
y_pred2 = np.argmax(a3, axis=1) + 1
y_pred2.shape
```

Out[19]: (5000, 1)

```
print(classification_report(y2, y_pred))
```

	precision	recall	f1-score	support
1	0.95	0.99	0.97	500
2	0.95	0.92	0.93	500
3	0.95	0.91	0.93	500
4	0.95	0.95	0.95	500
5	0.92	0.92	0.92	500
6	0.97	0.98	0.97	500
7	0.95	0.95	0.95	500
8	0.93	0.92	0.92	500
9	0.92	0.92	0.92	500
10	0.97	0.99	0.98	500
accuracy			0.94	5000
macro avg	0.94	0.94	0.94	5000
weighted avg	0.94	0.94	0.94	5000

1 神经网络

再次处理手写数字数据集。这次使用反向传播的前馈神经网络，自动学习神经网络的参数。

In [1]:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib
from scipy.io import loadmat #使用模块scipy.io的函数loadmat和savemat可以实现Python对mat数据的读写
from sklearn.preprocessing import OneHotEncoder
#独热编码:用于将表示分类的数据扩维,分类编码变量, 将每个类可能取值的特征变换为二进制特征向量, 每一类特征向量只有一个地方是1, 其余位置都是0
```

In [2]:

```
data=loadmat('Coursera-ML-using-matlab-python-master\ex3data1.mat')
data
```

Out[2]:

```
{'__header__': b'MATLAB 5.0 MAT-file, Platform: GLNXA64, Created on: Sun Oct 16 13:09:09 2011',
 '__version__': '1.0',
 '__globals__': [],
 'X': array([[0., 0., 0., ..., 0., 0., 0.],
            [0., 0., 0., ..., 0., 0., 0.],
            [0., 0., 0., ..., 0., 0., 0.],
            ...,
            [0., 0., 0., ..., 0., 0., 0.],
            [0., 0., 0., ..., 0., 0., 0.],
            [0., 0., 0., ..., 0., 0., 0.]]),
 'y': array([[10],
            [10],
            [10],
            ...,
            [ 9],
            [ 9],
            [ 9]], dtype=uint8)}
```

In [3]:

```
X = data['X']
y = data['y']

X.shape, y.shape #5000个训练样本
```

Out[3]:

```
((5000, 400), (5000, 1))
```

In [4]:

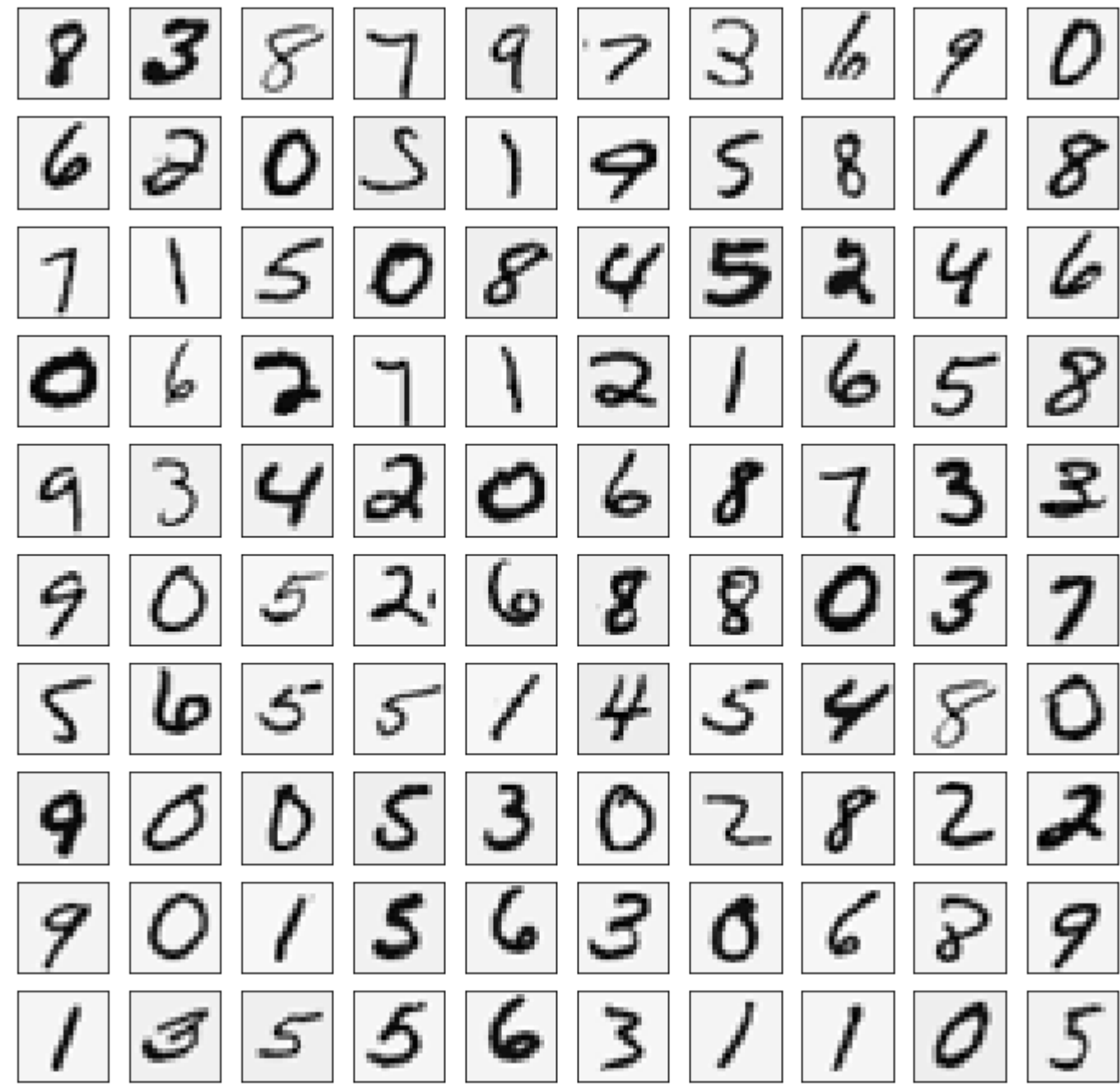
```
weight = loadmat("Coursera-ML-using-matlab-python-master\ex3weights.mat")
theta1, theta2 = weight['Theta1'], weight['Theta2']
theta1.shape, theta2.shape
```

Out[4]:

```
((25, 401), (10, 26))
```

In [5]:

```
sample_idx = np.random.choice(np.arange(data['X'].shape[0]), 100)
sample_images = data['X'][sample_idx, :]
fig,ax_array=plt.subplots(nrows=10,ncols=10,sharey=True,sharex=True,figsize=(12,12)) #设为True或 ‘all’ 时，所有子图共享 x 轴或者 y 轴
for r in range(10):
    for c in range(10):
        ax_array[r,c].imshow(np.array(sample_images[10*r+c].reshape((20,20))).T,cmap=matplotlib.cm.binary) #取1-100的所有图片
        plt.xticks(np.array([])) #x轴的刻度(tick)内容的范围
        plt.yticks(np.array([])) #去除刻度，美观
```



前向传播和代价函数

首先，实现神经网络的代价函数和梯度函数 要求：代码应该适用于任何数据集，包括任意数量的输入输出单元

In [6]:

```
def sigmoid(z):
    return 1 / (1 + np.exp(-z))
```

In [7]:

```
# 前向传播函数
def forward_propagate(X, theta1, theta2):
    m = X.shape[0]

    a1 = np.insert(X, 0, values=np.ones(m), axis=1) #注意插入零元素
    z2 = a1 * theta1.T
    a2 = np.insert(sigmoid(z2), 0, values=np.ones(m), axis=1) #注意插入零元素
    z3 = a2 * theta2.T
    h = sigmoid(z3)

    return a1, z2, a2, z3, h
```

In [8]:

```
def cost(theta1, theta2, input_size, hidden_size, num_labels, X, y, learning_rate):
# 神经网络参数，输入层维度，隐藏层维度，训练数据及标签，正则化参数

    m = len(X) #获取样本个数
    X = np.matrix(X)#将矩阵X,y转换为numpy型矩阵
```

```
y = np.matrix(y)

a1, z2, a2, z3, h = forward_propagate(X, thetal, theta2) #得到前向传播返回值

J = 0 #cost
for i in range(m): #遍历每个样本
    first_term = np.multiply(- y[i,:],np.log(h[i,:]))
    second_term = np.multiply((1 - y[i,:]), np.log(1 - h[i,:]))
    J += np.sum(first_term - second_term)
J = J / m

#J += sum([i*i for i in params])*lamda/(2*m)
J += (float(learning_rate) / (2 * m)) * (np.sum(np.power(thetal[:,1:], 2)) + np.sum(np.power(theta2[:,1:], 2))) #正则化部分

return J
```

```
In [9]: #独热编码:用于将表示分类的数据扩维，将每一个类可能取值的特征变换为二进制特征向量，每一类的特征向量只有一个地方是1，其余位置都是0
encoder = OneHotEncoder(sparse=False)
y_onehot = encoder.fit_transform(y)
y_onehot.shape,y[0], y_onehot[0,:] # y0是数字0
```

```
Out[9]: ((5000, 10),
array([10], dtype=uint8),
array([0., 0., 0., 0., 0., 0., 0., 0., 0., 1.]))
```

```
In [10]: # 初始化设置
input_size = 400
hidden_size = 25
num_labels = 10
learning_rate = 1
cost(thetal, theta2, input_size, hidden_size, num_labels, X, y_onehot, learning_rate)
```

```
Out[10]: 0.38376985909092354
```

```
In [11]: # 初始化设置
input_size = 400
hidden_size = 25
num_labels = 10
lamda = 1

# 随机初始化完整网络参数大小的参数数组
params = (np.random.random(size=hidden_size * (input_size + 1) + num_labels * (hidden_size + 1)) - 0.5) * 0.25

m = X.shape[0]
X = np.matrix(X)
y = np.matrix(y)

# 将参数数组解开为每个层的参数矩阵
thetal = np.matrix(np.reshape(params[:hidden_size * (input_size + 1)], (hidden_size, (input_size + 1))))
theta2 = np.matrix(np.reshape(params[hidden_size * (input_size + 1):], (num_labels, (hidden_size + 1))))

params.shape,thetal.shape, theta2.shape,X.shape,y.shape
```

```
Out[11]: ((10285,), (25, 401), (10, 26), (5000, 400), (5000, 1))
```

2 反向传播

这一部分需要实现反向传播的算法，来计算神经网络代价函数的梯度。获得了梯度的数据，就可以使用工具库来计算代价函数的最小值。反向传播的步骤是，给定训练集，先计算正向传播，再对于l层的每个节点j，计算误差项 δ ，这个数据衡量这个节点对最后输出的误差“贡献”了多少。对于每个输出节点，我们可以直接计算输出值与目标值的差值，定义为 δ 。对于每个隐藏节点，需要基于现有权重及 (l+1) 层的误差，计算 δ

```
In [12]: #实现sigmoid函数的梯度
def sigmoid_gradient(z):
    return np.multiply(sigmoid(z), (1 - sigmoid(z)))
sigmoid_gradient(0)
```

```
Out[12]: 0.25
```

```
In [13]: #随机初始：当我们训练神经网络的时候，需要将设定 0 的随机初始值，此处我们设定(-0.12,0.12)，这个范围保证了参数足够小，使参数学习更高效
#np.random.random(size) 返回size大小的0-1随机浮点数
params = (np.random.random(size=hidden_size * (input_size + 1) + num_labels * (hidden_size + 1)) - 0.5) * 0.24
```

```
In [14]: def backpropReg(params, input_size, hidden_size, num_labels, X, y, learning_rate):
    m = X.shape[0]
    X = np.matrix(X)
    y = np.matrix(y)

    # 从params中获取神经网络参数，并按照输入层维度和隐藏层维度重新定义参数的维度
    thetal = np.matrix(np.reshape(params[:hidden_size * (input_size + 1)], (hidden_size, (input_size + 1))))
    theta2 = np.matrix(np.reshape(params[hidden_size * (input_size + 1):], (num_labels, (hidden_size + 1))))

    # 前向传播
    a1, z2, a2, z3, h = forward_propagate(X, thetal, theta2)

    # 初始化
    J = 0
    delta1 = np.zeros(thetal.shape) # (25, 401)
    delta2 = np.zeros(theta2.shape) # (10, 26)

    # compute the cost
    for i in range(m):
        first_term = np.multiply(-y[i,:], np.log(h[i,:]))
        second_term = np.multiply((1 - y[i,:]), np.log(1 - h[i,:]))
        J += np.sum(first_term - second_term)
    J = J / m
    J += (float(learning_rate) / (2 * m)) * (np.sum(np.power(thetal[:,1:], 2)) + np.sum(np.power(theta2[:,1:], 2))) #正则化

    # perform backpropagation
    for t in range(m):
        alt = a1[t,:] # (1, 401)
        z2t = z2[t,:] # (1, 25)
        a2t = a2[t,:] # (1, 26)
        ht = h[t,:] # (1, 10)
        yt = y[t,:] # (1, 10)

        d3t = ht - yt # (1, 10)

        z2t = np.insert(z2t, 0, values=np.ones(1)) # (1, 26)
        d2t = np.multiply((theta2.T * d3t.T).T, sigmoid_gradient(z2t)) # (1, 26)

        delta1 = delta1 + (d2t[:,1:]).T * alt
        delta2 = delta2 + d3t.T * a2t

    delta1 = delta1 / m
    delta2 = delta2 / m

    # 梯度正则化项
    delta1[:,1:] = delta1[:,1:] + (thetal[:,1:] * learning_rate) / m
    delta2[:,1:] = delta2[:,1:] + (theta2[:,1:] * learning_rate) / m

    # unravel the gradient matrices into a single array
    grad = np.concatenate((np.ravel(delta1), np.ravel(delta2)))

    return J, grad
```

```
In [15]: from scipy.optimize import minimize

# minimize the objective function
fmin = minimize(fun=backpropReg, x0=(params), args=(input_size, hidden_size, num_labels, X, y_onehot, learning_rate),
               method='TNC', jac=True, options={'maxiter': 250})

fmin
```

```
Out[15]: fun: 0.3248285273568172
        jac: array([-1.06785941e-04,  1.78374521e-07,  4.40051105e-07, ...,
        -5.84885723e-05, -1.60413813e-05, -4.53168661e-05])
        message: 'Max. number of function evaluations reached'
        nfev: 250
        nit: 21
        status: 3
        success: False
        x: array([-6.31228535e-01,  8.91872603e-04,  2.20025552e-03, ...,
        -2.10326076e-01, -1.50895618e+00, -1.22614118e+00])
```

```
In [16]: X = np.matrix(X)
        thetafinal1 = np.matrix(np.reshape(fmin.x[:hidden_size * (input_size + 1)], (hidden_size, (input_size + 1))))
        thetafinal2 = np.matrix(np.reshape(fmin.x[hidden_size * (input_size + 1):], (num_labels, (hidden_size + 1))))

# 计算使用优化后的 θ 得出的预测
a1, z2, a2, z3, h = forward_propagate(X, thetafinal1, thetafinal2 )
y_pred = np.array(np.argmax(h, axis=1) + 1)
y_pred
```

```
Out[16]: array([[10],
        [10],
        [10],
        ...,
        [ 9],
        [ 9],
        [ 9]], dtype=int64)
```

```
In [17]: # 预测值与实际值比较
        from sklearn.metrics import classification_report#这个包是评价报告
        print(classification_report(y, y_pred))
```

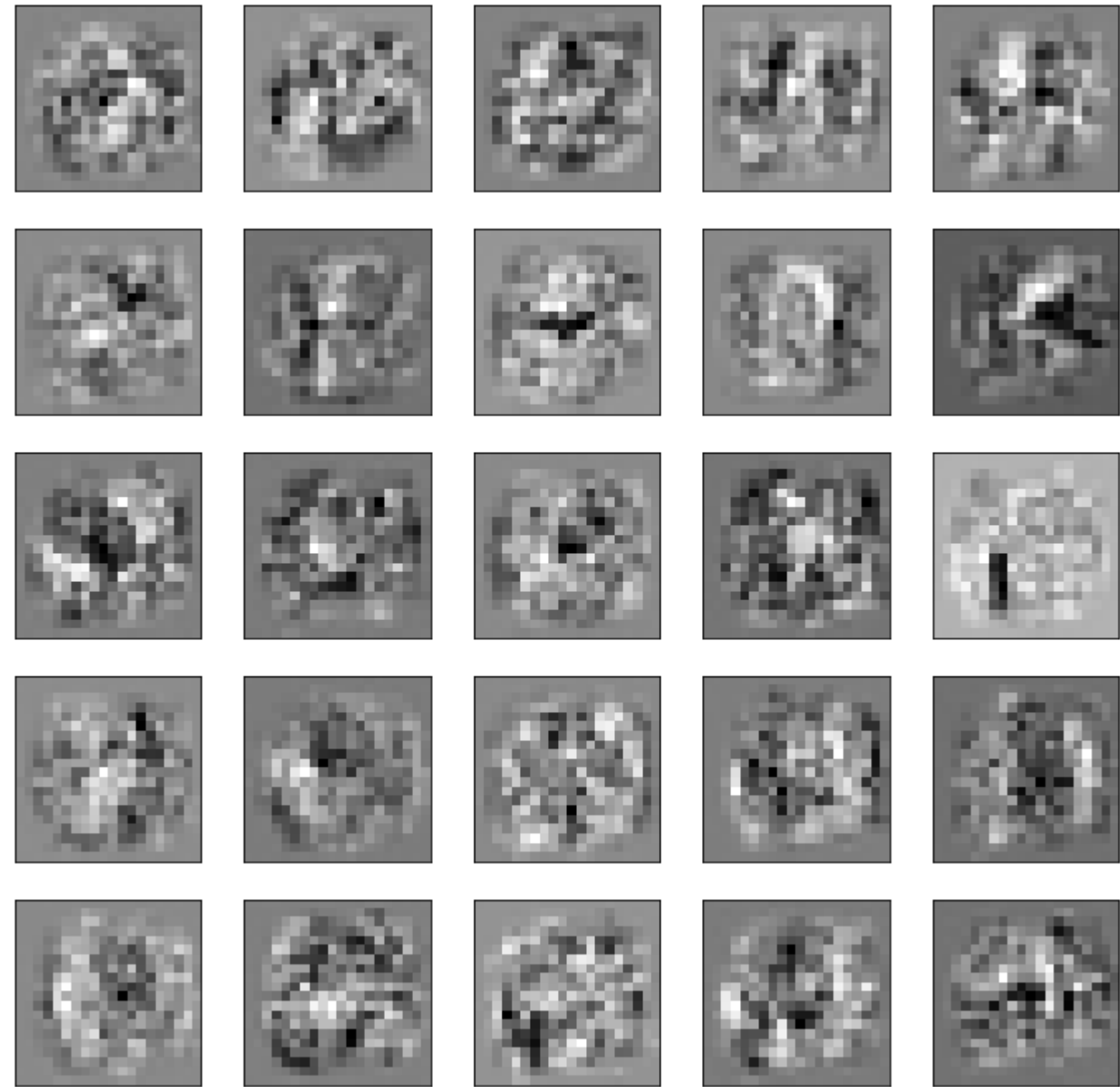
	precision	recall	f1-score	support
1	0.99	1.00	0.99	500
2	1.00	0.99	0.99	500
3	0.99	0.98	0.99	500
4	1.00	0.99	1.00	500
5	1.00	1.00	1.00	500
6	1.00	1.00	1.00	500
7	0.99	1.00	0.99	500
8	0.99	1.00	1.00	500
9	0.99	0.99	0.99	500
10	0.99	1.00	0.99	500
accuracy			0.99	5000
macro avg	0.99	0.99	0.99	5000
weighted avg	0.99	0.99	0.99	5000

3 可视化隐藏层

```
In [18]: hidden_layer = thetafinal1[:, 1:]
        hidden_layer.shape
```

```
Out[18]: (25, 400)
```

```
In [19]: fig, ax_array = plt.subplots(nrows=5, ncols=5, sharey=True, sharex=True, figsize=(12, 12))
        for r in range(5):
            for c in range(5):
                ax_array[r, c].imshow(np.array(hidden_layer[5 * r + c].reshape((20, 20))), cmap=matplotlib.cm.binary)
                plt.xticks(np.array([]))
                plt.yticks(np.array([]))
```



1 正则化线性回归

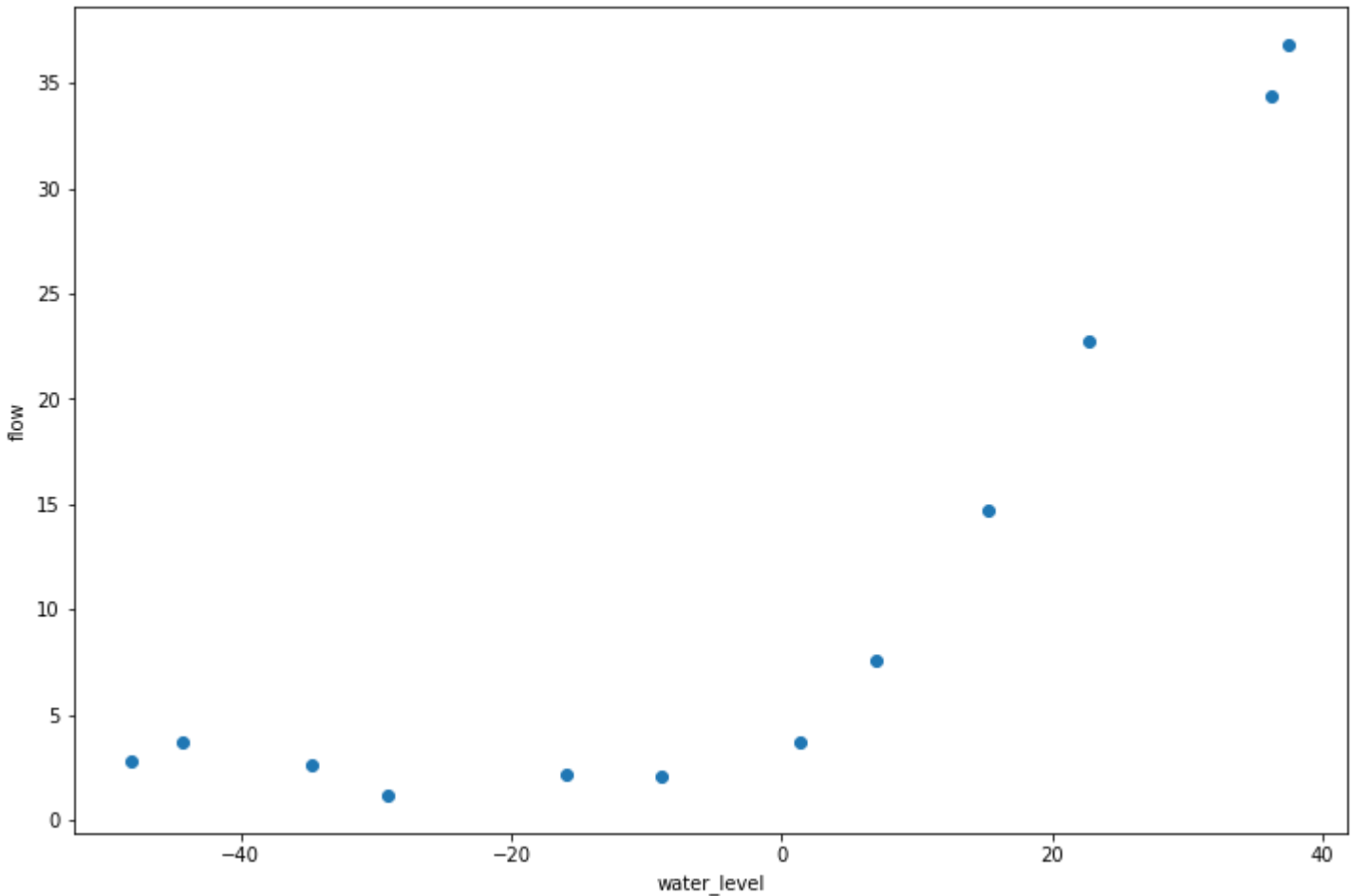
先对一个水库的流出水量以及水库水位进行正则化线性回归，然后探讨方差-偏差的问题

```
In [1]: import numpy as np
import scipy.io as sio
import scipy.optimize as opt
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns #Seaborn是一种开源的数据可视化工具, 在Matplotlib的基础上进行了更高级的API封装, 因此可进行更复杂的图形设计和输出
```

```
In [2]: data = sio.loadmat('Coursera-ML-using-matlab-python-master/ex5data1.mat')
X, y, Xval, yval, Xtest, ytest = map(np.ravel,[data['X'], data['y'], data['Xval'], data['yval'], data['Xtest'], data['ytest']])
X.shape, y.shape, Xval.shape, yval.shape, Xtest.shape, ytest.shape
```

Out[2]: ((12,), (12,), (21,), (21,), (21,), (21,))

```
In [3]: fig, ax = plt.subplots(figsize=(12,8))
ax.scatter(X, y)
ax.set_xlabel('water_level')
ax.set_ylabel('flow')
plt.show()
```



```
In [4]: X, Xval, Xtest = [np.insert(x.reshape(x.shape[0], 1), 0, np.ones(x.shape[0]), axis=1) for x in (X, Xval, Xtest)]
```

```
In [5]: def cost(theta, X, y, reg = 1):

    m = X.shape[0]
    inner = X @ theta - y          #matmul(A,C)=np.dot(A,C)=A@C都属于叉乘, 而multiply(A,C)= A*C=A@C属于点乘
    square_sum = inner.T @ inner
    cost = square_sum / (2 * m)
    reg_term = (reg / (2 * m)) * np.power(theta[1:], 2).sum()
    cost+=reg_term

    return cost
```

```
In [6]: theta = np.ones(X.shape[1])
cost(theta, X, y, 1)
```

Out[6]: 303.9931922202643

```
In [7]: def gradient(theta, X, y, reg):
    m = X.shape[0]

    inner = (1/m)*X.T @ (X @ theta - y)  # (m,n).T @ (m, 1) -> (n, 1)

    regularized_term = theta.copy()  # same shape as theta
    regularized_term[0] = 0  # don't regularize intercept theta
    regularized_term = (reg / m) * regularized_term
    inner+=regularized_term

    return inner
```

```
In [8]: gradient(theta, X, y, 1)
```

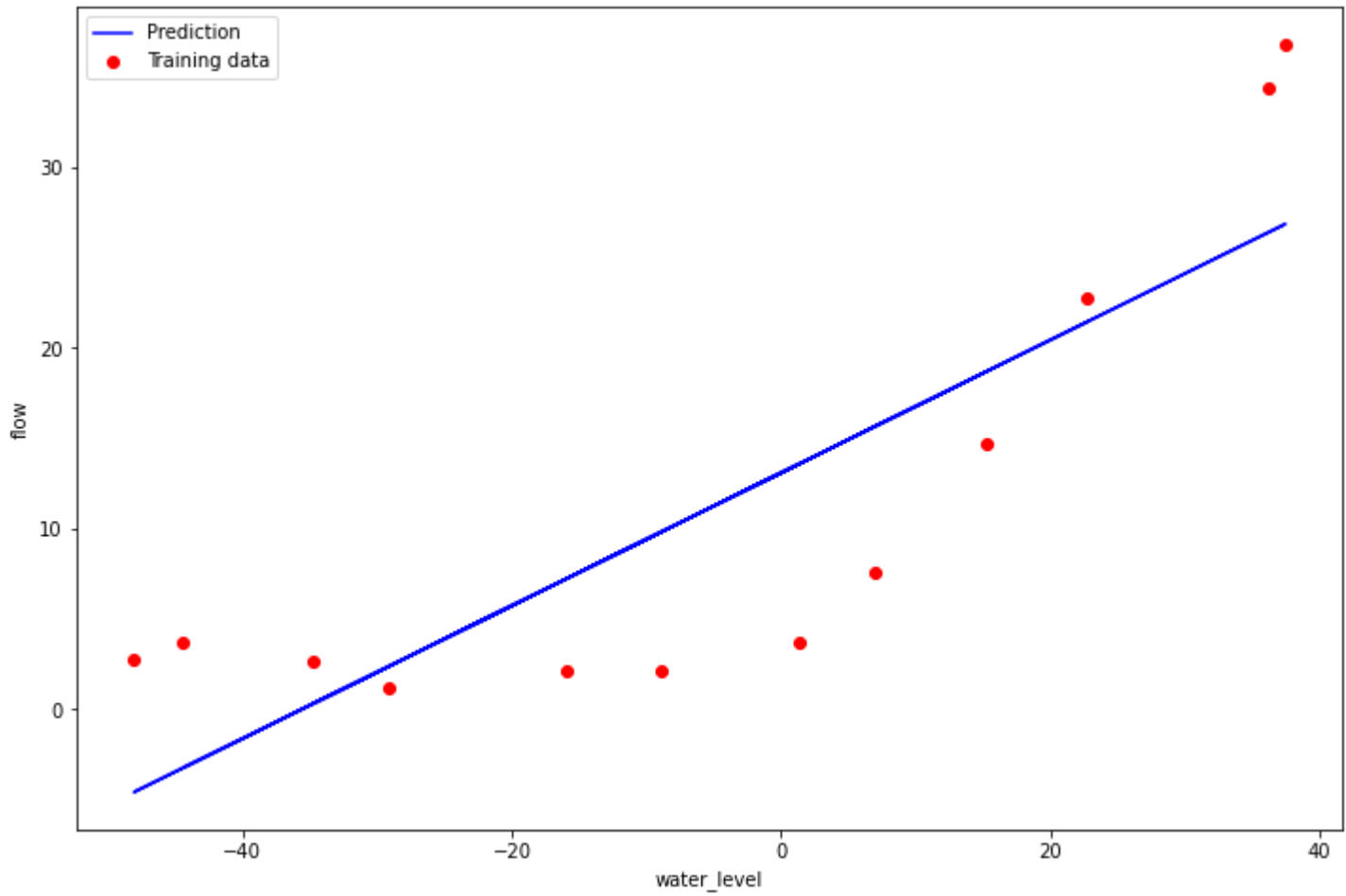
Out[8]: array([-15.30301567, 598.25074417])

```
In [9]: #令λ=0, 因为训练的是2维的, 所以正则化不会对这种低维的有很大的帮助
theta = np.ones(X.shape[1])
final_theta = opt.minimize(fun=cost, x0=theta, args=(X, y, 0), method='TNC', jac=gradient, options={'disp': True}).x
final_theta
```

Out[9]: array([13.08790352, 0.36777923])

```
In [10]: b = final_theta[0] # intercept
m = final_theta[1] # slope

fig, ax = plt.subplots(figsize=(12,8))
plt.scatter(X[:,1], y, c='r', label="Training data")
plt.plot(X[:, 1], X[:, 1]*m + b, c='b', label="Prediction")
ax.set_xlabel('water_level')
ax.set_ylabel('flow')
ax.legend()
plt.show()
```



2 方差和偏差

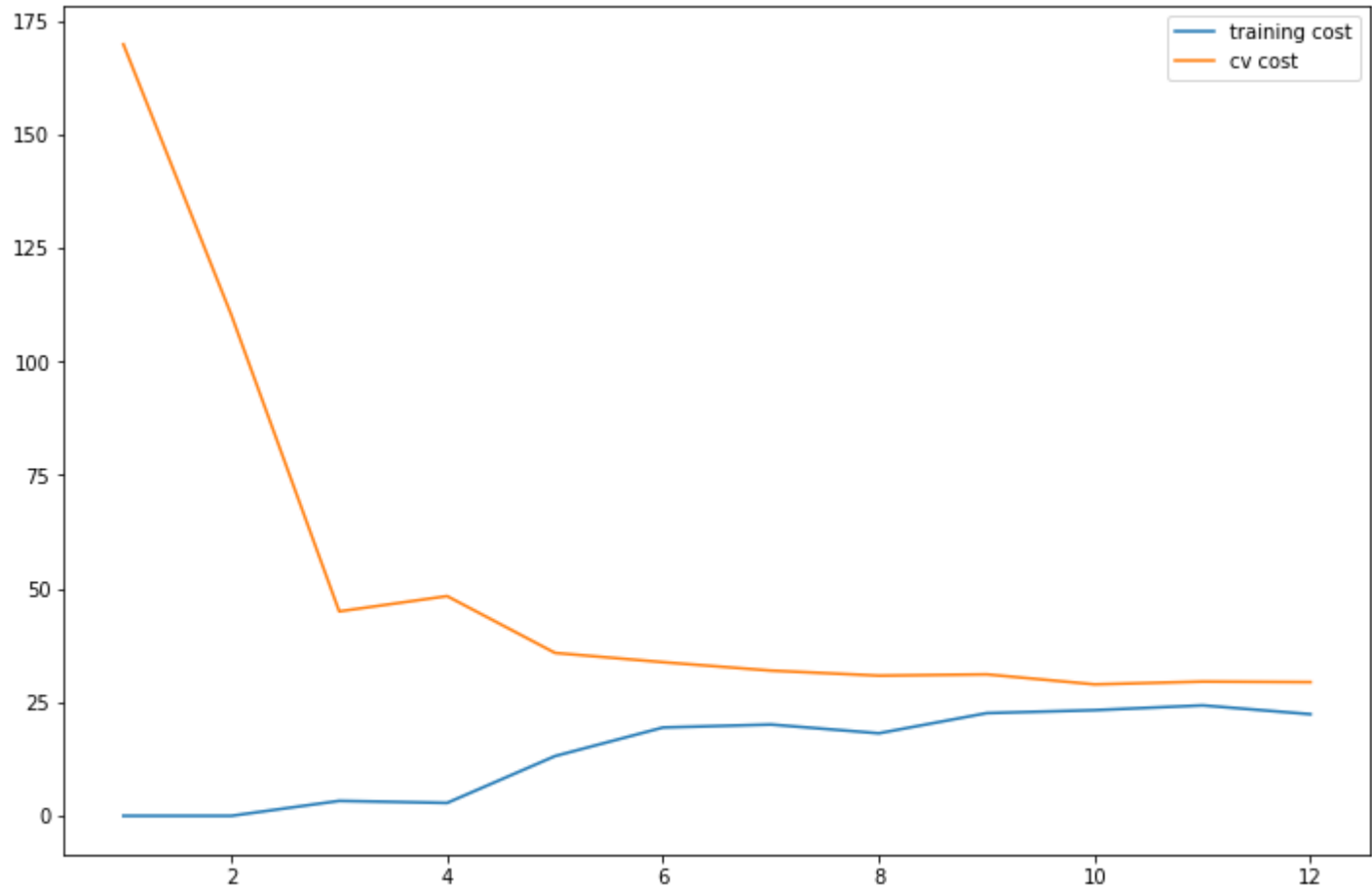
机器学习中的一个重要概念是偏差-方差权衡。偏差较大的模型会欠拟合，而方差较大的模型会过拟合。这部分要画出学习曲线来判断方差和偏差的问题。

```
In [11]: def linear_regression(X, y, l=1):  
  
    theta = np.ones(X.shape[1])  
    res = opt.minimize(fun=cost, x0=theta, args=(X, y, l), method='TNC', jac=gradient, options={'disp': True})  
    return res
```

```
In [12]: training_cost, cv_cost = [], []
```

```
In [13]: m = X.shape[0]  
for i in range(1, m+1):  
    res = linear_regression(X[:i, :], y[:i], 0)  
  
    tc = cost(res.x, X[:i, :], y[:i], 0)  
    cv = cost(res.x, Xval, yval, 0)  
  
    training_cost.append(tc) #append: 在列表末尾添加新的对象  
    cv_cost.append(cv)
```

```
In [14]: fig, ax = plt.subplots(figsize=(12,8))  
plt.plot(np.arange(1, m+1), training_cost, label='training cost')  
plt.plot(np.arange(1, m+1), cv_cost, label='cv cost')  
plt.legend()  
plt.show()  
#欠拟合
```



3 多项式回归

线性回归对于现有数据来说太简单，会欠拟合，需要多添加一些特征。

```
In [15]: #写一个函数，输入原始X，和幂的次数p，返回X的1到p次幂  
def poly_features(x, power, as_ndarray=False): #特征映射  
    data = {'f{}'.format(i): np.power(x, i) for i in range(1, power + 1)}  
    df = pd.DataFrame(data) #类似excel，是一种二维表。  
  
    return df.values if as_ndarray else df  
  
In [16]: data = sio.loadmat('Coursera-ML-using-matlab-python-master/ex5data1.mat') #读取 mat 文件  
X, y, Xval, yval, Xtest, ytest = map(np.ravel, [data['X'], data['y'], data['Xval'], data['yval'], data['Xtest'], data['ytest']])  
#ravel: 扁平化函数  
poly_features(X, power=3)
```

Out[16]:

	f1	f2	f3
0	-15.936758	253.980260	-4047.621971
1	-29.152979	849.896197	-24777.006175
2	36.189549	1309.683430	47396.852168
3	37.492187	1405.664111	52701.422173
4	-48.058829	2309.651088	-110999.127750
5	-8.941458	79.949670	-714.866612
6	15.307793	234.328523	3587.052500
7	-34.706266	1204.524887	-41804.560890
8	1.389154	1.929750	2.680720
9	-44.383760	1969.918139	-87432.373590

	f1	f2	f3
10	7.013502	49.189211	344.988637
11	22.762749	518.142738	11794.353058

```
In [17]: #归一化
def normalize_feature(df):
    return df.apply(lambda column: (column - column.mean()) / column.std())
```

```
In [18]: def prepare_poly_data(*args, power): ##args: 将不定数量的参数传递给一个函数

    """
    args: keep feeding in X, Xval, or Xtest
    单引号和双引号是单行字符串
    三引号是多行字符串，可以直接输入回车，而不需要用\n来表示
    也可以用来表示多行注释

    """

    #扩展特征
    def prepare(x):
        df = poly_features(x, power=power)
        ndarr = normalize_feature(df).values
        return np.insert(ndarr, 0, np.ones(ndarr.shape[0]), axis=1)
    return [prepare(x) for x in args]
```

```
In [19]: #扩展特征到8阶特征
X_poly, Xval_poly, Xtest_poly= prepare_poly_data(X, Xval, Xtest, power=8)
X_poly[:,3, :]
```

Out[19]: array([[1.00000000e+00, -3.62140776e-01, -7.55086688e-01,
 1.82225876e-01, -7.06189908e-01, 3.06617917e-01,
-5.90877673e-01, 3.44515797e-01, -5.08481165e-01],
 [1.00000000e+00, -8.03204845e-01, 1.25825266e-03,
-2.47936991e-01, -3.27023420e-01, 9.33963187e-02,
-4.35817606e-01, 2.55416116e-01, -4.48912493e-01],
 [1.00000000e+00, 1.37746700e+00, 5.84826715e-01,
 1.24976856e+00, 2.45311974e-01, 9.78359696e-01,
-1.21556976e-02, 7.56568484e-01, -1.70352114e-01]])

```
In [20]: def plot_learning_curve(X, Xinit, y, Xval, yval, l=0):
    training_cost, cv_cost = [], []
    m = X.shape[0]

    for i in range(1, m + 1):
        res = linear_regression(X[:i, :], y[:i], l=1)
        tc = cost(res.x, X[:i, :], y[:i])
        cv = cost(res.x, Xval, yval)

        training_cost.append(tc)
        cv_cost.append(cv)

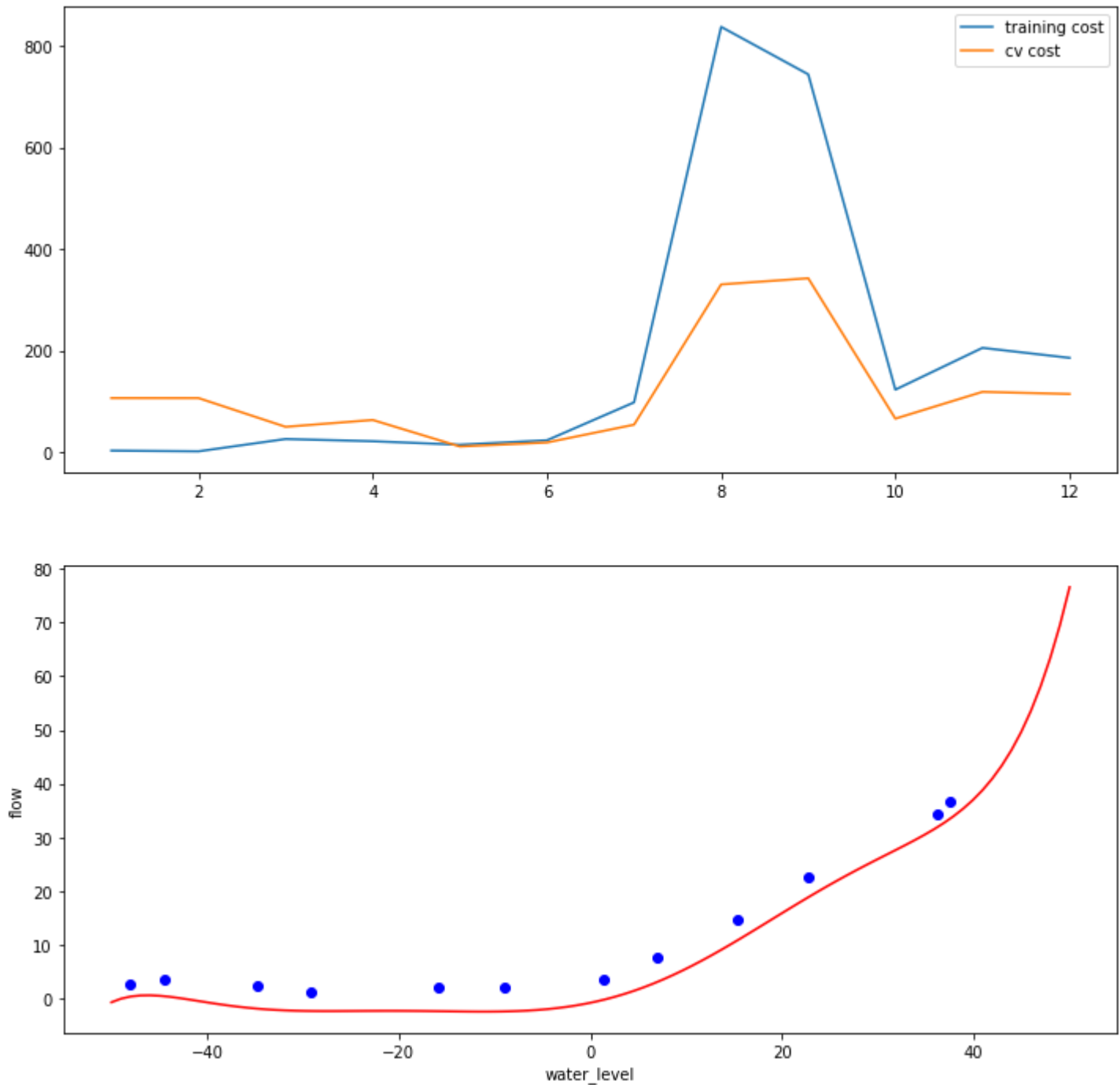
    #绘图
    fig, ax = plt.subplots(2, 1, figsize=(12, 12))
    ax[0].plot(np.arange(1, m + 1), training_cost, label='training cost')
    ax[0].plot(np.arange(1, m + 1), cv_cost, label='cv cost')
    ax[0].legend()

    fitx = np.linspace(-50, 50, 100)
    fitxtmp = prepare_poly_data(fitx, power=8)
    fity = np.dot(prepare_poly_data(fitx, power=8)[0], linear_regression(X, y, 1).x.T)

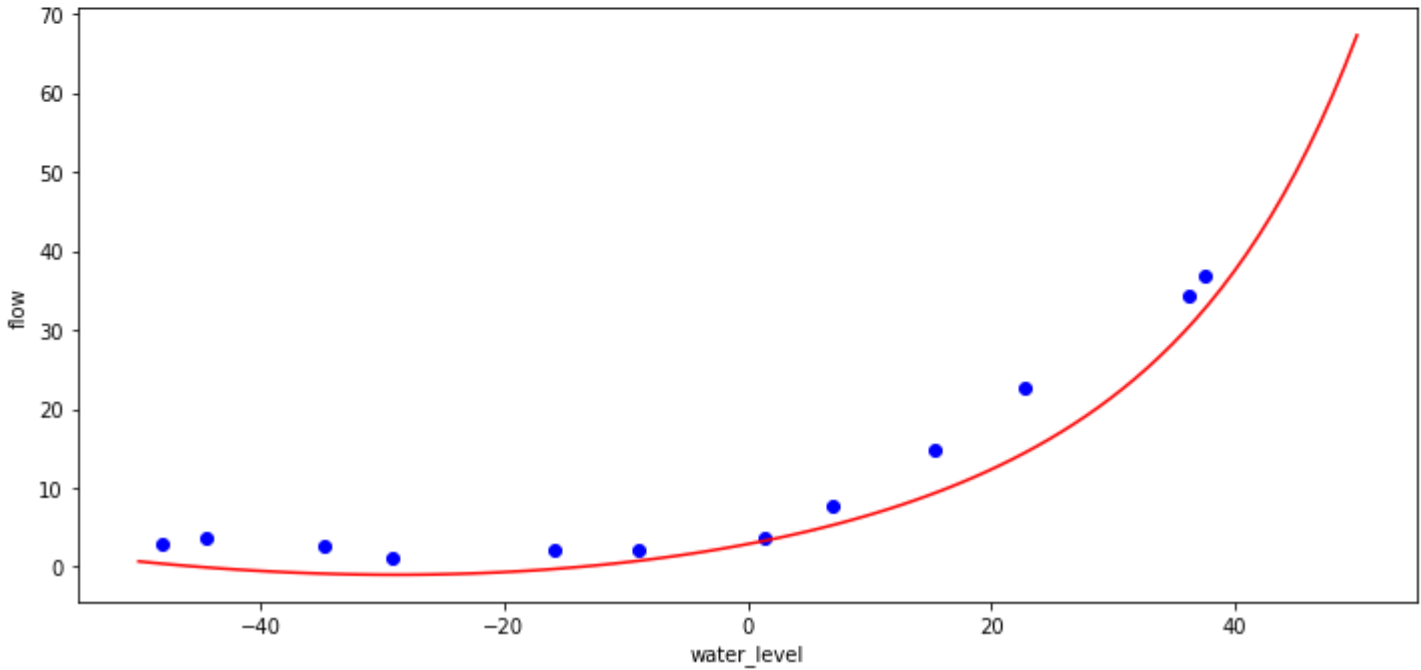
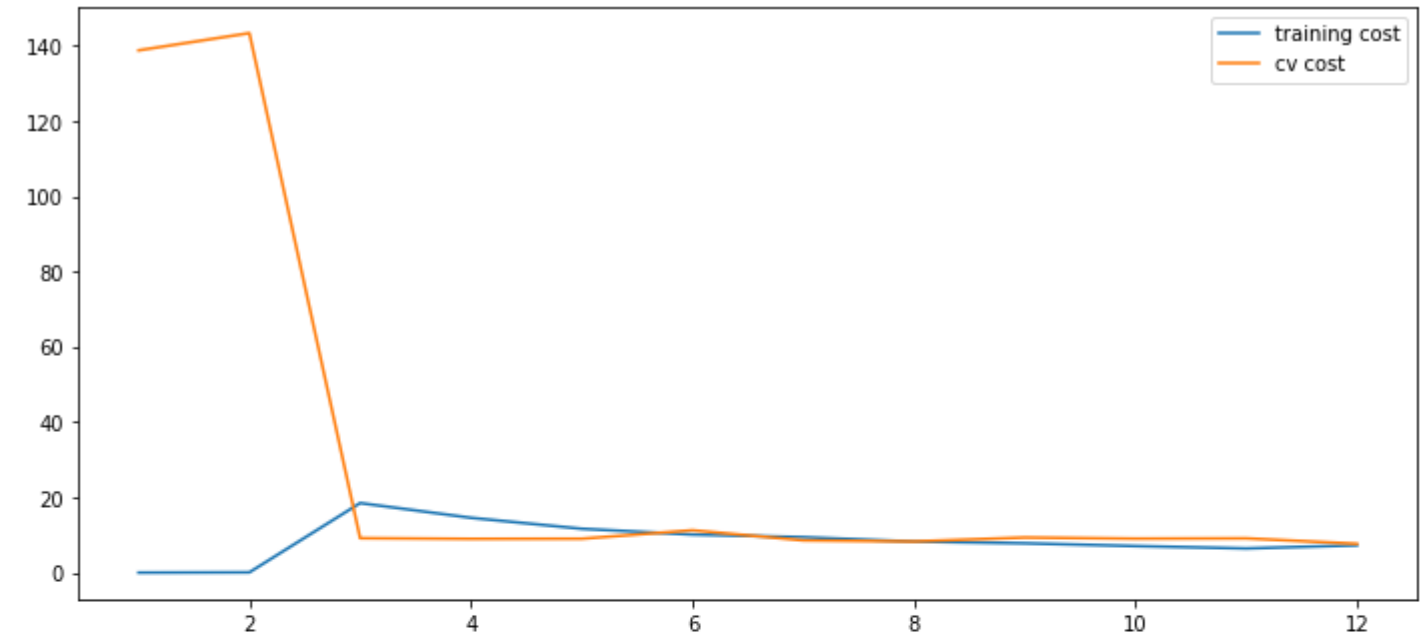
    ax[1].plot(fitx, fity, c='r', label='fitcurve')
    ax[1].scatter(Xinit, y, c='b', label='initial_Xy')

    ax[1].set_xlabel('water_level')
    ax[1].set_ylabel('flow')
```

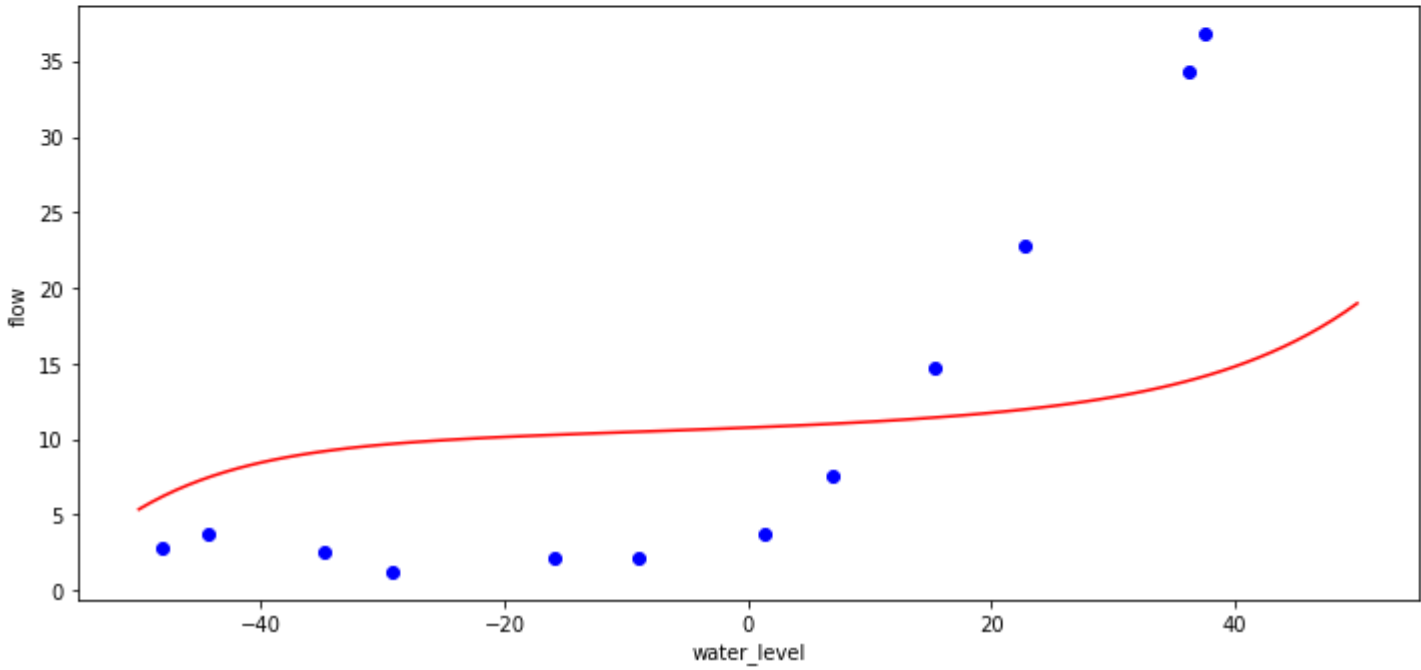
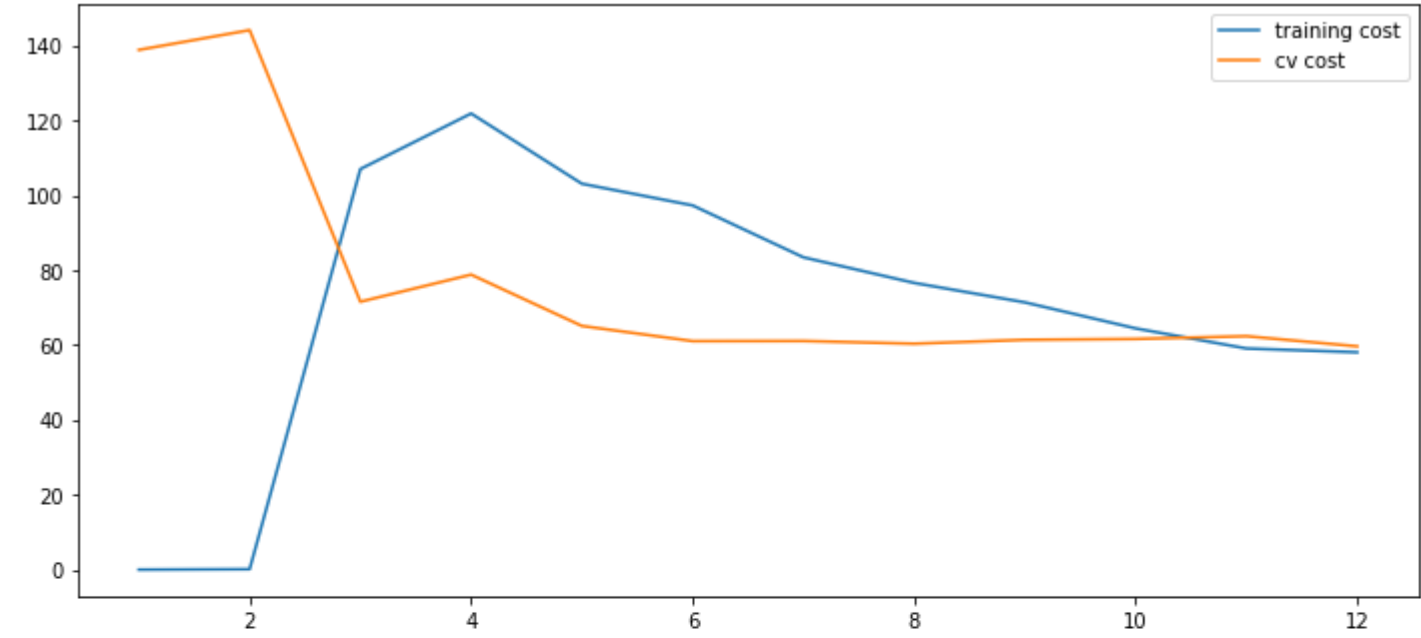
```
In [21]: plot_learning_curve(X_poly, X, y, Xval_poly, yval, l=0)
plt.show()
#过拟合
```



```
In [22]: #调整正则化系数 λ
plot_learning_curve(X_poly, X, y, Xval_poly, yval, l=1)
plt.show()
#减轻过拟合
```



```
In [23]: plot_learning_curve(X_poly, X, y, Xval_poly, yval, l=100)
plt.show()
#正则化惩罚太多，变成欠拟合状态
```



```
In [24]: #使用验证集去评价的表现好坏，然后选择表现最好的λ
l_candidate = [0, 0.001, 0.003, 0.01, 0.03, 0.1, 0.3, 1, 3, 10]
training_cost, cv_cost = [], []
```

```
In [25]: for l in l_candidate:
    res = linear_regression(X_poly, y, l)

    tc = cost(res.x, X_poly, y)
    cv = cost(res.x, Xval_poly, yval)

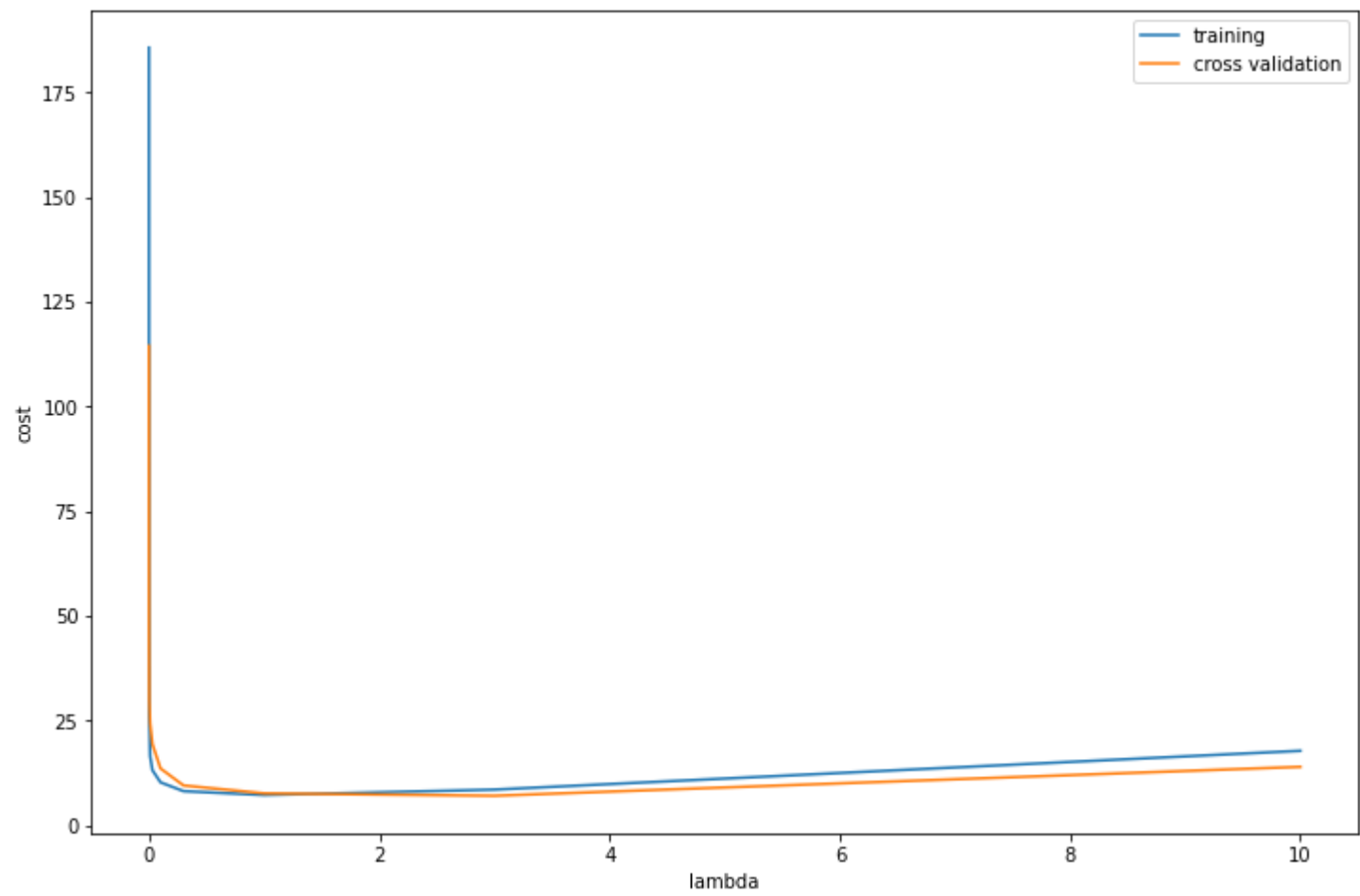
    training_cost.append(tc)
    cv_cost.append(cv)
```

```
In [26]: fig, ax = plt.subplots(figsize=(12,8))
ax.plot(l_candidate, training_cost, label='training')
ax.plot(l_candidate, cv_cost, label='cross validation')
plt.legend()

plt.xlabel('lambda')

plt.ylabel('cost')
plt.show()

#对应的λ的值约为1
```



```
In [27]: #计算测试集上的误差，把最终的模型用在一个从来没有在计算中出现过的测试集上，即既没有被用作  $\lambda$  选择，也没有被用作选择  $\lambda$  的数据
for l in l_candidate:
    theta = linear_regression(X_poly, y, l).x
    print('test cost(l={}) = {}'.format(l, cost(theta, Xtest_poly, ytest)))
```

```
test cost(l=0) = 116.69690232713525
test cost(l=0.001) = 46.10351993962107
test cost(l=0.003) = 26.29790945333314
test cost(l=0.01) = 20.22676949634041
test cost(l=0.03) = 17.40181639037931
test cost(l=0.1) = 14.255345281059459
test cost(l=0.3) = 11.488568070045726
test cost(l=1) = 10.433102088110367
test cost(l=3) = 13.732759897867563
test cost(l=10) = 28.70250800661905
```

1 支持向量

使用高斯核函数的支持向量机（SVM）来构建垃圾邮件分类器。

```
In [1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sb
from scipy.io import loadmat
```

```
In [2]: raw_data = loadmat('data/ex6data1.mat')
data = pd.DataFrame(raw_data.get('X'), columns=['X1', 'X2'])
data['y'] = raw_data.get('y')

data.head()
```

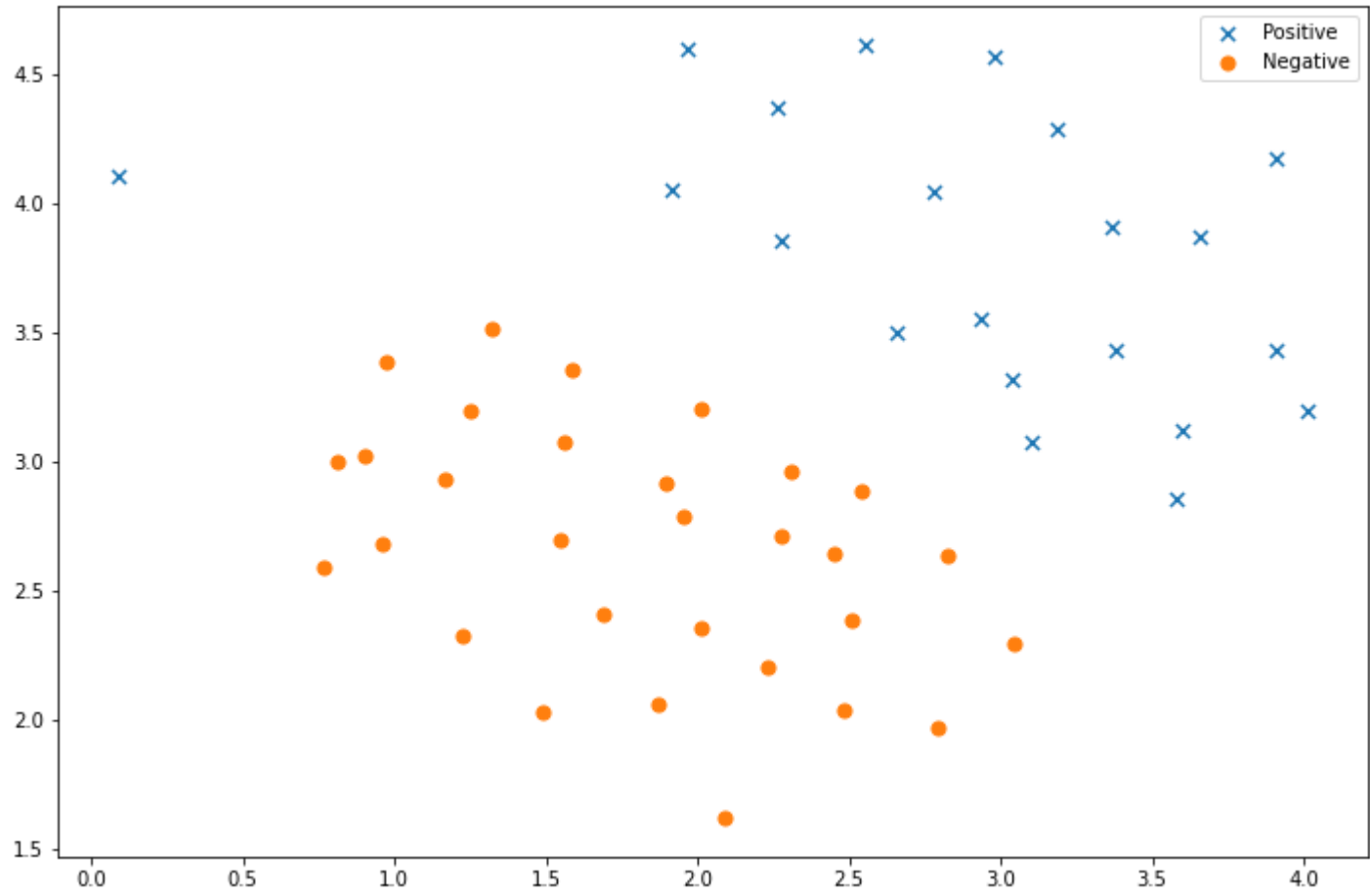
Out[2]:

	X1	X2	y
0	1.9643	4.5957	1
1	2.2753	3.8589	1
2	2.9781	4.5651	1
3	2.9320	3.5519	1
4	3.5772	2.8560	1

```
In [3]: def plot_init_data(data, fig, ax):
    positive = data[data['y'].isin([1])]
    negative = data[data['y'].isin([0])] #清洗数据，删选过滤掉DataFrame中一些行

    ax.scatter(positive['X1'], positive['X2'], s=50, marker='x', label='Positive')
    ax.scatter(negative['X1'], negative['X2'], s=50, marker='o', label='Negative')
```

```
In [4]: fig, ax = plt.subplots(figsize=(12,8))
plot_init_data(data, fig, ax)
ax.legend()
plt.show()
```



```
In [5]: from sklearn import svm
svc = svm.LinearSVC(C=1, loss='hinge', max_iter=10000)
"""
C : float, optional (default=1.0)错误项的惩罚参数
loss : string, 'hinge' or 'squared_hinge' (default=' squared_hinge' ) 指定损失函数。
“hinge”是标准的SVM损失（例如由SVC类使用），而“squared_hinge”是hinge损失的平方。
max_iter : int, (default=1000) 要运行的最大迭代次数。
"""
svc
```

Out[5]: LinearSVC(C=1, loss='hinge', max_iter=10000)

```
In [6]: svc.fit(data[['X1', 'X2']], data['y']) #输入参数训练模型
svc.score(data[['X1', 'X2']], data['y']) #得到分数
```

Out[6]: 0.9803921568627451

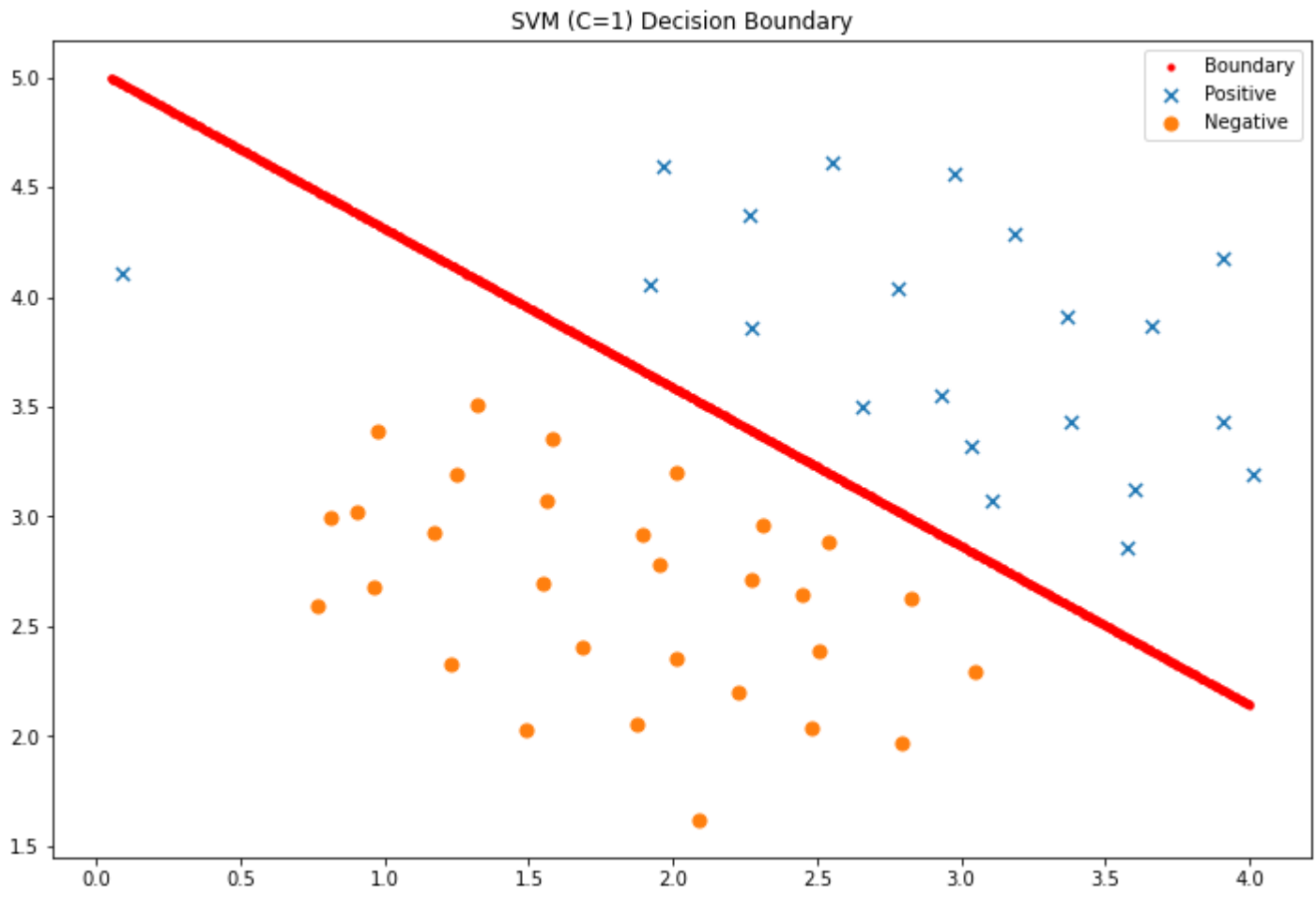
```
In [7]: def find_decision_boundary(svc, x1min, x1max, x2min, x2max, diff):
    x1 = np.linspace(x1min, x1max, 1000)
    x2 = np.linspace(x2min, x2max, 1000) #创建等差数列

    cordinates = [(x, y) for x in x1 for y in x2]
    x_cord, y_cord = zip(*cordinates) #zip(*) 解压
    c_val = pd.DataFrame({'x1':x_cord, 'x2':y_cord})
    c_val['cval'] = svc.decision_function(c_val[['x1', 'x2']])

    decision = c_val[np.abs(c_val['cval']) < diff]

    return decision.x1, decision.x2
```

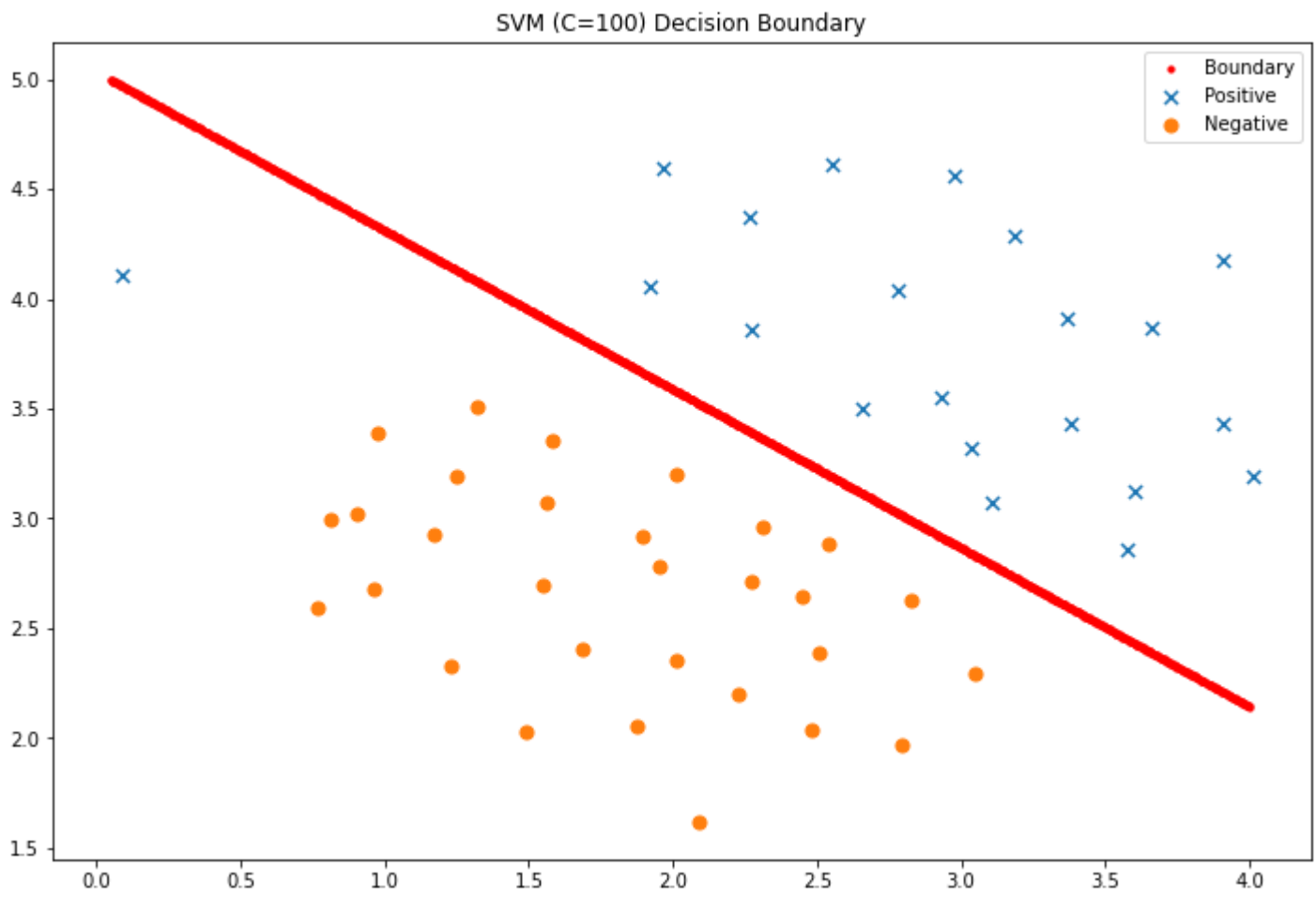
```
In [8]: x1, x2 = find_decision_boundary(svc, 0, 4, 1.5, 5, 2 * 10**-3)
fig, ax = plt.subplots(figsize=(12,8))
ax.scatter(x1, x2, s=10, c='r', label='Boundary')
plot_init_data(data, fig, ax)
ax.set_title(' SVM (C=1) Decision Boundary')
ax.legend()
plt.show()
```



```
In [9]: svc2 = svm.LinearSVC(C=100, loss='hinge', max_iter=20000)
svc2.fit(data[['X1', 'X2']], data['y'])
svc2.score(data[['X1', 'X2']], data['y'])
```

Out[9]: 0.9803921568627451

```
In [10]: x1, x2 = find_decision_boundary(svc, 0, 4, 1.5, 5, 2 * 10**-3)
fig, ax = plt.subplots(figsize=(12,8))
ax.scatter(x1, x2, s=10, c='r', label='Boundary')
plot_init_data(data, fig, ax)
ax.set_title('SVM (C=100) Decision Boundary')
ax.legend()
plt.show()
```



1.2 高斯内核的SVM

将从线性SVM转移到能够使用内核进行非线性分类的SVM。虽然scikit-learn具有内置的高斯内核，但为实现更清楚，将从头开始实现。

```
In [11]: def gaussian_kernel(x1, x2, sigma):
return np.exp(-(np.sum((x1 - x2) ** 2) / (2 * (sigma ** 2))))
```

```
In [12]: x1 = np.array([1.0, 2.0, 1.0])
x2 = np.array([0.0, 4.0, -1.0])
sigma = 2

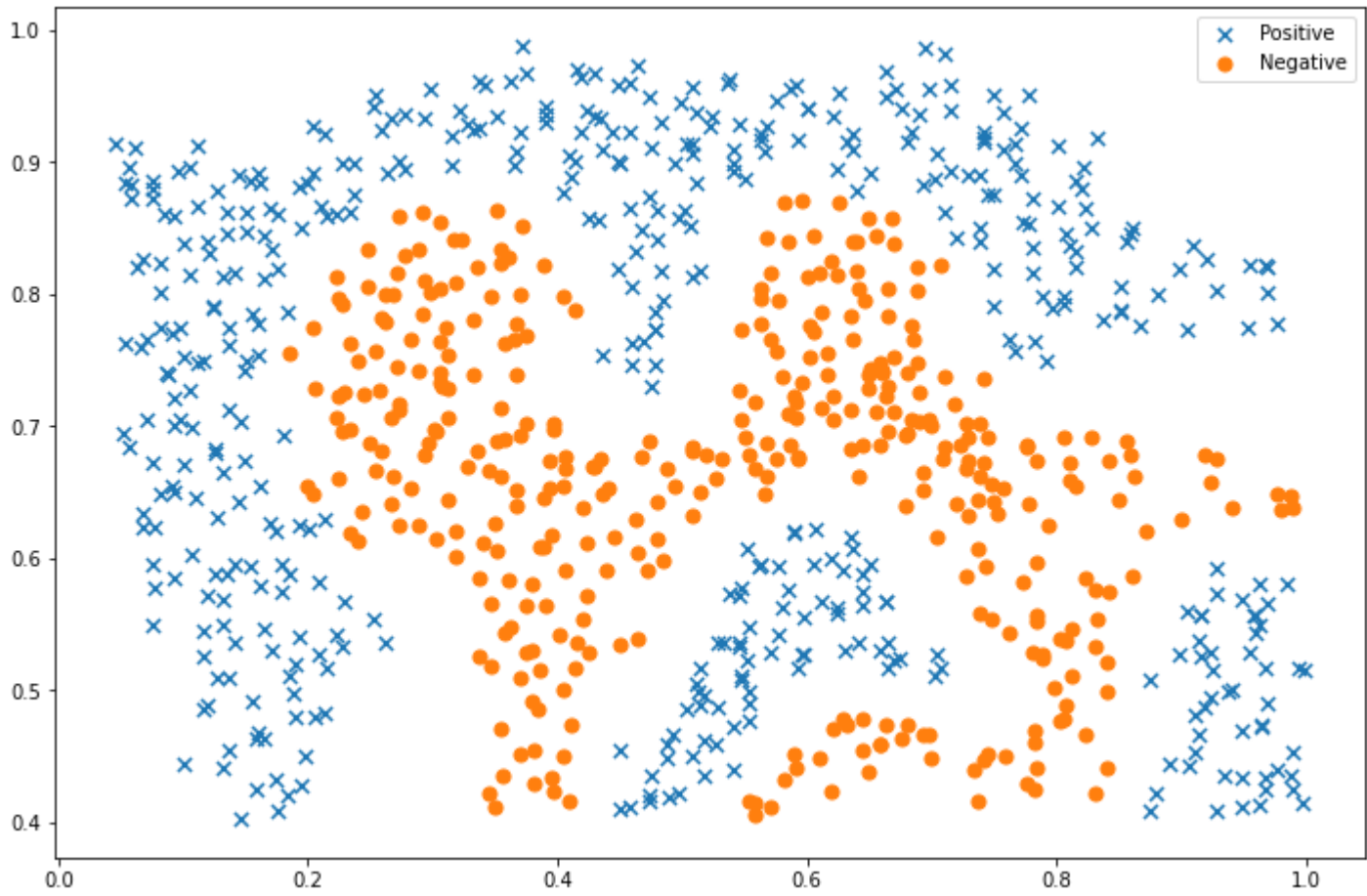
gaussian_kernel(x1, x2, sigma)
```

Out[12]: 0.32465246735834974

```
In [13]: raw_data = loadmat('data/ex6data2.mat')

data = pd.DataFrame(raw_data['X'], columns=['X1', 'X2'])
data['y'] = raw_data['y']

fig, ax = plt.subplots(figsize=(12,8))
plot_init_data(data, fig, ax)
ax.legend()
plt.show()
```

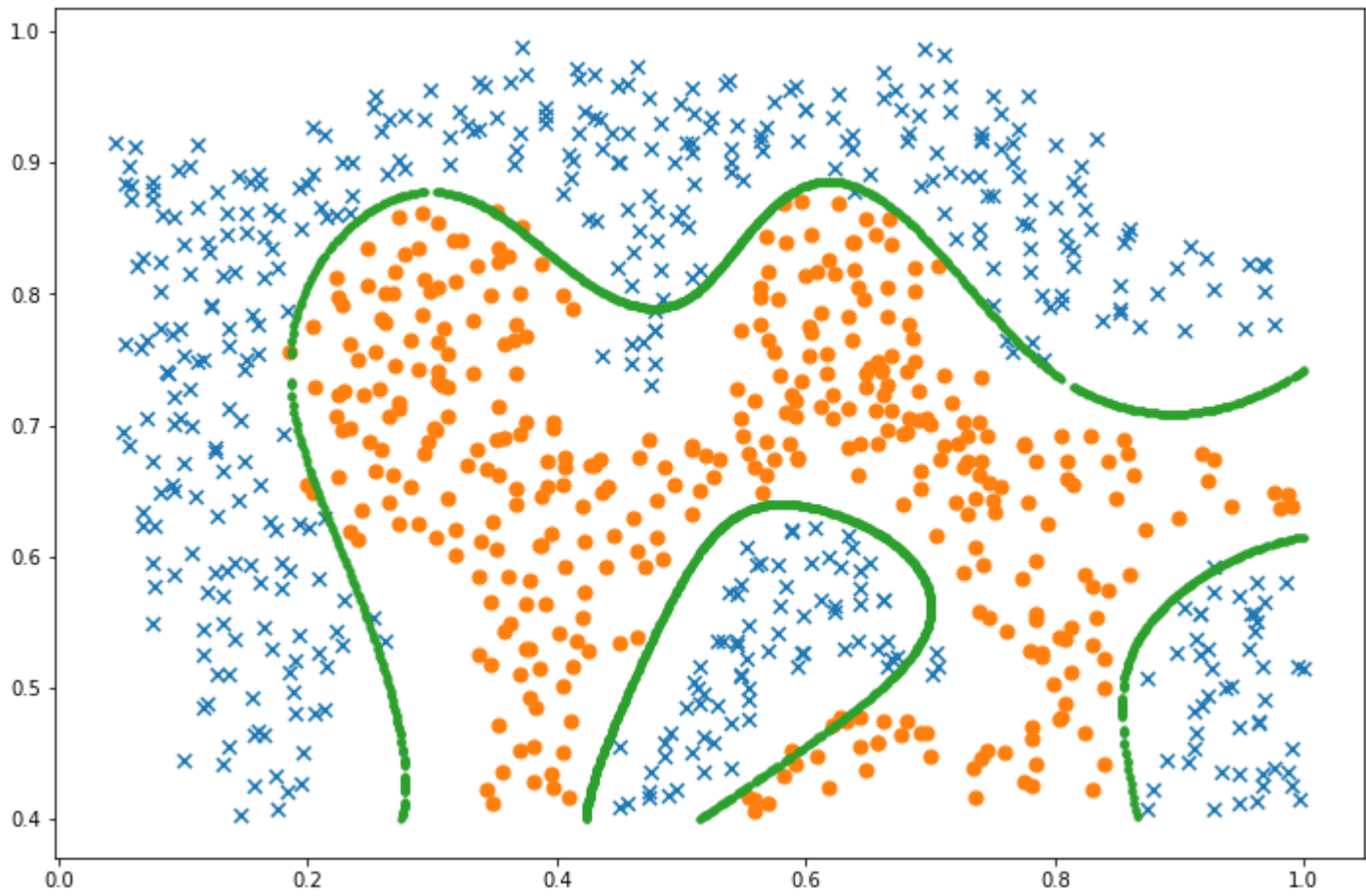


```
In [14]: svc = svm.SVC(C=100, gamma=10, probability=True)
"""
gamma : 'rbf', 'poly' 和 'sigmoid' 的核函数参数。默认是' auto'，则会选择1/n_features
kernel : 核函数，默认是rbf，可以是'linear'，'poly'，'rbf'，'sigmoid'，'precomputed'
RBF函数: exp(-gamma|u-v|^2)

"""
svc.fit(data[['X1', 'X2']], data['y'])
svc.score(data[['X1', 'X2']], data['y'])
```

Out[14]: 0.9698725376593279

```
In [15]: x1, x2 = find_decision_boundary(svc, 0, 1, 0.4, 1, 0.01)
fig, ax = plt.subplots(figsize=(12,8))
plot_init_data(data, fig, ax)
ax.scatter(x1, x2, s=10)
plt.show()
```



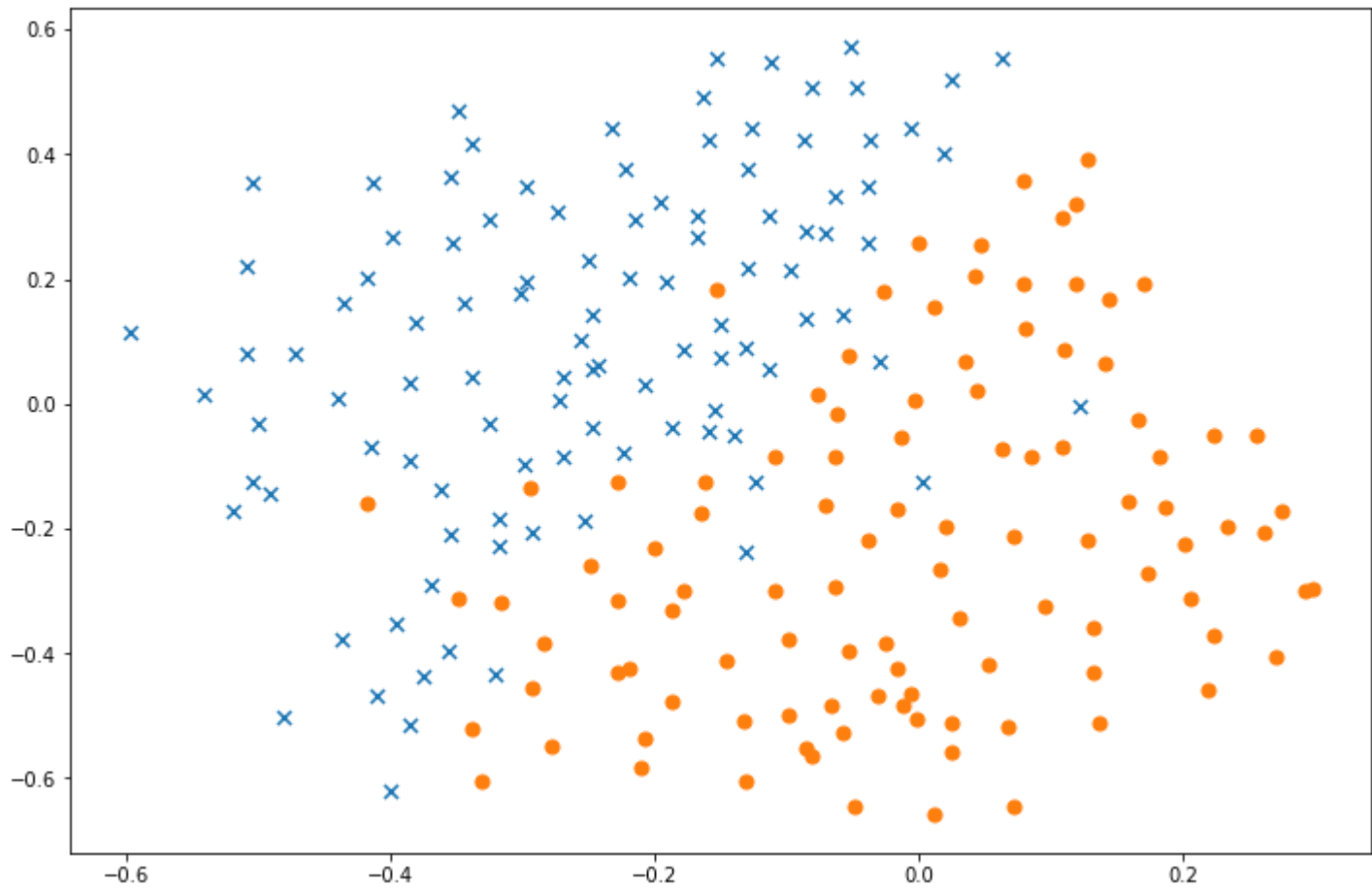
1.2.3 数据集3

对于第三个数据集，我们给出了训练和验证集，并且基于验证集性能为SVM模型找到最优超参数。超参数是在开始学习过程之前设置值的参数 我们现在需要寻找最优和，候选数值为[0.01, 0.03, 0.1, 0.3, 1, 3, 10, 30, 100]

```
In [16]: raw_data = loadmat('data/ex6data3.mat')

X = raw_data['X']
Xval = raw_data['Xval']
y = raw_data['y'].ravel()
yval = raw_data['yval'].ravel()

fig, ax = plt.subplots(figsize=(12,8))
data = pd.DataFrame(raw_data.get('X'), columns=['X1', 'X2'])
data['y'] = raw_data.get('y')
plot_init_data(data, fig, ax)
plt.show()
```



```
In [17]: C_values = [0.01, 0.03, 0.1, 0.3, 1, 3, 10, 30, 100]
gamma_values = [0.01, 0.03, 0.1, 0.3, 1, 3, 10, 30, 100]

best_score = 0
best_params = {'C': None, 'gamma': None}
```

```
for C in C_values:
    for gamma in gamma_values:
        svc = svm.SVC(C=C, gamma=gamma)
        svc.fit(X, y)
        score = svc.score(Xval, yval)

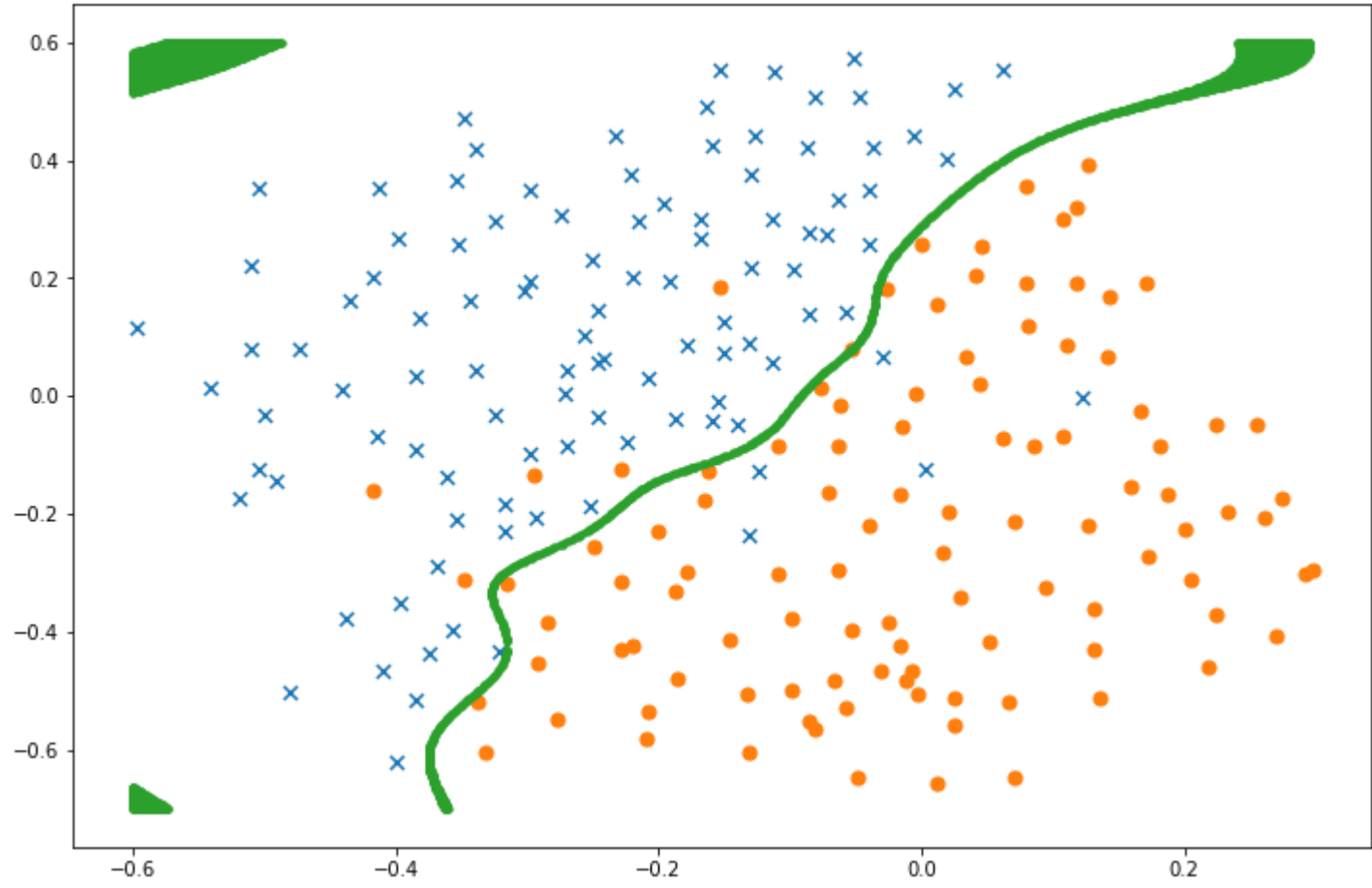
        if score > best_score:
            best_score = score
            best_params['C'] = C
            best_params['gamma'] = gamma

best_score, best_params
```

Out[17]: (0.965, {'C': 0.3, 'gamma': 100})

```
In [18]: svc = svm.SVC(C=best_params['C'], gamma=best_params['gamma'])
svc.fit(X, y)

x1, x2 = find_decision_boundary(svc, -0.6, 0.3, -0.7, 0.6, 0.005)
fig, ax = plt.subplots(figsize=(12,8))
plot_init_data(data, fig, ax)
ax.scatter(x1, x2, s=10)
plt.show()
```



2 垃圾邮件分类

使用SVM来构建垃圾邮件过滤器。预处理步骤（如HTML删除，词干，标准化等）已经完成，只需将字词映射到为练习提供的字典中的ID

```
In [19]: spam_train = loadmat('data/machine-learning-ex6/ex6/spamTrain.mat')
spam_test = loadmat('data/machine-learning-ex6/ex6/spamTest.mat')

spam_train
```

```
Out[19]: {'__header__': b'MATLAB 5.0 MAT-file, Platform: GLNXA64, Created on: Sun Nov 13 14:27:25 2011',
 '__version__': '1.0',
 '__globals__': [],
 'X': array([[0, 0, 0, ..., 0, 0, 0],
            [0, 0, 0, ..., 0, 0, 0],
            [0, 0, 0, ..., 0, 0, 0],
            ...,
            [0, 0, 0, ..., 0, 0, 0],
            [0, 0, 1, ..., 0, 0, 0],
            [0, 0, 0, ..., 0, 0, 0]], dtype=uint8),
 'y': array([[1],
            [1],
            [0],
            ...,
            [1],
            [0],
            [0]], dtype=uint8))
```

```
In [20]: #每个文档转换为一个向量，其中1,899个维对应于词汇表中的1,899个单词。 它们的值为二进制，表示文档中是否存在单词
X = spam_train['X']
Xtest = spam_test['Xtest']
y = spam_train['y'].ravel()
ytest = spam_test['ytest'].ravel()

X.shape, y.shape, Xtest.shape, ytest.shape
```

Out[20]: ((4000, 1899), (4000,), (1000, 1899), (1000,))

```
In [21]: svc = svm.SVC()
svc.fit(X, y)
```

Out[21]: SVC()

```
In [22]: print('Training accuracy = {0}%'.format(np.round(svc.score(X, y) * 100, 2))) #np.round: 取整
print('Test accuracy = {0}%'.format(np.round(svc.score(Xtest, ytest) * 100, 2)))
#.format(), 字符串格式化，例： '数字{1}{2}和{0}'.format("123",456,'789')>>>'数字456789和123'

Training accuracy = 99.32%
Test accuracy = 98.7%
```

```
In [23]: kw = np.eye(1899) #生成对角阵
kw[:3,:]

array([[1., 0., 0., ..., 0., 0., 0.],
       [0., 1., 0., ..., 0., 0., 0.],
       [0., 0., 1., ..., 0., 0., 0.]])
```

```
In [24]: spam_val = pd.DataFrame({'idx':range(1899)})
spam_val['isspam'] = svc.decision_function(kw)
spam_val['isspam'].describe() #打印数据的统计信息
```

Out[24]: count 1899.000000
mean -0.110039
std 0.049094
min -0.428396
25% -0.131213
50% -0.111985
75% -0.091973
max 0.396286
Name: isspam, dtype: float64

```
In [25]: decision = spam_val[spam_val['isspam'] > -0.55]
decision
```

Out[25]:

	idx	isspam
	0	0 -0.093653
	1	1 -0.083078
	2	2 -0.109401
	3	3 -0.119685
	4	4 -0.165824
	...	...
	1894	1894 0.101613
	1895	1895 -0.016065
	1896	1896 -0.151573
	1897	1897 -0.109022
	1898	1898 -0.091970

1899 rows × 2 columns

```
In [26]: path = 'data/machine-learning-ex6/ex6/vocab.txt'
voc = pd.read_csv(path, header=None, names=['idx', 'voc'], sep = '\t')
voc.head()
```

Out[26]:

	idx	voc
0	1	aa
1	2	ab
2	3	abil
3	4	abl
4	5	about

```
In [27]: spamvoc = voc.loc[list(decision['idx'])]
spamvoc
```

Out[27]:

	idx	voc
	0	1 aa
	1	2 ab
	2	3 abil
	3	4 abl
	4	5 about
	...	...
	1894	1895 your
	1895	1896 yourself
	1896	1897 zdnet
	1897	1898 zero
	1898	1899 zip

1899 rows × 2 columns

K-means 聚类

1.找到数据中每个实例最近的聚类中心的函数。

```
In [1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sb
from scipy.io import loadmat
```

```
In [2]: def find_closest_centroids(X, centroids):
    m = X.shape[0]
    k = centroids.shape[0]
    idx = np.zeros(m)

    for i in range(m):
        min_dist = 1000000
        for j in range(k):
            dist = np.sum((X[i,:] - centroids[j,:]) ** 2) # **2—平方
            if dist < min_dist:
                min_dist = dist
                idx[i] = j

    return idx
```

```
In [3]: #test
data = loadmat('data/machine-learning-ex7/ex7/ex7data2.mat')
X = data['X']
initial_centroids = initial_centroids = np.array([[3, 3], [6, 2], [8, 5]])
idx = find_closest_centroids(X, initial_centroids)
idx[0:3]
```

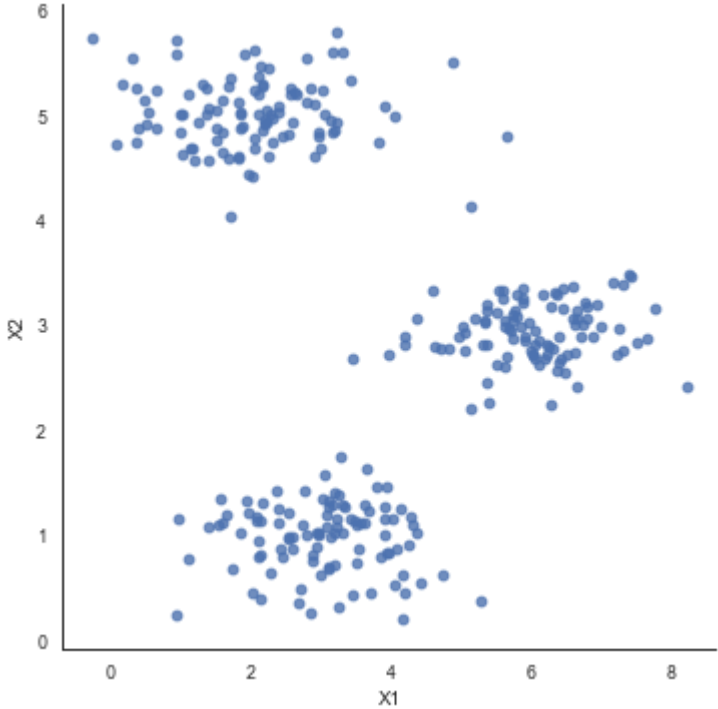
Out[3]: array([0., 2., 1.])

```
In [4]: data2 = pd.DataFrame(data.get('X'), columns=['X1', 'X2'])
data2.head()
```

Out[4]:

	X1	X2
0	1.842080	4.607572
1	5.658583	4.799964
2	6.352579	3.290854
3	2.904017	4.612204
4	3.231979	4.939894

```
In [5]: sb.set(context="paper", style="white") #绘图风格
sb.lmplot(x='X1', y='X2', data=data2, fit_reg=False) #sb.lmplot:多图叠加,一个x多个y
plt.show()
```



```
In [6]: def compute_centroids(X, idx, k):
    m, n = X.shape
    centroids = np.zeros((k, n))

    for i in range(k):
        indices = np.where(idx == i) #只有条件 (condition), 没有x和y, 则输出满足条件 (即非0) 元素的坐标
        centroids[i,:] = (np.sum(X[indices,:], axis=1) / len(indices[0])).ravel()
        #[i,:]第一维度元素保留到i, 保留第二个维度所有元素,
    return centroids
```

```
In [7]: compute_centroids(X, idx, 3)
```

Out[7]: array([[2.42830111, 3.15792418],
[5.81350331, 2.63365645],
[7.11938687, 3.6166844]])

```
In [8]: #为了运行算法, 只需要在将样本分配给最近的簇并重新计算簇的聚类中心。
def run_k_means(X, initial_centroids, max_iters):
    m, n = X.shape
    k = initial_centroids.shape[0]
    idx = np.zeros(m)
    centroids = initial_centroids

    for i in range(max_iters):
        idx = find_closest_centroids(X, centroids)
        centroids = compute_centroids(X, idx, k)

    return idx, centroids
```

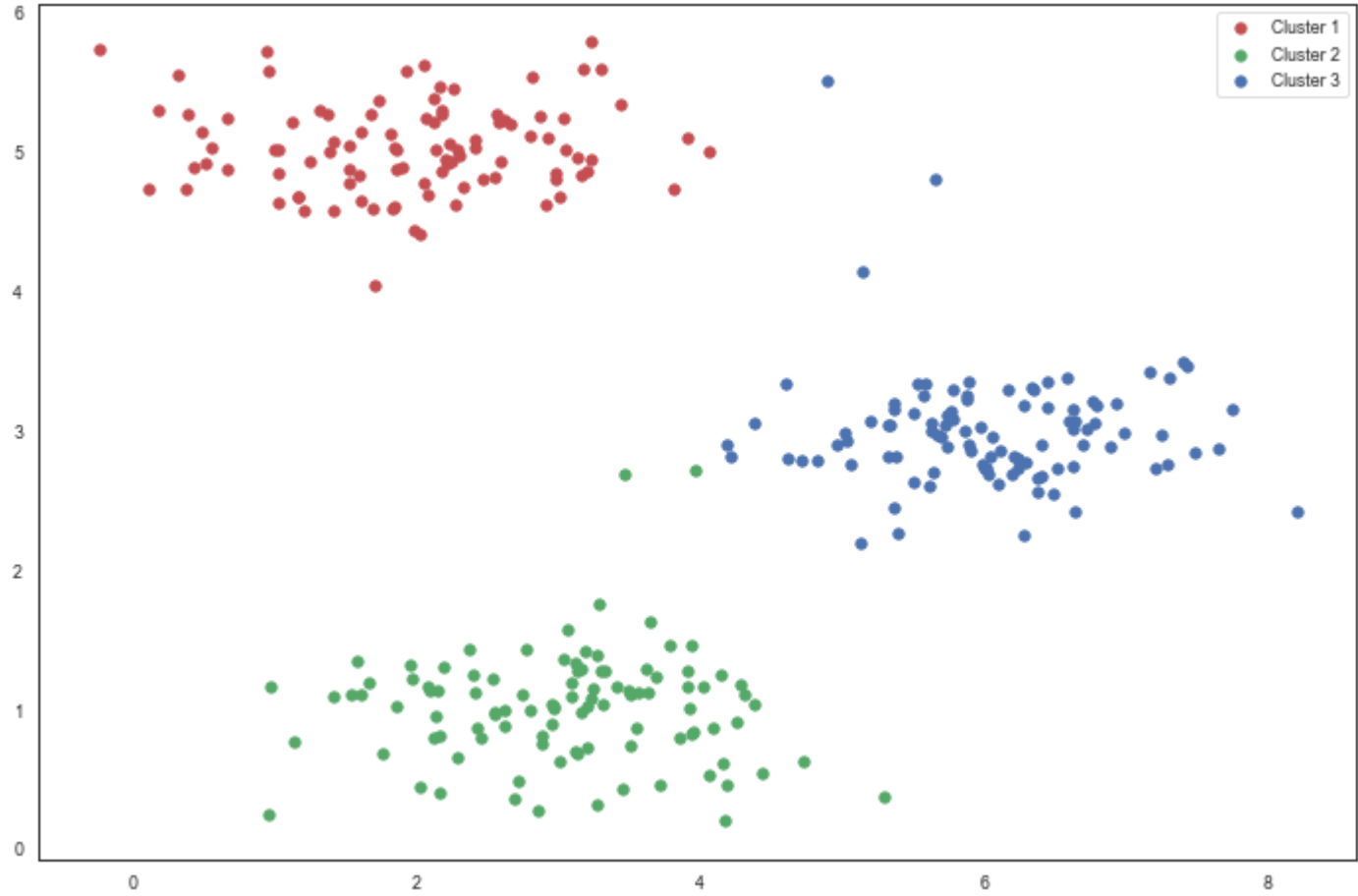
```
In [9]: idx, centroids = run_k_means(X, initial_centroids, 10)
centroids
```

Out[9]: array([[1.95399466, 5.02557006],
[3.04367119, 1.01541041],
[6.03366736, 3.00052511]])

```
In [10]: cluster1 = X[np.where(idx == 0)[0],:]
cluster2 = X[np.where(idx == 1)[0],:]
cluster3 = X[np.where(idx == 2)[0],:]

fig, ax = plt.subplots(figsize=(12,8))
ax.scatter(cluster1[:,0], cluster1[:,1], s=30, color='r', label='Cluster 1')
ax.scatter(cluster2[:,0], cluster2[:,1], s=30, color='g', label='Cluster 2')
```

```
ax.scatter(cluster3[:,0], cluster3[:,1], s=30, color='b', label='Cluster 3')
ax.legend()
plt.show()
```



```
In [11]: #初始化聚类中心的过程可以影响算法的收敛，因此，创建一个选择随机样本并将其用作初始聚类中心的函数。
def init_centroids(X, k):
    m, n = X.shape
    centroids = np.zeros((k, n))
    idx = np.random.randint(0, m, k)

    for i in range(k):
        centroids[i,:] = X[idx[i],:]

    return centroids
```

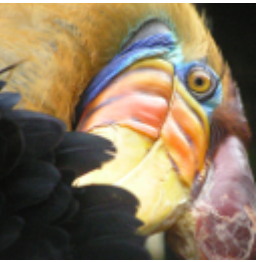
```
In [12]: init_centroids(X, 3)
```

```
Out[12]: array([[1.82975993, 4.59657288],
                [0.10511804, 4.72916344],
                [1.70725482, 4.04231479]])
```

2.将K-means应用于图像压缩。

使用聚类来找到最具代表性的少数颜色，并使用聚类分配将原始的24位颜色映射到较低维的颜色空间。

```
In [13]: from IPython.display import Image
Image(filename='data/machine-learning-ex7/ex7/bird_small.png')
```



```
In [14]: #像素点数值
image_data = loadmat('data/machine-learning-ex7/ex7/bird_small.mat')
image_data
```

```
Out[14]: {'__header__': b'MATLAB 5.0 MAT-file, Platform: GLNXA64, Created on: Tue Jun  5 04:06:24 2012',
          '__version__': '1.0',
          '__globals__': [],
          'A': array([[219, 180, 103],
                    [230, 185, 116],
                    [226, 186, 110],
                    ...,
                    [ 14,  15,  13],
                    [ 13,  15,  12],
                    [ 12,  14,  12]],

                    [[230, 193, 119],
                    [224, 192, 120],
                    [226, 192, 124],
                    ...,
                    [ 16,  16,  13],
                    [ 14,  15,  10],
                    [ 11,  14,   9]],

                    [[228, 191, 123],
                    [228, 191, 121],
                    [220, 185, 118],
                    ...,
                    [ 14,  16,  13],
                    [ 13,  13,  11],
                    [ 11,  15,  10]],

                    ...,

                    [[ 15,  18,  16],
                    [ 18,  21,  18],
                    [ 18,  19,  16],
                    ...,
                    [ 81,  45,  45],
                    [ 70,  43,  35],
                    [ 72,  51,  43]],

                    [[ 16,  17,  17],
                    [ 17,  18,  19],
                    [ 20,  19,  20],
                    ...,
                    [ 80,  38,  40],
                    [ 68,  39,  40],
                    [ 59,  43,  42]],

                    [[ 15,  19,  19],
                    [ 20,  20,  18],
                    [ 18,  19,  17],
                    ...,
                    [ 65,  43,  39],
                    [ 58,  37,  38],
                    [ 52,  39,  34]]], dtype=uint8)}
```

```
In [15]: A = image_data['A']
A.shape
```

```
Out[15]: (128, 128, 3)
```

```
In [16]: #预处理
# normalize value ranges
```

```
A = A / 255.
# reshape the array
X = np.reshape(A, (A.shape[0] * A.shape[1], A.shape[2])) #在不改变数据内容的情况下，改变一个数组的格式，参数及返回值
X.shape
```

Out[16]: (16384, 3)

```
In [17]: # randomly initialize the centroids
initial_centroids = init_centroids(X, 16)

# run the algorithm
idx, centroids = run_k_means(X, initial_centroids, 10)

# get the closest centroids one last time
idx = find_closest_centroids(X, centroids)

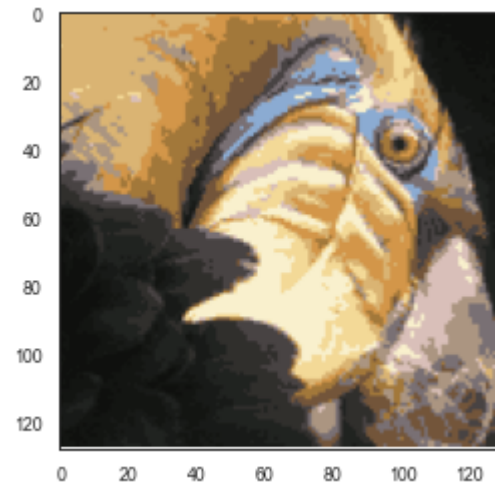
# map each pixel to the centroid value
X_recovered = centroids[idx.astype(int),:] #astype: 转换数组的数据类型。
X_recovered.shape
```

Out[17]: (16384, 3)

```
In [18]: # reshape to the original dimensions
X_recovered = np.reshape(X_recovered, (A.shape[0], A.shape[1], A.shape[2]))
X_recovered.shape
```

Out[18]: (128, 128, 3)

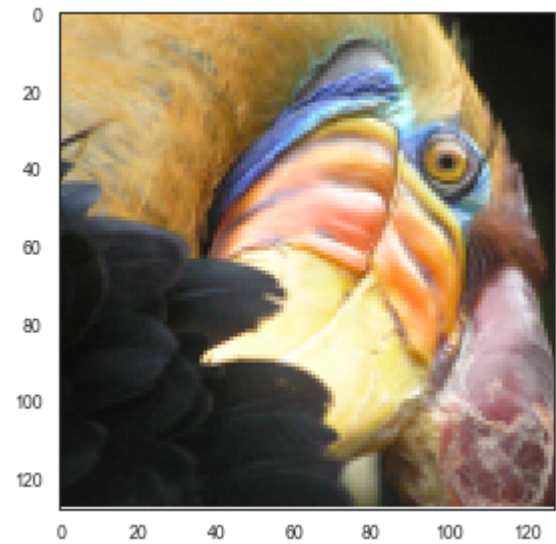
```
In [19]: plt.imshow(X_recovered)
plt.show()
#对图像进行了压缩，但图像的主要特征仍然存在,这就是K-means。
```



下面用scikit-learn实现K-means。

```
In [20]: from skimage import io

# cast to float, you need to do this otherwise the color would be weird after clustering
pic = io.imread('data/machine-learning-ex7/ex7/bird_small.png') / 255.
io.imshow(pic)
plt.show()
```



```
In [21]: data = pic.reshape(128*128, 3)
data.shape
```

Out[21]: (16384, 3)

```
In [22]: from sklearn.cluster import KMeans#导入kmeans库
model = KMeans(n_clusters=16, n_init=100)
model.fit(data)
```

Out[22]: KMeans(n_clusters=16, n_init=100)

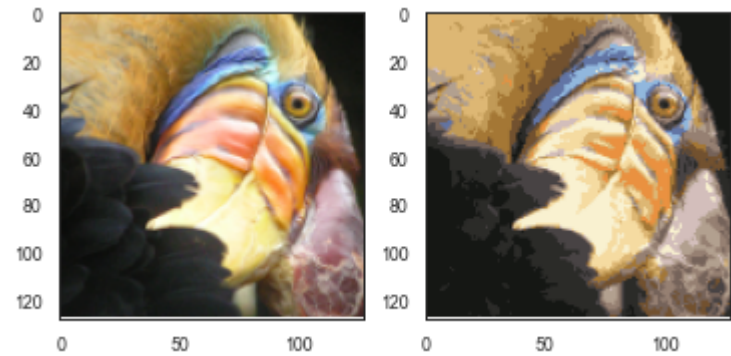
```
In [23]: centroids = model.cluster_centers_ #KMeans.cluster_centers_返回中心的坐标
print(centroids.shape)

C = model.predict(data)
print(C.shape)
print(centroids[C].shape)
```

(16, 3)
(16384,)
(16384, 3)

```
In [24]: compressed_pic = centroids[C].reshape((128,128,3))
```

```
In [25]: fig, ax = plt.subplots(1, 2)
ax[0].imshow(pic)
ax[1].imshow(compressed_pic)
plt.show()
```



Principal component analysis（主成分分析）

PCA是在数据集中找到“主成分”或最大方差方向的线性变换。 它可以用于降维。

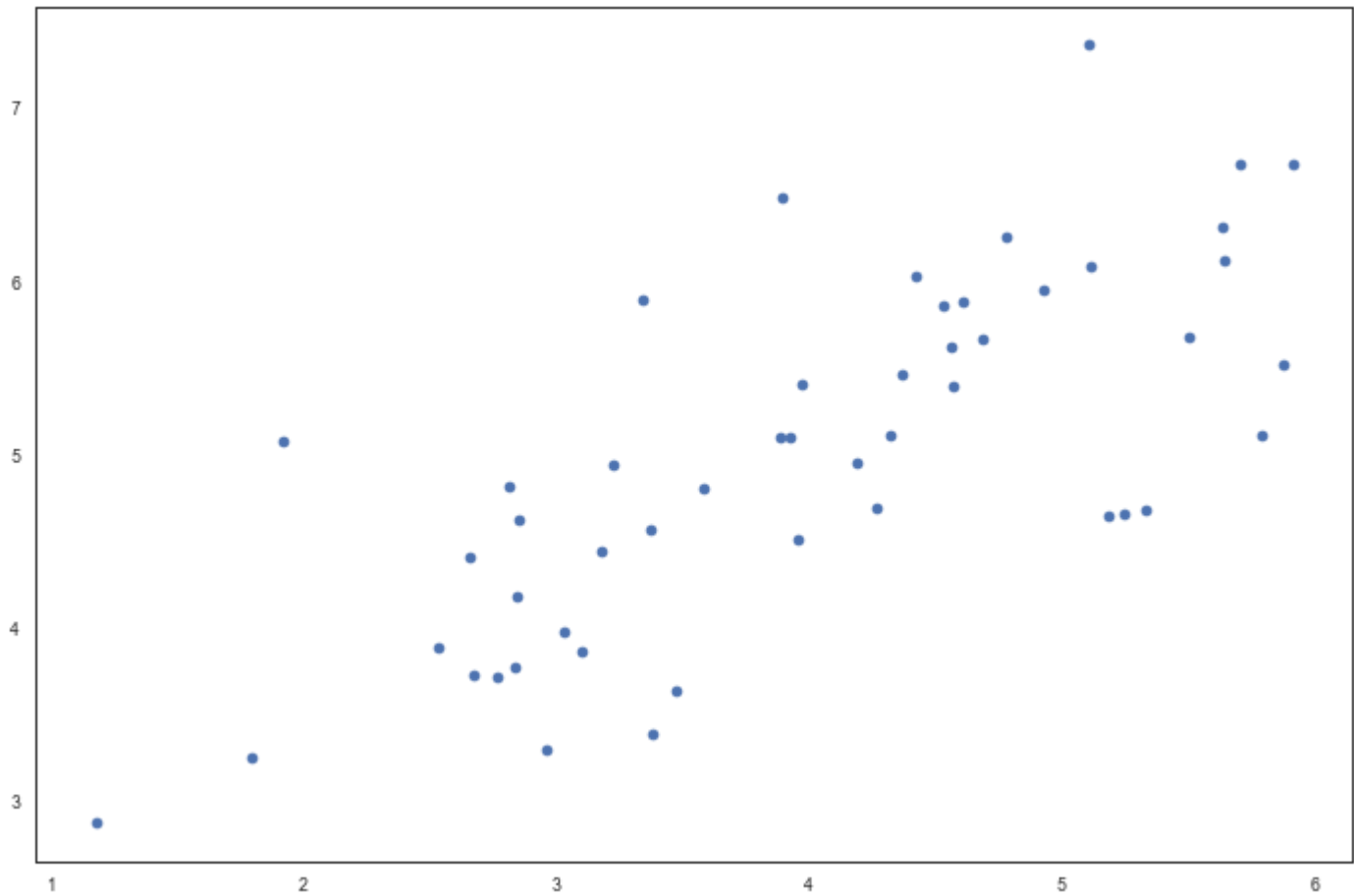
首先实现PCA并将其应用于一个简单的二维数据集，以了解它是如何工作的。从加载和可视化数据集开始。

```
In [26]: data = loadmat('data/machine-learning-ex7/ex7/ex7data1.mat')
data
```

```
Out[26]: {'__header__': b'MATLAB 5.0 MAT-file, Platform: PCWIN64, Created on: Mon Nov 14 22:41:44 2011',
__version__': '1.0',
__globals__': [],
'X': array([[3.38156267, 3.38911268],
[4.52787538, 5.8541781 ],
[2.65568187, 4.41199472],
[2.76523467, 3.71541365],
[2.84656011, 4.17550645],
[3.89067196, 6.48838087],
[3.47580524, 3.63284876],
[5.91129845, 6.68076853],
[3.92889397, 5.09844661],
[4.56183537, 5.62329929],
[4.57407171, 5.39765069],
[4.37173356, 5.46116549],
[4.19169388, 4.95469359],
[5.24408518, 4.66148767],
[2.8358402 , 3.76801716],
[5.63526969, 6.31211438],
[4.68632968, 5.6652411 ],
[2.85051337, 4.62645627],
[5.1101573 , 7.36319662],
[5.18256377, 4.64650909],
[5.70732809, 6.68103995],
[3.57968458, 4.80278074],
[5.63937773, 6.12043594],
[4.26346851, 4.68942896],
[2.53651693, 3.88449078],
[3.22382902, 4.94255585],
[4.92948801, 5.95501971],
[5.79295774, 5.10839305],
[2.81684824, 4.81895769],
[3.88882414, 5.10036564],
[3.34323419, 5.89301345],
[5.87973414, 5.52141664],
[3.10391912, 3.85710242],
[5.33150572, 4.68074235],
[3.37542687, 4.56537852],
[4.77667888, 6.25435039],
[2.6757463 , 3.73096988],
[5.50027665, 5.67948113],
[1.79709714, 3.24753885],
[4.3225147 , 5.11110472],
[4.42100445, 6.02563978],
[3.17929886, 4.43686032],
[3.03354125, 3.97879278],
[4.6093482 , 5.879792 ],
[2.96378859, 3.30024835],
[3.97176248, 5.40773735],
[1.18023321, 2.87869409],
[1.91895045, 5.07107848],
[3.95524687, 4.5053271 ],
[5.11795499, 6.08507386]]])}
```

```
In [27]: X = data['X']

fig, ax = plt.subplots(figsize=(12,8))
ax.scatter(X[:, 0], X[:, 1])
plt.show()
```



```
In [28]: #PCA的算法相当简单。 在确保数据被归一化之后，输出仅仅是原始数据的协方差矩阵的奇异值分解。
def pca(X):
    #均值归一化
    X = (X - X.mean()) / X.std()

    #计算协方差矩阵
    X = np.matrix(X)
    cov = (X.T * X) / X.shape[0]

    """
    奇异值分解SVD，U叫做左奇异值，S叫做奇异值，V叫做右奇异值。
    其中s是对矩阵a的奇异值分解。s除了对角元素不为0，其他元素都为0，并且对角元素从大到小排列。
    s中有n个奇异值，一般排在后面的比较接近0，所以仅保留比较大的r个奇异值。
    """

    U, S, V = np.linalg.svd(cov)

    return U, S, V
```

```
In [29]: U, S, V = pca(X)
U, S, V
```

```
Out[29]: (matrix([[ -0.79241747, -0.60997914],
[ -0.60997914, 0.79241747]]),
array([1.43584536, 0.56415464]),
matrix([[ -0.79241747, -0.60997914],
[ -0.60997914, 0.79241747]]))
```

```
In [30]: #现在我们有主成分（矩阵U），我们可以用这些来将原始数据投影到一个较低维的空间中。
#对于这个任务，我们将实现一个计算投影并且仅选择顶部K个分量的函数，有效地减少了维数。
def project_data(X, U, k):
    U_reduced = U[:, :k]
    return np.dot(X, U_reduced)
```

```
In [31]: Z = project_data(X, U, 1)
Z
```

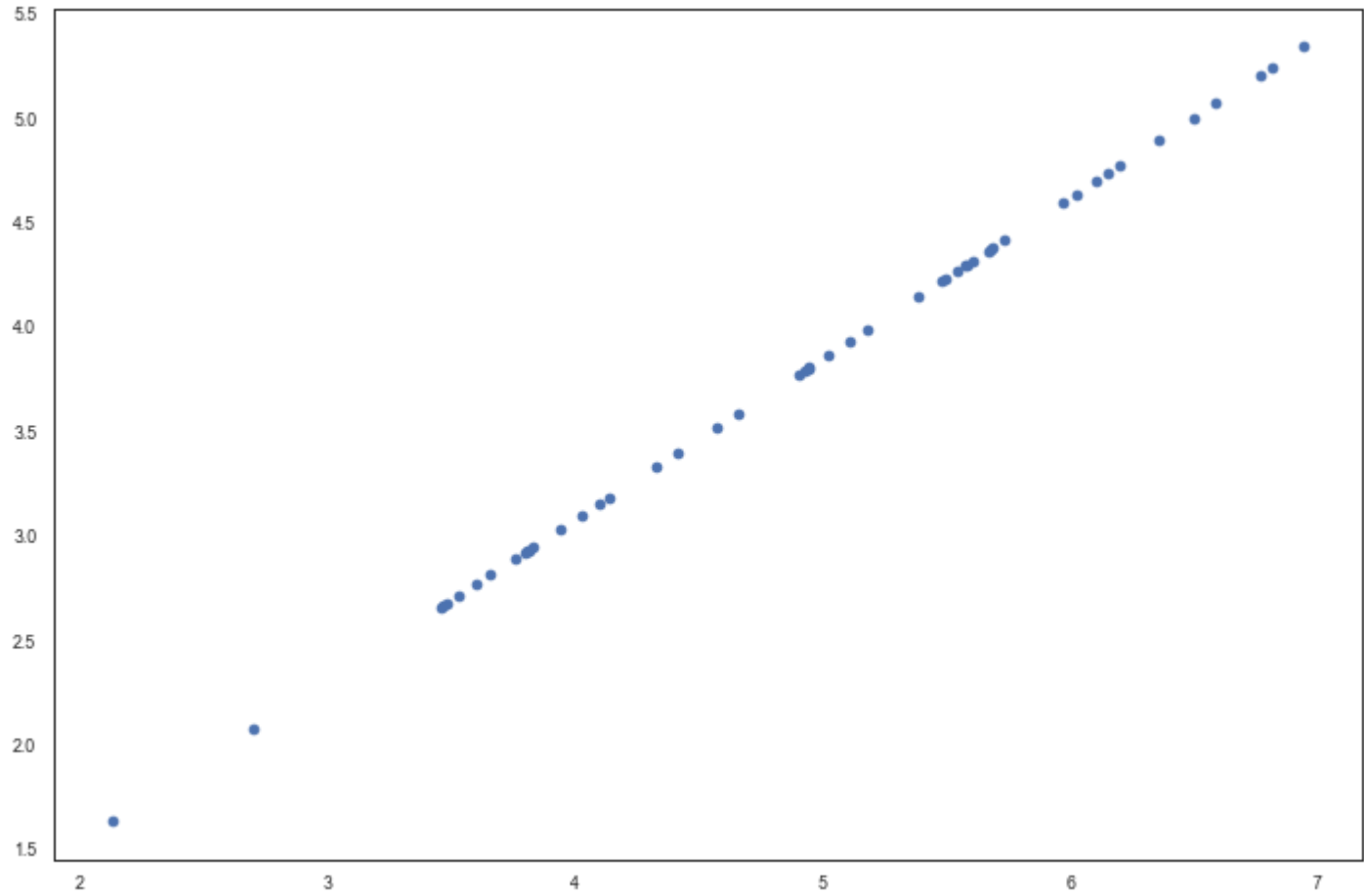
```
Out[31]: matrix([[ -4.74689738],
                [ -7.15889408],
                [ -4.79563345],
                [ -4.45754509],
                [ -4.80263579],
                [ -7.04081342],
                [ -4.97025076],
                [ -8.75934561],
                [ -6.2232703 ],
                [ -7.04497331],
                [ -6.91702866],
                [ -6.79543508],
                [ -6.3438312 ],
                [ -6.99891495],
                [ -4.54558119],
                [ -8.31574426],
                [ -7.16920841],
                [ -5.08083842],
                [ -8.54077427],
                [ -6.94102769],
                [ -8.5978815 ],
                [ -5.76620067],
                [ -8.2020797 ],
                [ -6.23890078],
                [ -4.37943868],
                [ -5.56947441],
                [ -7.53865023],
                [ -7.70645413],
                [ -5.17158343],
                [ -6.19268884],
                [ -6.24385246],
                [ -8.02715303],
                [ -4.81235176],
                [ -7.07993347],
                [ -5.45953289],
                [ -7.60014707],
                [ -4.39612191],
                [ -7.82288033],
                [ -3.40498213],
                [ -6.54290343],
                [ -7.17879573],
                [ -5.22572421],
                [ -4.83081168],
                [ -7.23907851],
                [ -4.36164051],
                [ -6.44590096],
                [ -2.69118076],
                [ -4.61386195],
                [ -5.88236227],
                [ -7.76732508]])
```

```
In [32]: #也可以通过反向转换步骤来恢复原始数据。
def recover_data(Z, U, k):
    U_reduced = U[:, :k]
    return np.dot(Z, U_reduced.T)
```

```
In [33]: X_recovered = recover_data(Z, U, 1)
X_recovered
```

```
Out[33]: matrix([[ 3.76152442,  2.89550838],
                [ 5.67283275,  4.36677606],
                [ 3.80014373,  2.92523637],
                [ 3.53223661,  2.71900952],
                [ 3.80569251,  2.92950765],
                [ 5.57926356,  4.29474931],
                [ 3.93851354,  3.03174929],
                [ 6.94105849,  5.3430181 ],
                [ 4.93142811,  3.79606507],
                [ 5.58255993,  4.29728676],
                [ 5.48117436,  4.21924319],
                [ 5.38482148,  4.14507365],
                [ 5.02696267,  3.8696047 ],
                [ 5.54606249,  4.26919213],
                [ 3.60199795,  2.77270971],
                [ 6.58954104,  5.07243054],
                [ 5.681006  ,  4.37306758],
                [ 4.02614513,  3.09920545],
                [ 6.76785875,  5.20969415],
                [ 5.50019161,  4.2338821 ],
                [ 6.81311151,  5.24452836],
                [ 4.56923815,  3.51726213],
                [ 6.49947125,  5.00309752],
                [ 4.94381398,  3.80559934],
                [ 3.47034372,  2.67136624],
                [ 4.41334883,  3.39726321],
                [ 5.97375815,  4.59841938],
                [ 6.10672889,  4.70077626],
                [ 4.09805306,  3.15455801],
                [ 4.90719483,  3.77741101],
                [ 4.94773778,  3.80861976],
                [ 6.36085631,  4.8963959 ],
                [ 3.81339161,  2.93543419],
                [ 5.61026298,  4.31861173],
                [ 4.32622924,  3.33020118],
                [ 6.02248932,  4.63593118],
                [ 3.48356381,  2.68154267],
                [ 6.19898705,  4.77179382],
                [ 2.69816733,  2.07696807],
                [ 5.18471099,  3.99103461],
                [ 5.68860316,  4.37891565],
                [ 4.14095516,  3.18758276],
                [ 3.82801958,  2.94669436],
                [ 5.73637229,  4.41568689],
                [ 3.45624014,  2.66050973],
                [ 5.10784454,  3.93186513],
                [ 2.13253865,  1.64156413],
                [ 3.65610482,  2.81435955],
                [ 4.66128664,  3.58811828],
                [ 6.1549641 ,  4.73790627]])
```

```
In [34]: fig, ax = plt.subplots(figsize=(12,8))
ax.scatter(list(X_recovered[:, 0]), list(X_recovered[:, 1]))
plt.show()
#第一主成分的投影轴基本上是数据集中的对角线，当将数据减少到一维时，失去了该对角线周围的变化，所以在再现中，一切都沿着该对角线。
```



将PCA应用于脸部图像。通过使用相同的降维技术，我们可以使用比原始图像少得多的数据来捕获图像的“本质”。

```
In [35]: faces = loadmat('data/machine-learning-ex7/ex7/ex7faces.mat')
X = faces['X']
X.shape

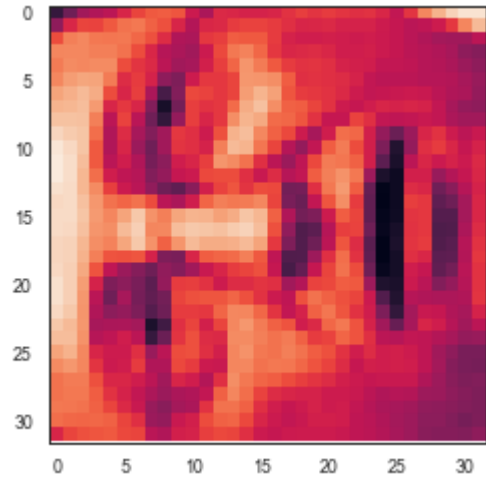
Out[35]: (5000, 1024)
```

```
In [36]: def plot_n_image(X, n): #plot first n images, n has to be a square number

    first_n_images = X[:n, :]
    fig, ax_array = plt.subplots(nrows=grid_size, ncols=grid_size,sharey=True, sharex=True, figsize=(8, 8))

    for r in range(grid_size):
        for c in range(grid_size):
            ax_array[r, c].imshow(first_n_images[grid_size * r + c].reshape((pic_size, pic_size)))
            plt.xticks(np.array([]))
            plt.yticks(np.array([]))
```

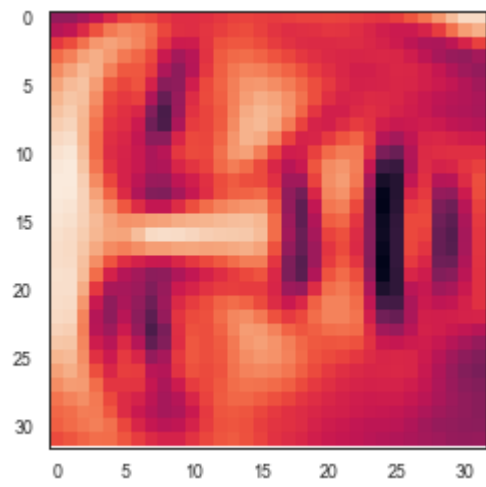
```
In [37]: face = np.reshape(X[3,:], (32, 32))
plt.imshow(face)
plt.show()
```



```
In [38]: #在面数据集上运行PCA，并取得前100个主要特征。
U, S, V = pca(X)
Z = project_data(X, U, 100)
Z
```

```
Out[38]: matrix([[ 526.09608833,  734.37008142,  194.48322788, ..., -19.27422565,
                  -3.22314155,  20.93551538],
                 [ 304.5906028 ,  493.0633805 , -162.10424193, ..., -20.94839919,
                  17.86358442,  -8.14045979],
                 [-389.99893833,  600.20010851, -293.91694459, ..., -27.86998851,
                  48.74829475,  17.98452065],
                 ...,
                 [ 487.55926046,  430.86037345,  490.71749378, ..., -31.76395627,
                  23.77770829,  51.74592358],
                 [1358.99575656,  402.85437502, -136.10305216, ..., -10.45305753,
                  -2.76084233,   2.96467067],
                 [ 372.01599145,  360.59923883,  105.10564415, ..., -48.29644614,
                  -8.75071522, -30.24094867]])
```

```
In [39]: #尝试恢复原来的结构并再次渲染
X_recovered = recover_data(Z, U, 100)
face = np.reshape(X_recovered[3,:], (32, 32))
plt.imshow(face)
plt.show()
```



1 Anomaly detection（异常检测）

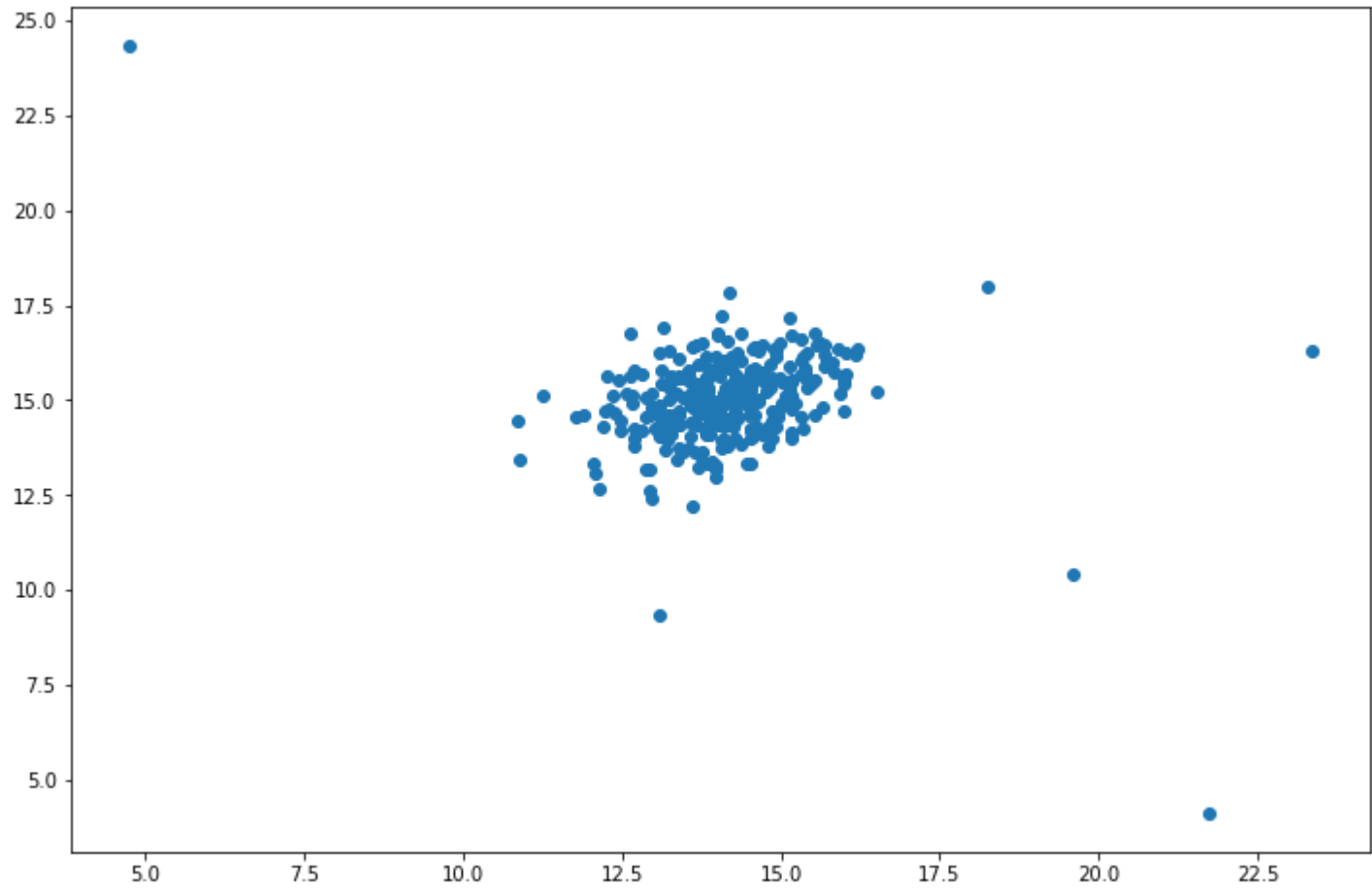
使用高斯模型来检测数据集中未标记的示例是否应被视为异常。先从简单的二维数据集开始。

```
In [1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sb
from scipy.io import loadmat
```

```
In [2]: data = loadmat('data/ex8data1.mat')
X = data['X']
X.shape
```

Out[2]: (307, 2)

```
In [3]: fig, ax = plt.subplots(figsize=(12,8))
ax.scatter(X[:,0], X[:,1])
plt.show()
```



```
In [4]: #输入一个X矩阵，输出2个n维的向量，mu包含了每一个维度的平均值，sigma2包含了每一个维度的方差。
def estimate_gaussian(X):
    mu = X.mean(axis=0)
    sigma = X.var(axis=0)

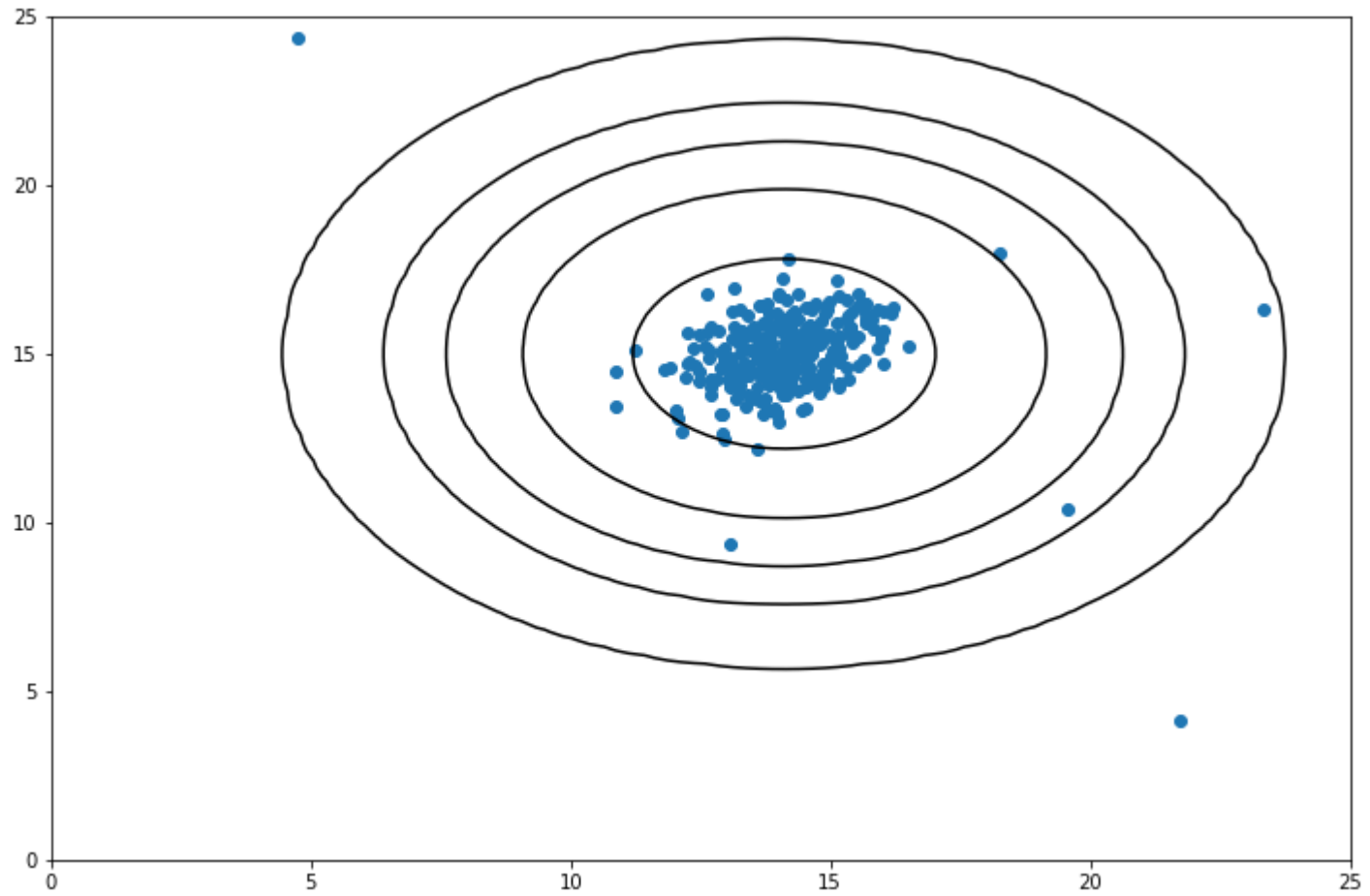
    return mu, sigma
```

```
In [5]: mu, sigma = estimate_gaussian(X)
mu, sigma
```

Out[5]: (array([14.11222578, 14.99771051]), array([1.83263141, 1.70974533]))

```
In [6]: xplot = np.linspace(0,25,100)
yplot = np.linspace(0,25,100)
Xplot, Yplot = np.meshgrid(xplot,yplot) #从坐标向量中返回坐标矩阵
Z = np.exp((-0.5)*((Xplot-mu[0])**2/sigma[0]+(Yplot-mu[1])**2/sigma[1])) #高斯分布公式

fig, ax = plt.subplots(figsize=(12,8))
contour = plt.contour(Xplot, Yplot, Z,[10**-11, 10**-7, 10**-5, 10**-3, 0.1],colors='k')
ax.scatter(X[:,0], X[:,1])
plt.show()
```



选择阈值ε

有了参数后，可以估计每组数据的概率，低概率的数据点更可能是异常的。确定异常点需要先确定一个阈值，我们可以通过验证集集来确定这个阈值

```
In [7]: Xval = data['Xval']
yval = data['yval']

Xval.shape, yval.shape
```

Out[7]: ((307, 2), (307, 1))

```
In [8]: #SciPy内置一种计算数据点属于正态分布的概率的方法。
from scipy import stats
dist = stats.norm(mu[0], sigma[0])
dist.pdf(15)
```

Out[8]: 0.1935875044615038

```
In [9]: #将数组传递给概率密度函数，并获得数据集中每个点的概率密度
dist.pdf(X[:,0])[0:50]

Out[9]: array([0.183842, 0.20221694, 0.21746136, 0.19778763, 0.20858956,
0.21652359, 0.16991291, 0.15123542, 0.1163989, 0.1594734,
0.21716057, 0.21760472, 0.20141857, 0.20157497, 0.21711385,
0.21758775, 0.21695576, 0.2138258, 0.21057069, 0.1173018,
0.20765108, 0.21717452, 0.19510663, 0.21702152, 0.17429399,
0.15413455, 0.21000109, 0.20223586, 0.21031898, 0.21313426,
0.16158946, 0.2170794, 0.17825767, 0.17414633, 0.1264951,
0.19723662, 0.14538809, 0.21766361, 0.21191386, 0.21729442,
0.21238912, 0.18799417, 0.21259798, 0.21752767, 0.20616968,
0.21520366, 0.1280081, 0.21768113, 0.21539967, 0.16913173])

In [10]: #计算并保存给定上述的高斯模型参数的数据集中每个值的概率密度。
p = np.zeros((X.shape[0], X.shape[1]))
p[:,0] = stats.norm(mu[0], sigma[0]).pdf(X[:,0])
p[:,1] = stats.norm(mu[1], sigma[1]).pdf(X[:,1])

p.shape

Out[10]: (307, 2)

In [11]: #为验证集（使用相同的模型参数）执行此操作。 使用与真实标签组合的这些概率来确定将数据点分配为异常的最佳概率阈值。
pval = np.zeros((Xval.shape[0], Xval.shape[1]))
pval[:,0] = stats.norm(mu[0], sigma[0]).pdf(Xval[:,0])
pval[:,1] = stats.norm(mu[1], sigma[1]).pdf(Xval[:,1])

pval.shape

Out[11]: (307, 2)

In [12]: #为找到给定概率密度值和真实标签的最佳阈值，给不同的epsilon值计算F1分数。
def select_threshold(pval, yval):
    best_epsilon = 0
    best_f1 = 0
    f1 = 0

    step = (pval.max() - pval.min()) / 1000

    for epsilon in np.arange(pval.min(), pval.max(), step): #返回一个有终点和起点的固定步长的排列
        preds = pval < epsilon

        tp = np.sum(np.logical_and(preds == 1, yval == 1)).astype(float) #np.logical_and: 逻辑与
        fp = np.sum(np.logical_and(preds == 1, yval == 0)).astype(float)
        fn = np.sum(np.logical_and(preds == 0, yval == 1)).astype(float)

        precision = tp / (tp + fp)
        recall = tp / (tp + fn)
        f1 = (2 * precision * recall) / (precision + recall)

        if f1 > best_f1:
            best_f1 = f1
            best_epsilon = epsilon

    return best_epsilon, best_f1

In [13]: epsilon, f1 = select_threshold(pval, yval)
epsilon, f1

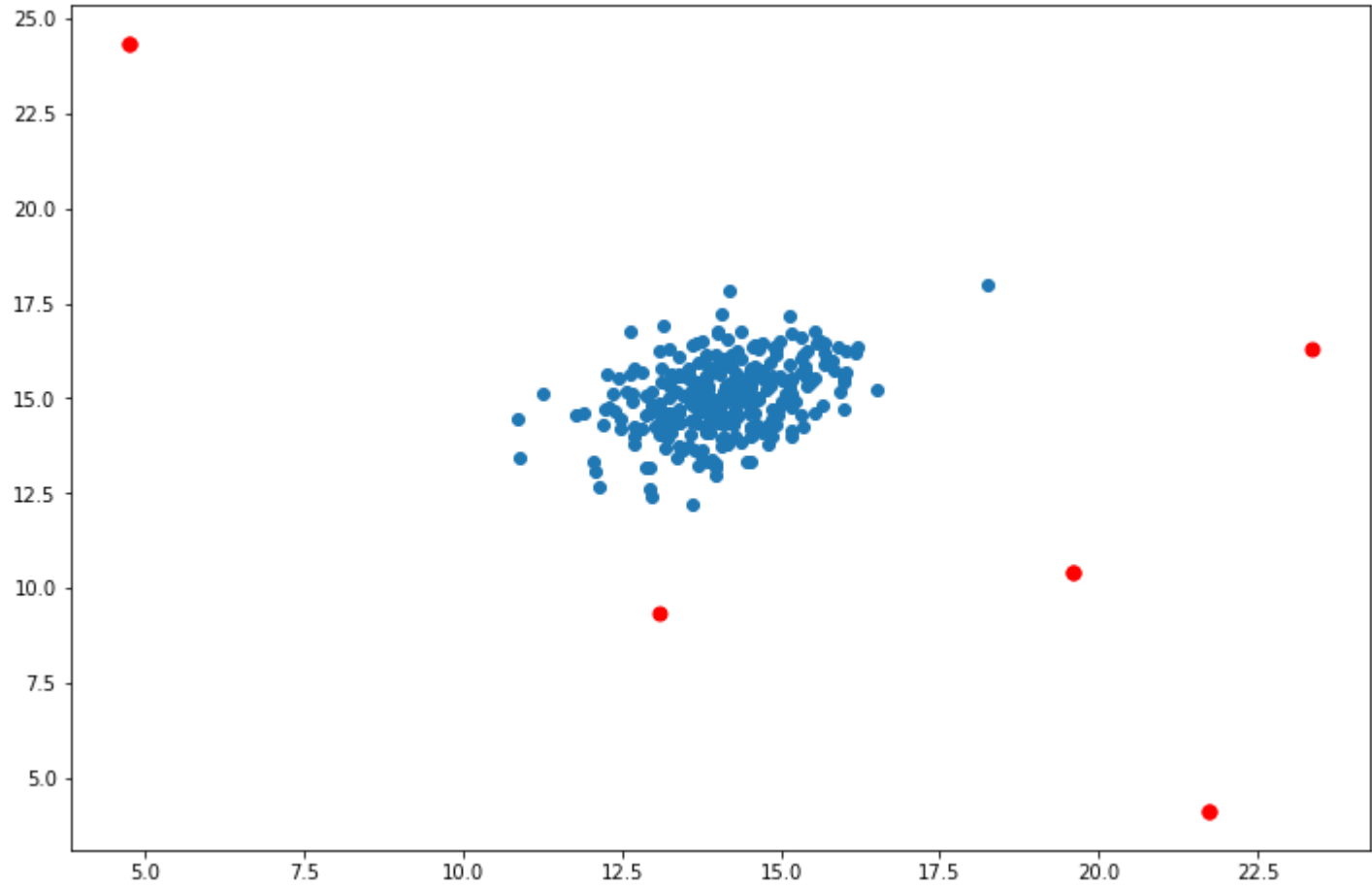
<ipython-input-12-616f3c672ce8>:16: RuntimeWarning: invalid value encountered in double_scalars
precision = tp / (tp + fp)

Out[13]: (0.009566706005956842, 0.7142857142857143)

In [14]: #将阈值应用于数据集，并可视化结果。
outliers = np.where(p < epsilon)
outliers

Out[14]: (array([300, 301, 301, 303, 303, 304, 306, 306], dtype=int64),
array([1, 0, 1, 0, 1, 0, 0, 1], dtype=int64))

In [15]: fig, ax = plt.subplots(figsize=(12,8))
ax.scatter(X[:,0], X[:,1])
ax.scatter(X[outliers[0],0], X[outliers[0],1], s=50, color='r', marker='o')
plt.show()
#红点是被标记为异常值的点
```



推荐系统：协同过滤

推荐引擎使用基于项目和用户的相似性度量来检查用户的历史偏好，以便为用户可能感兴趣的新“事物”提供建议。

将实现一种称为协作过滤的特定推荐系统算法，并将其应用于 电影评分的数据集。

```
In [16]: data = loadmat('data/ex8_movies.mat')
data

#Y是包含从1到5的等级的数组.R是包含指示用户是否给电影评分的二进制值的“指示符”数组。 两者应该具有相同的维度。

Out[16]: {'__header__': b'MATLAB 5.0 MAT-file, Platform: GLNXA64, Created on: Thu Dec 1 17:19:26 2011',
 '__version__': '1.0',
 '__globals__': [],
 'Y': array([[5, 4, 0, ..., 5, 0, 0],
 [3, 0, 0, ..., 0, 0, 5],
```

```

[4, 0, 0, ..., 0, 0, 0],
...,
[0, 0, 0, ..., 0, 0, 0],
[0, 0, 0, ..., 0, 0, 0],
[0, 0, 0, ..., 0, 0, 0]], dtype=uint8),
'R': array([[1, 1, 0, ..., 1, 0, 0],
[1, 0, 0, ..., 0, 0, 1],
[1, 0, 0, ..., 0, 0, 0],
...,
[0, 0, 0, ..., 0, 0, 0],
[0, 0, 0, ..., 0, 0, 0],
[0, 0, 0, ..., 0, 0, 0]], dtype=uint8)}
```

In [17]:

```
Y = data['Y']
R = data['R']
Y.shape, R.shape
```

Out[17]: ((1682, 943), (1682, 943))

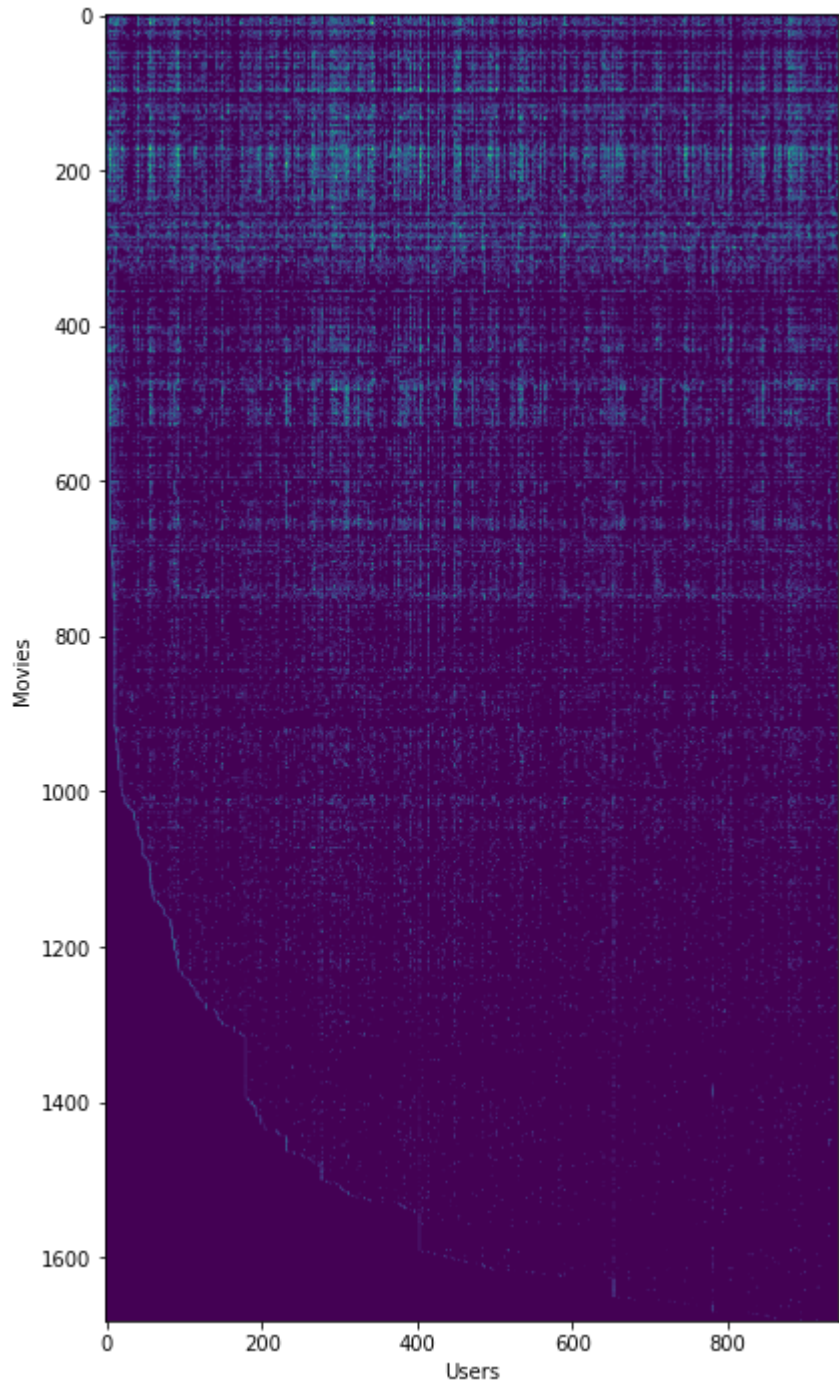
In [18]:

```
#通过平均排序Y来评估电影的平均评级
Y[1,np.where(R[1,:]==1)[0]].mean()
```

Out[18]: 3.2061068702290076

In [19]:

```
#将矩阵渲染成图像来尝试“可视化”数据。
ig, ax = plt.subplots(figsize=(12,12))
ax.imshow(Y)
ax.set_xlabel('Users')
ax.set_ylabel('Movies')
fig.tight_layout()
plt.show()
```



In [20]:

```
#序列化两个矩阵
def serialize(X, theta):

    # X (movie, feature), (1682, 10): movie features
    # theta (user, feature), (943, 10): user preference

    return np.concatenate((X.ravel(), theta.ravel())) #多个数组的拼接

#逆序列化
def deserialize(param, n_movie, n_user, n_features):
    return param[:n_movie * n_features].reshape(n_movie, n_features), param[n_movie * n_features:].reshape(n_user, n_features)

# recommendation fn
def cost(param, Y, R, n_features,l=1):

    """compute cost for every r(i, j)=1
    Args:
        param: serialized X, theta
        Y (movie, user), (1682, 943): (movie, user) rating
        R (movie, user), (1682, 943): (movie, user) has rating
    """

    # theta (user, feature), (943, 10): user preference
    # X (movie, feature), (1682, 10): movie features
    n_movie, n_user = Y.shape
    X, theta = deserialize(param, n_movie, n_user, n_features)

    inner = np.multiply(X @ theta.T - Y, R)

    reg_term = np.power(param, 2).sum() * (1 / 2)
    c=np. power(inner, 2).sum() / 2
    return c+reg_term
```

In [21]:

```
params_data = loadmat('data/ex8_movieParams.mat')
X = params_data['X']
theta = params_data['Theta']
X.shape, theta.shape
```

Out[21]: ((1682, 10), (943, 10))

In [22]:

```
#为评估时间的少点，只看一小段数据。
users = 4
movies = 5
features = 3

X_sub = X[:movies, :features]
theta_sub = theta[:users, :features]
Y_sub = Y[:movies, :users]
R_sub = R[:movies, :users]

param_sub = serialize(X_sub, theta_sub)
```

```
cost(param_sub, Y_sub, R_sub, features,l=1)
```

Out[22]: 28.304238738078038

```
In [23]: param = serialize(X, theta) # total real params
cost(param_sub, Y_sub, R_sub, features, l=1.5)
```

Out[23]: 31.34405624427422

```
In [24]: cost(param, Y, R, 10, l=1) # total regularized cost
```

Out[24]: 32520.682450229557

gradient

接下来需要实现梯度计算。 像在ex4中使用神经网络实现一样，扩展代价函数来计算梯度

```
In [25]: def gradient(param, Y, R, n_features,l=1):

    # theta (user, feature), (943, 10): user preference
    # X (movie, feature), (1682, 10): movie features
    n_movies, n_user = Y.shape
    X, theta = deserialize(param, n_movies, n_user, n_features)

    inner = np.multiply(X @ theta.T - Y, R) # (1682, 943)

    # X_grad (1682, 10)
    X_grad = inner @ theta #@: 矩阵乘法

    # theta_grad (943, 10)
    theta_grad = inner.T @ X

    # roll them together and return
    reg_term = np.power(param, 2).sum() * (1 / 2)
    g=serialize(X_grad, theta_grad)

    return g + reg_term
```

```
In [26]: n_movie, n_user = Y.shape

X_grad, theta_grad = deserialize(gradient(param, Y, R, 10),n_movie, n_user, 10)
X_grad, theta_grad
```

Out[26]: (array([[4595.78048424, 4604.50168614, 4595.1667224 , ..., 4597.22620673,
4605.8857409 , 4600.15445872],
[4598.23301123, 4603.84726823, 4599.40354614, ..., 4598.48652511,
4604.21327418, 4604.69361601],
[4598.91142452, 4604.59086529, 4602.28117147, ..., 4597.85454049,
4605.14770863, 4607.51556177],
...,
[4600.99458398, 4603.03453344, 4601.55311669, ..., 4601.28890423,
4602.36839891, 4601.15178932],
[4601.25811389, 4602.80369429, 4600.78618126, ..., 4600.9918476 ,
4603.68138003, 4601.89340607],
[4601.65440274, 4603.1065851 , 4601.69885504, ..., 4599.99319685,
4603.41831424, 4602.2378424]]),
array([[4600.49503691, 4611.12356038, 4601.40014208, ..., 4598.12197248,
4607.70651317, 4603.20698174],
[4599.45402655, 4604.56574904, 4600.51829864, ..., 4596.57439078,
4607.86712465, 4603.92731109],
[4604.18821468, 4604.05122147, 4597.72041856, ..., 4595.20866887,
4603.83184632, 4602.87119356],
...,
[4597.44415748, 4605.68190958, 4599.51323473, ..., 4598.53346561,
4605.04092135, 4602.69164745],
[4597.64579889, 4602.59268931, 4600.05780764, ..., 4595.29508867,
4605.89620318, 4605.99134305],
[4598.29038842, 4603.48626454, 4596.34979238, ..., 4595.48158823,
4607.24691757, 4604.6923652]]))

```
In [27]: #创建自己的电影评分，以便我们可以使用该模型来生成个性化的推荐。提供一个连接电影索引到其标题的文件，接着我们将文件加载到字典中。

movie_list = []
f = open('data/movie_ids.txt',encoding= 'ISO-8859-1')

for line in f:
    tokens = line.strip().split(' ')
    movie_list.append(' '.join(tokens[1:]))

movie_list = np.array(movie_list)
```

```
In [28]: movie_list[0]
```

Out[28]: 'Toy Story (1995)'

```
In [29]: ratings = np.zeros((1682, 1))

ratings[0] = 4
ratings[6] = 3
ratings[11] = 5
ratings[53] = 4
ratings[63] = 5
ratings[65] = 3
ratings[68] = 5
ratings[97] = 2
ratings[182] = 4
ratings[225] = 5
ratings[354] = 5

ratings.shape
```

Out[29]: (1682, 1)

```
In [30]: #将自己的评级向量添加到现有数据集中以包含在模型中。
Y = data['Y']
Y = np.append(ratings,Y, axis=1) # now I become user, append()函数用于合并两个数组
print(Y.shape)

(1682, 944)
```

```
In [31]: R = data['R']
R = np.append( ratings != 0, R,axis=1)
R.shape

Out[31]: (1682, 944)
```

```
In [32]: #不只是训练协同过滤模型。只需要定义一些变量并对评级进行规一化。
movies = Y.shape[0] # 1682
users = Y.shape[1] # 944
features = 10
learning_rate = 10.
```

```
X = np.random.random(size=(movies, features))
theta = np.random.random(size=(users, features))
params = serialize(X, theta)

X.shape, theta.shape, params.shape
```

Out[32]: ((1682, 10), (944, 10), (26260,))

```
In [33]: Y_norm = Y - Y.mean()
Y_norm.mean()
```

Out[33]: 4.6862111343939375e-17

```
In [34]: from scipy.optimize import minimize

fmin = minimize(fun=cost, x0=params, args=(Y_norm, R, features, learning_rate),
               method='TNC', jac=gradient)

fmin
```

Out[34]: fun: 164354.45710054896
jac: array([43657.25288838, 43621.38324232, 43647.9193398 , ...,
43796.52426922, 43807.59661344, 43814.97972157])
message: 'Converged (|f_n-f_(n-1)| ^= 0)'
nfev: 50
nit: 1
status: 1
success: True
x: array([0.82354806, 0.10065069, 0.16825318, ..., 0.15231596, 0.65069116,
0.67262395])

```
In [35]: #训练好的参数是X和Theta。 可以使用这些来为添加的用户创建一些建议。
X_trained, theta_trained = deserialize(fmin.x, movies, users, features)
X_trained.shape, theta_trained.shape
```

Out[35]: ((1682, 10), (944, 10))

```
In [36]: #最后，使用训练出的数据给出推荐电影
prediction = X_trained @ theta_trained.T
my_preds = prediction[:, 0] + Y.mean()
idx = np.argsort(my_preds)[:10] # Descending order
idx.shape
```

Out[36]: (1682,)

```
In [37]: # top ten idx
my_preds[idx][:10]
```

Out[37]: array([3.40845367, 3.35233029, 3.3353386 , 3.3285559 , 3.28978316,
3.25244248, 3.22523844, 3.21240938, 3.21019452, 3.18948874])

```
In [38]: for m in movie_list[idx][:10]:
print(m)
```

Walkabout (1971)
Maximum Risk (1996)
Primary Colors (1998)
Sleeper (1973)
Highlander III: The Sorcerer (1994)
Fire Down Below (1997)
Usual Suspects, The (1995)
Here Comes Cookie (1935)
Lay of the Land, The (1997)
Portrait of a Lady, The (1996)